

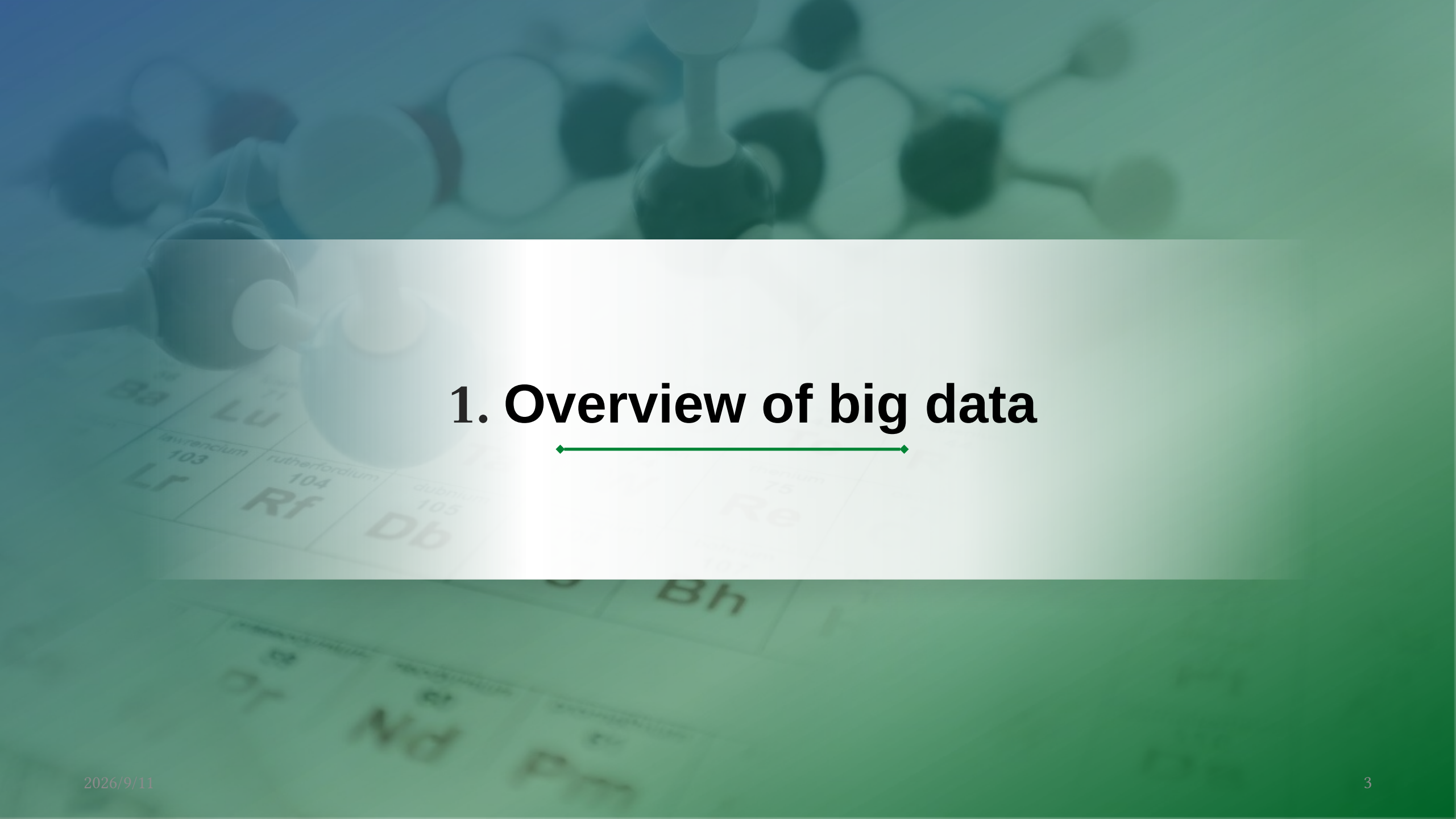


Chapter 2

Data collection preprocessing and visualization

Contents

- 1. Overview of big data**
- 2. Introduction of Python**
- 3. Data collection**
- 4. Data preprocessing**
- 5. Data visualization**



1. Overview of big data

Why big data?



◆ Era of big data

In recent years, with the rapid development and widespread application of computers and information technology, the scale of industry application systems has expanded rapidly, leading to an explosive growth in data generated by these applications.



Hundreds of **terabytes (TB)** or even tens to hundreds of **petabytes (PB)** in scale



Seek effective big data processing technologies, methods, and means.....

Why big data?



- The term "Big Data" was first introduced by McKinsey & Company in 2006. ()

Informatization	Time	Landmark events	Solving problems	Businesses
First wave	1980s	Personal computer	Information processing	Intel, AMD, IBM, apple, Microsoft, Lenovo, Dell, hp, etc.
Second wave	1995	Internet	Information transformation	Yahoo, Google, Alibaba, Baidu, Tencent, etc.
Third wave	2010	LOT, Cloud, big data	Information explosion	ByteDance, Facebook, etc.

Every wave of informatization takes a new wave of productivity growth and consumer surplus!

We need know it!



Why big data?



◆ . - 《 》

◆ Gartner

◆ McKinsey

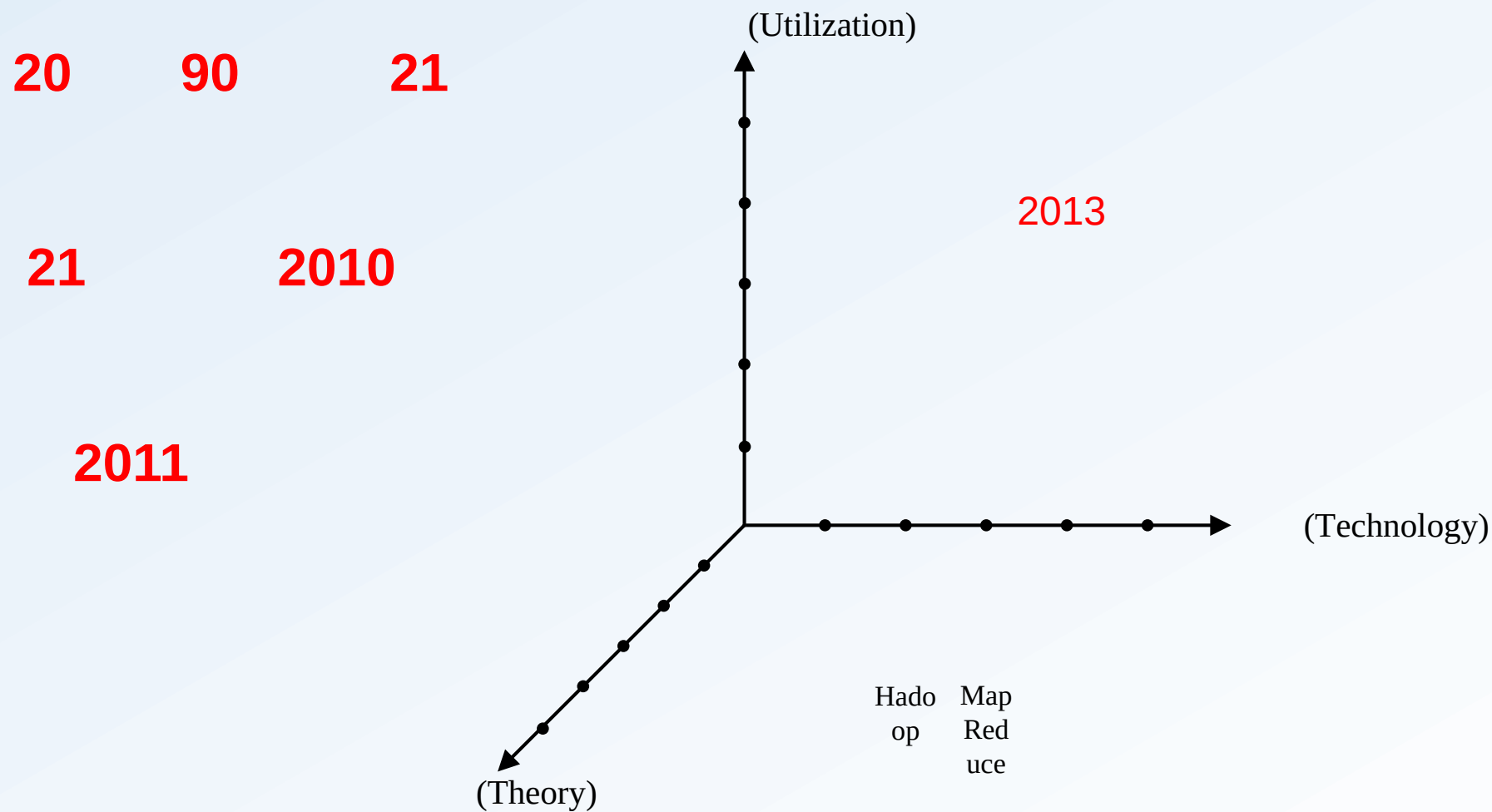
◆ (big data)

Value of data

Basis of Intelligent society

Digitization of social resources

Why big data?



Characteristics of big data



1. Volume()



2. Variety

3. Velocity

“1”

4. Value

5. Veracity

Techniques for big data



1.

2.

3.



4.

5.

6.

Thoughts for big data





1.



“ ”

“ ”



“ ”

“ ”



2.



3.



4.





1.

2.

3.

4.

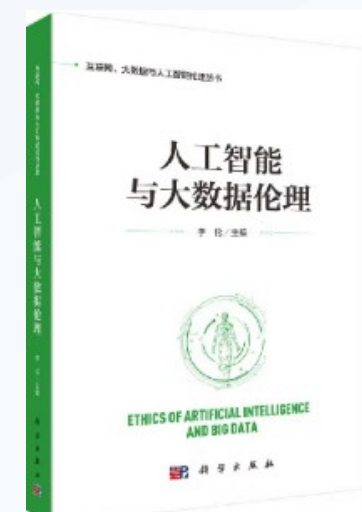
1.

2.

3.

4.

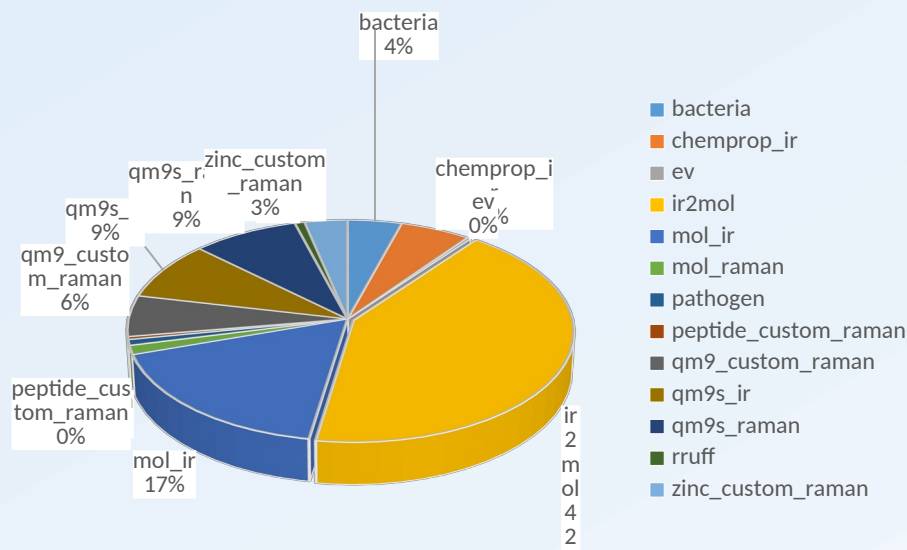
5.



Safety of big data for science



◆ We lost several chances and data.



bacteria	66000
chemprop_ir	85506
ev	4413
ir2mol	634583
mol_ir	261105
mol_raman	19908
pathogen	12678
peptide_custom_raman	6324
qm9_custom_raman	90867
qm9s_ir	129817
qm9s_raman	129817
rruff	12048
zinc_custom_raman	51968
	1505034

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.





OpenClaw Exposure Watchboard

This page lists publicly reachable active OpenClaw instances for defensive awareness. If this is your deployment, enable authentication, remove direct public exposure, and patch immediately.

Exposed Instances: 335624

Page: 1 / 3357 (100 per page)

Showing: 1-100

Last Imported: 11/03/2026, 03:56:01

BUILD WITH VIVGRID

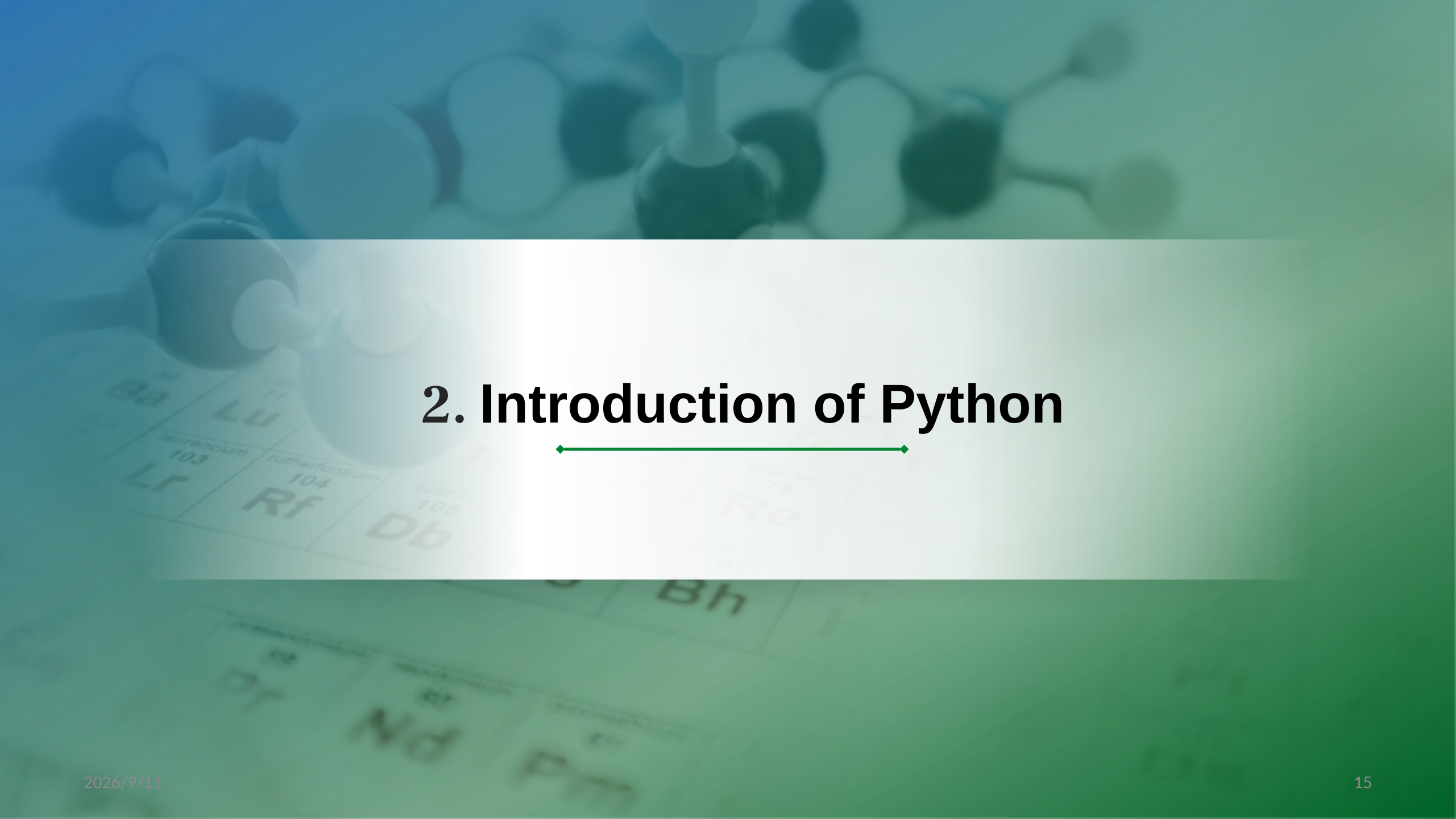
Ship Secure Enterprise AI Agents 10x Faster with vivgrid.com

Vivgrid gives you authentication, model gateway, tool control, cost tracking, and enterprise observability — everything you need to ship AI agents safely at scale.

Explore Vivgrid

Showing page 1 of 3357														Previous	Next
ENDPOINT	ASSISTANT NAME	COUNTRY	AUTH_REQUIRED	IS_ACTIVE	HAS_LEAKED_CREDS	ASN	ASN_NAME	ORG	FIRST_SEEN	LAST_SEEN	ASL_HAS_BREACH	ASL_HAS_THREAT_ACTOR	ASL_THREAT_ACTORS		
184.32.150.***:18789	-	US United States	-	true	Clean	AS16509	Amazon.com, Inc.	Amazon	04/03/2026, 19:25:33	11/03/2026, 03:25:38	-	-	-		
147.139.192.***:18789	Assistant	CN China mainland	Yes	true	Clean	AS45102	Alibaba (US) Technology Co., Ltd.	Alibaba Cloud ID	05/02/2026, 09:17:17	11/03/2026, 03:25:38	NO	NO	-		
140.143.155.***:18789	-	CN China mainland	-	true	Clean	AS45098	Shenzhen Tencent Computer Systems Company Limited	Tencent Cloud	06/02/2026, 02:13:18	11/03/2026, 03:25:38	NO	NO	-		
8.222.194.***:18789	Assistant	SG Singapore	Yes	true	Clean	AS45102	Alibaba (US) Technology Co., Ltd.	Alibaba Cloud Singapore	26/02/2026, 18:49:48	11/03/2026, 03:25:38	NO	YES	APT28, APT35, APT37, APT39, Cobalt Group, Kimsuky, Mustang Panda, Sandworm Team, The Shadow Brokers		
35.90.73.***:18789	-	US United States	-	true	Clean	AS16509	Amazon.com, Inc.	Amazon	07/03/2026, 11:55:48	11/03/2026, 03:25:38	NO	NO	-		
47.187.181.***:18789	Assistant	CN China mainland	Yes	true	Clean	AS37963	Hangzhou Alibaba Advertising Co.,Ltd.	Alisoft	05/02/2026, 15:25:46	11/03/2026, 03:25:38	NO	YES	APT14, APT28, APT35, APT37, APT39, APT40, APT41, Cobalt Group, Equation Group, Gamaredon Group, IroelHusky, Kimsuky, Lazarus Group, Mustang Panda, Sandworm Team, SharpPanda, TASEB5, The Sh		
157.245.226.***:18789	-	US United States	-	true	Clean	AS14061	DigitalOcean, LLC	DigitalOcean	03/02/2026, 22:46:46	11/03/2026, 03:25:38	-	-	-		
165.232.91.***:18789	-	NL Netherlands	-	true	Clean	AS14061	DigitalOcean, LLC	DigitalOcean	05/02/2026, 05:32:27	11/03/2026, 03:25:37	NO	NO	-		
47.79.93.***:18789	-	CN China mainland	-	true	Leaked	AS45102	Alibaba (US) Technology Co., Ltd.	Alibaba Cloud	09/03/2026, 02:59:38	11/03/2026, 03:25:37	YES	NO	-		
188.166.3.***:18789	-	NL Netherlands	-	true	Clean	AS14061	DigitalOcean, LLC	DigitalOcean	16/02/2026, 04:36:16	11/03/2026, 03:25:37	NO	NO	-		
104.21.64.***:18789	-	US United States	-	true	Leaked	AS12355	Cloudflare, Inc.	CloudFlare	28/02/2026, 22:06:19	11/03/2026, 03:25:37	YES	NO	-		
143.118.235.***:18789	-	US United States	-	true	Clean	AS14061	DigitalOcean, LLC	DigitalOcean	08/02/2026, 08:57:35	11/03/2026, 03:25:37	NO	YES	APT15, APT17, APT28, APT29, APT31, APT34, APT35, APT36, APT37, APT40, APT41, APT45, Bitter APT, Bluenoroff, Callisto Group, Carbanak, Chame1Gang, CloudS0rcer3r, Dagg3rF1y APT, Donot Team		
89.167.188.***:18789	-	FI Finland	-	true	Clean	AS24948	Hetzner Online GmbH	Hetzner	07/03/2026, 08:16:41	11/03/2026, 03:25:37	-	-	-		
47.93.142.***:18789	Assistant	CN China mainland	Yes	true	Clean	AS37963	Hangzhou Alibaba Advertising Co.,Ltd.	Alisoft	05/02/2026, 05:14:48	11/03/2026, 03:25:37	NO	NO	-		
64.23.238.***:18789	Assistant	US United States	Yes	true	Clean	AS14061	DigitalOcean, LLC	DigitalOcean	14/02/2026, 18:57:05	11/03/2026, 03:25:37	NO	NO	-		
47.80.19.***:18789	Assistant	CN China mainland	Yes	true	Leaked	AS45102	Alibaba (US) Technology Co., Ltd.	Alibaba Cloud	03/02/2026, 22:46:41	11/03/2026, 03:25:37	YES	NO	-		
159.65.113.***:18789	-	SG Singapore	-	true	Clean	AS14061	DigitalOcean, LLC	DigitalOcean	18/03/2026, 21:53:15	11/03/2026, 03:25:37	NO	YES	APT-C-23, APT14, APT15, APT28, APT29, APT31, APT34, APT35, APT37, APT39, APT40, APT41, Bitter APT, Bluenoroff, Callisto Group, Cobalt Group, Donot Team, Equation Group, Gamaredon Group, I		
115.190.116.***:18789	-	CN China mainland	-	true	Clean	AS137718	Beijing Volcano Engine Technology Co., Ltd.	Beijing Volcano Engine Technology	18/03/2026, 21:53:15	11/03/2026, 03:25:37	NO	NO	-		
134.209.227.***:18789	-	DE Germany	-	true	Clean	AS14061	DigitalOcean, LLC	DigitalOcean	22/02/2026, 19:44:16	11/03/2026, 03:25:37	NO	YES	APT15, APT28, APT29, APT31, APT34, APT41, Bitter APT, Bluenoroff, Callisto Group, Donot Team, Gamaredon Group, Gaza Cybergang, Hafnium Group, Inception Framework, Kimsuky, MuddyWater Gro		
3.106.225.***:18789	-	AU Australia	-	true	Clean	AS16509	Amazon.com, Inc.	Amazon Corporate Services	05/03/2026, 03:46:12	11/03/2026, 03:25:37	-	-	-		
79.108.228.***:18789	-	SG Singapore	-	true	Clean	AS36007	Kawata, Inc.	CloudWebManage	09/03/2026, 14:26:32	11/03/2026, 03:25:37	NO	NO	-		
47.253.237.***:18789	Assistant	CN China mainland	Yes	true	Clean	AS45102	Alibaba (US) Technology Co., Ltd.	Alibaba Cloud US	03/02/2026, 22:47:17	11/03/2026, 03:25:37	-	-	-		

Openclaws are good, but use them with caution.



2. Introduction of Python



Introduction of Python

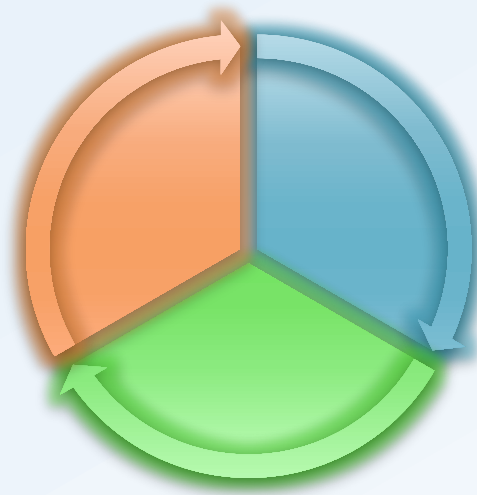
□ Python □ □ □

Python

Introduction of Python



- **Must-know**
- **Should-know**
- **Nice to know**



➤ Must-know

int			0 10 1234 -56789
float		308	3.1415926 5.0
str			'hello' ' '
list		5.3	[4.0, ' ', True]
tuple			(4.0, ' ', True)
dict		" " :	{ : ' ', :30}
set			{4.0, ' ', True}
bool		True False	a>b and b>c
bytes			b'\xe5\xa5\xbd'
complex			3+2.7j

➤ Must-know

数据属性	字符串	列表	元组	字典
数据定义	引号	方括号	圆括号或无括号	花括号
独立元素	独立符号	逗号分隔	逗号分隔	逗号分隔
索引号	有	有	有	无
序列操作	可以	可以	可以	无序, 不可
数据类型混用	不可, 仅字符	可以	可以	可以
元素修改	不可	可以	不可	不可
数据切片	可以, 生成新串	可以, 列表可改	可以, 生成新元组	不可
元素遍历	循环	循环	循环	循环

➤ Must-know

+	$a+b$	$a \ b$
-	$a-b$	$a \ b$
*	$a*b$	$a \ b$
/	a/b	$a \ b$
//	$a//b$	$a \ b$
**	$a**b$	$a \ b$
%	$a\%b$	$a \ b$
and	$a \text{ and } b$	
or	$a \text{ or } b$	
not	$\text{not } a$	

➤ Must-know

==	a==b	
!=	a!=b	
>	a>b	
<	a=	a>=b	
<=	a<=b	
&	a&b	
	a b	
^	a^b	
~	a~b	
<<	a<<b	a n 0
>>	a>>b	a n 0

➤ Must-know

=	a=c	c a
+=	a+=c	a=a+c
-+	a-=c	a=a-c
=	a=c	a=a*c
/=	a/=c	a=a/c
%=	a%=c	a=a%c
=	a=c	a=a**c
//=	a//=c	a=a//c

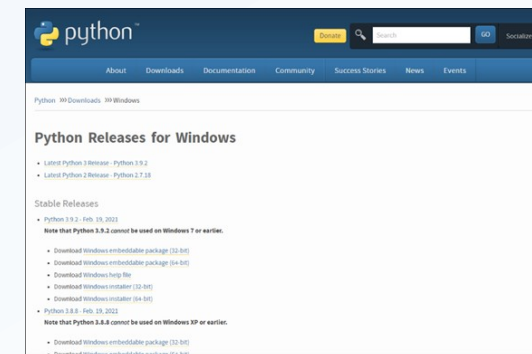
➤ Should-know

1. Python

- ◆ Windows Mac OS X Linux
- ◆ CPython IronPython Jython PyPy
- ◆ Python 2.x Python 3.x
- ◆ 32 64
- ◆ 32

2. Python

- ◆ 1 <https://www.python.org/>
- ◆ 2 “Downloads→Windows”
- ◆ 3 Python



➤ Should-know

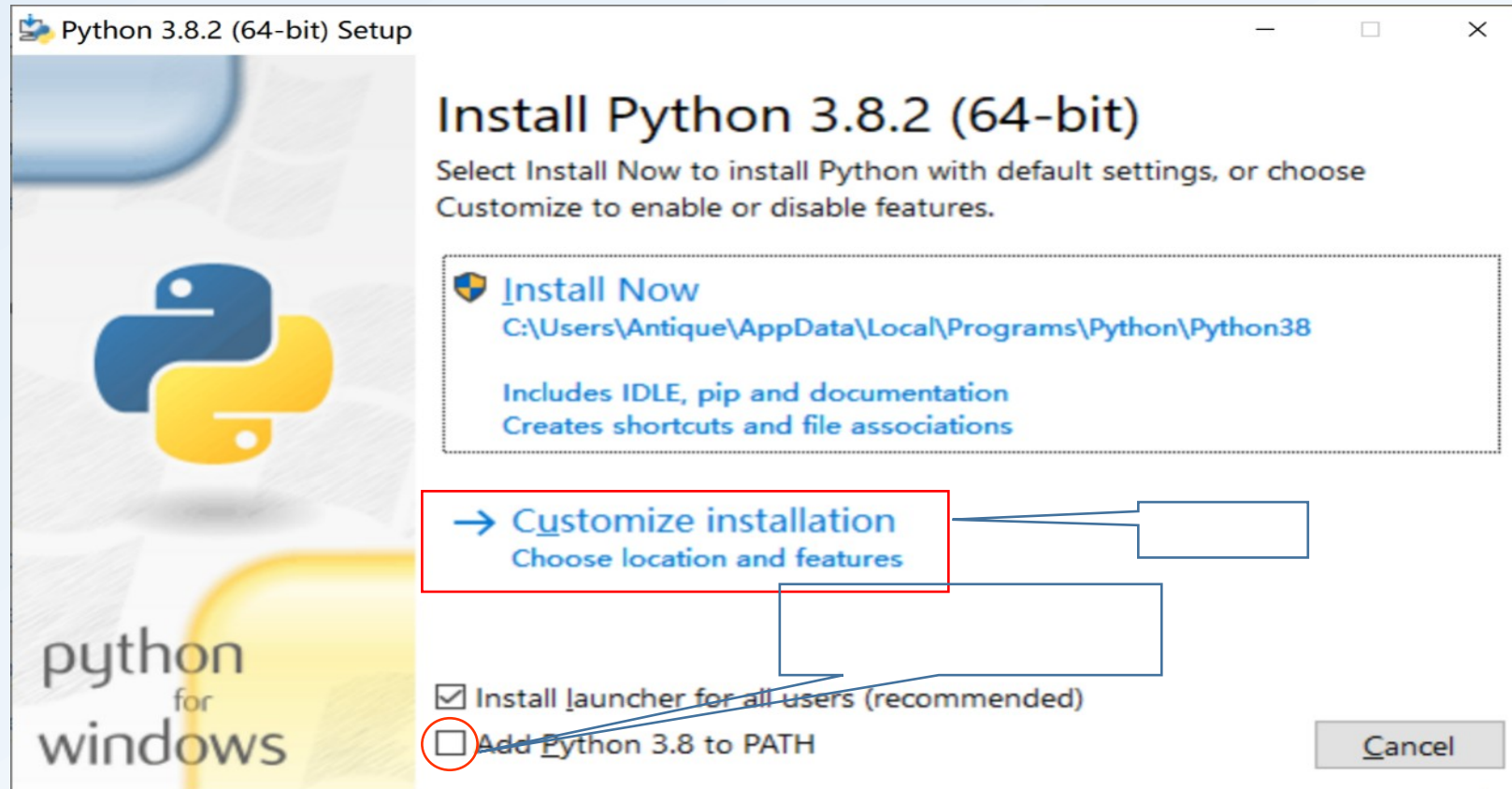
3. Python

- ◆ 1 Python-3.8.2.exe
- ◆ 2 “Add Python 3.8.2 to PATH”
- ◆ 3 “Customize Installation”
- ◆ 4 “Install for all users”
- ◆ “Customize install location” D:\Python38
- ◆ “Install”
- ◆ 5 “Disable path length limit”

Introduction of Python



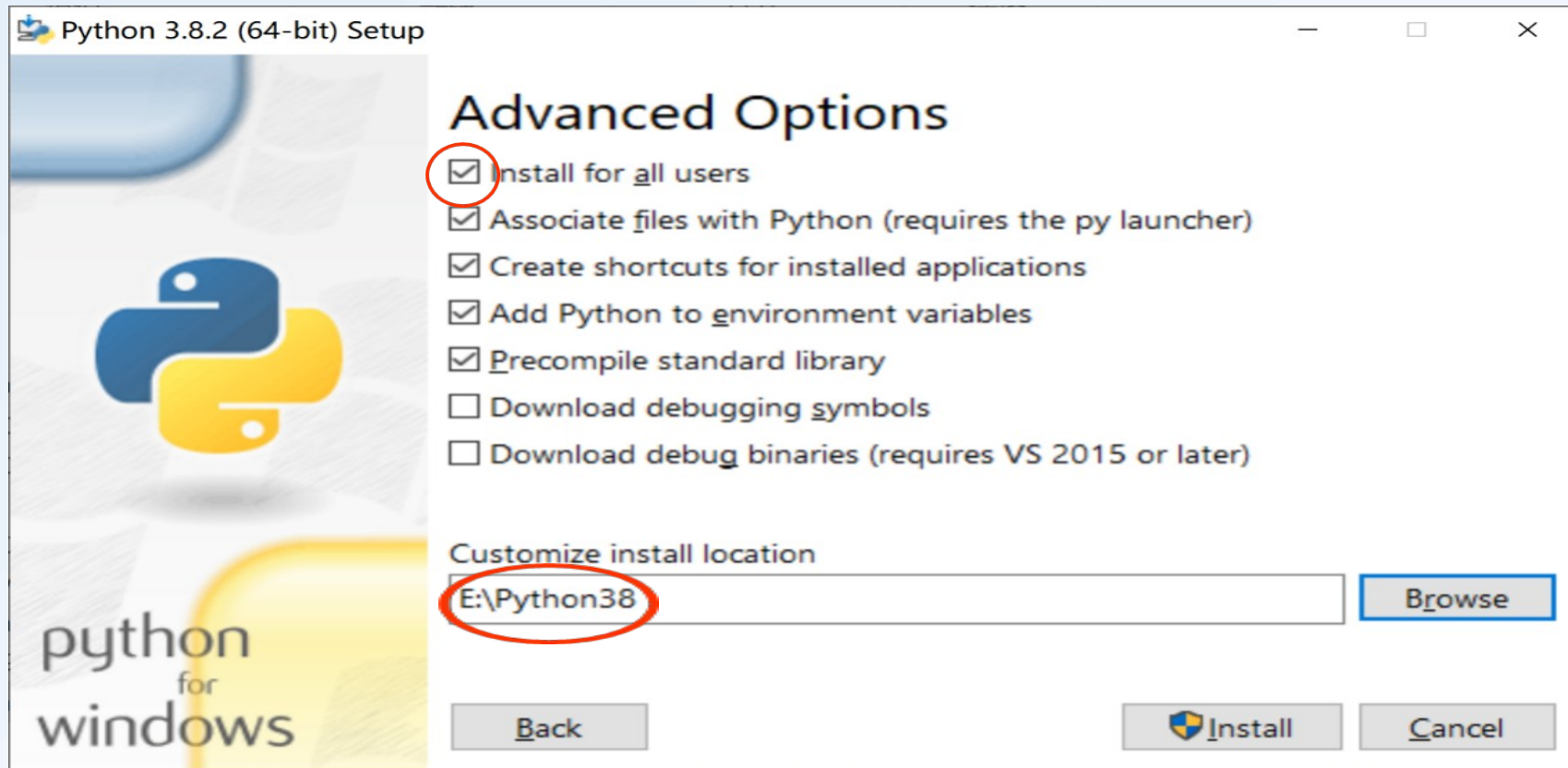
➤ Should-know



Introduction of Python



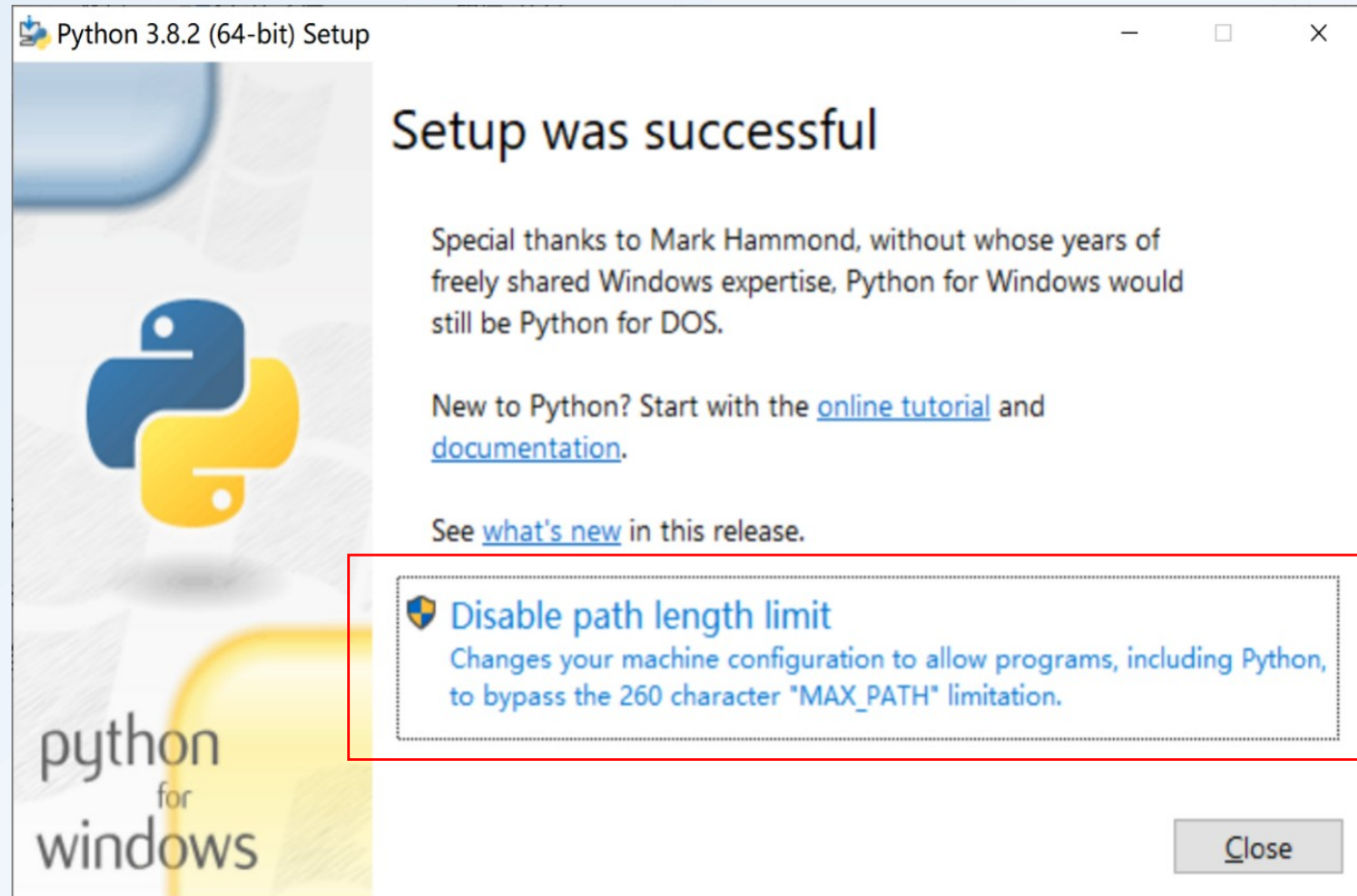
➤ Should-know



Introduction of Python



➤ Should-know



Introduction of Python



➤ Should-know

◆ Python

Python

Python

Python

◆ cmd

```
1 C:\Users\Administrator>cd [CR]      # d
2 D:\>cd python[CR]                  # Python
3 D:\Python>cd lib[CR]                # Lib
4 D:\Python\Lib>dir[CR]               #
.....                               #
5 D:\Python\Lib>cd.. [CR]             #
6 C:\>where python                    # Python
```

Introduction of Python



➤ Should-know

1.

◆ “ ”

◆ [Win+R] “ ” “cmd” → “ ”

◆ “ ” “Python” Python shell

◆ >>> Python

◆ shell

```
C:\Windows\system32\cmd.exe - python
Microsoft Windows [版本 10.0.17763.316]
(c) 2018 Microsoft Corporation。保留所有权利。

C:\Users\Antique>python
Python 3.8.2 (tags/v3.8.2:7b3ab59, Feb 25 2020, 23:03:10) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> print("Hello World")
Hello World
>>>
```


Introduction of Python



➤ Should-know

1.

- ◆ Windows “ ” “ Python 3.8 ” “ IDLE ” IDLE Python
- ◆ IDLE

```
Python 3.8.2 Shell
File Edit Shell Debug Options Window Help
Python 3.8.2 (tags/v3.8.2:7b3ab59, Feb 25 2020, 23:03:10) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> print("Hello World")
Hello World
>>>
```

Ln: 5 Col: 4

➤ Should-know 2.



Python

Windows

.py

“python XXX.py”

Python



```
C:\Windows\system32\cmd.exe
Microsoft Windows [版本 10.0.17763.316]
(c) 2018 Microsoft Corporation。保留所有权利。

C:\Users\Antique>cd /d d:\workspace

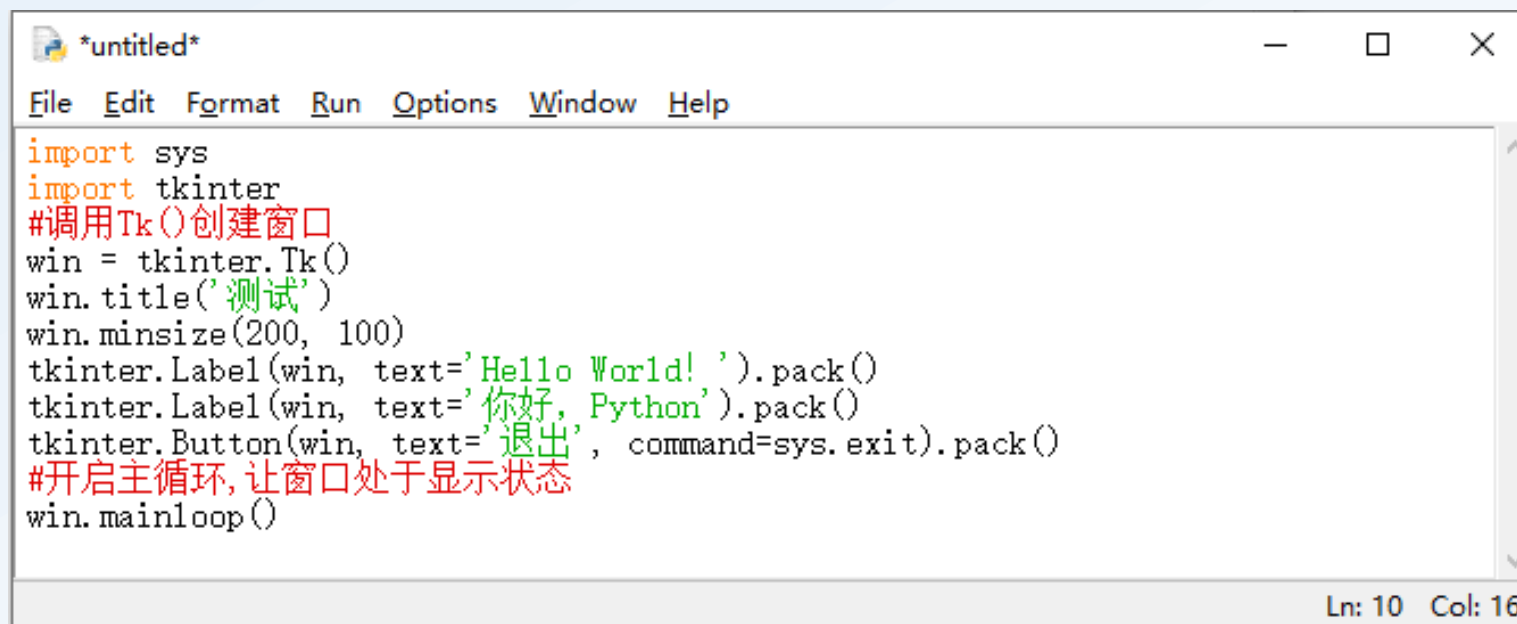
d:\workspace>python hello.py
Hello World

d:\workspace>
```

➤ Should-know

2.

◆ Windows “ ” “Python 3.8” “IDLE” IDLE **File → New File** Python

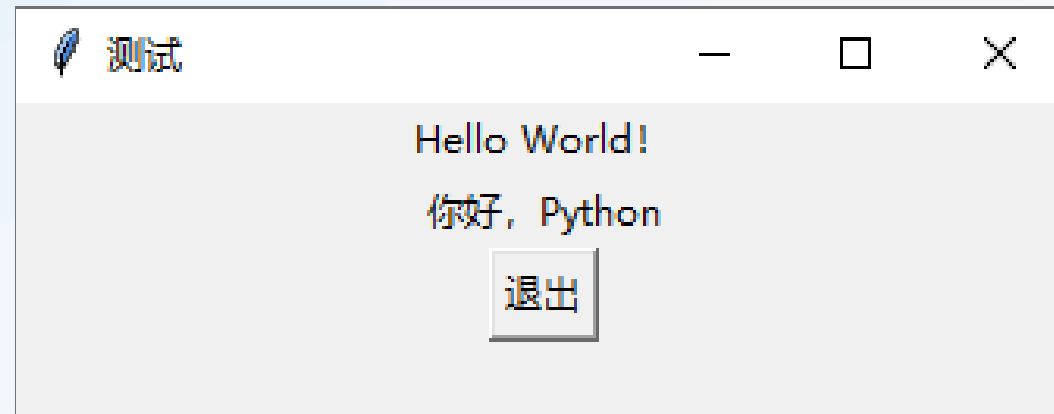


```
File Edit Format Run Options Window Help
import sys
import tkinter
#调用Tk()创建窗口
win = tkinter.Tk()
win.title('测试')
win.minsize(200, 100)
tkinter.Label(win, text='Hello World! ').pack()
tkinter.Label(win, text='你好, Python').pack()
tkinter.Button(win, text='退出', command=sys.exit).pack()
#开启主循环, 让窗口处于显示状态
win.mainloop()
```

Ln: 10 Col: 16

➤ Should-know

- ◆ “File” → “Save As” → “ ”
- ◆ “Run” → “Check Module”
- ◆ “Run” → “Run Module”



➤ Should-know

◆ pip



◆ Python

pip

pip3

pip3.x

➤ Should-know

命令格式 ↵	说·····明 ↵
<code>pip -help</code> ↵	查看 pip 帮助信息 ↵
<code>pip install 包名</code> ↵	从官方网站在线安装第三方软件包 ↵
<code>pip install -i 镜像网站名 包名</code> ↵	从镜像网站在线安装第三方软件包 ↵
<code>pip install 包名 -upgrade</code> ↵	从官方网站在线升级第三方软件包 ↵
<code>pip install -i 镜像网站名 包名 -U</code> ↵	从镜像网站在线升级第三方软件包 ↵
<code>pip uninstall 包名</code> ↵	卸载本地（本机）指定的第三方软件包 ↵
<code>pip show 包名</code> ↵	查看指定安装第三方软件包详细信息 ↵
<code>pip list</code> ↵	查看当前已安装的第三方软件包和版本号 ↵
<code>pip list --outdated</code> ↵	检查哪些第三方软件包需要更新 ↵
<code>pip -V</code> ↵	查看 pip 版本和安装目录（注意 V 大写） ↵

➤ Should-know 3.

“ ”→“ Windows ”→“ ” →

1	> pip install numpy	#	numpy
2	> pip list	#	
3	> pip uninstall numpy	#	numpy
4	> pip install numpy=1.18.2	#	numpy
5	> pip install numpy--upgrade	#	numpy
6	> pip install numpy -i https://pypi.tuna.tsinghua.edu.cn/simple	#	numpy
7	> pip list	#	

➤ Should-know

4.

◆ 1 Python pip

```
>python -m pip install --upgrade pip # pip
```

◆ 2 pip

```
>pip install -i https://pypi.tuna.tsinghua.edu.cn/simple pip -U # pip
```

◆ 1 -m= -i= simple

◆ 2 <https://mirrors.tuna.tsinghua.edu.cn/help/pypi/>

➤ Should-know

5.

◆ .whl

pip

◆ .gz Linux

WordCloud



1

WordCloud



cp37

Python 3.7

win32

Windows

32



wordcloud-1.6.0-cp37-cp37m-win32.whl



2

“

”

.whl



3

“pip install

.whl”



2.1.5 Python





1.



79

2.

1

2

3

() [] {}

4

\

5

6

2.1.5 Python



1

2

```
1 plt.pie(x = edu,  
2     labeldistance = 1.15,  
3     radius = 1.5,  
4     counterclock = False,  
5     frame = 1)
```

```
1 plt.pie(x = edu,\n2     labeldistance = 1.15, radius = 1.5,\n3     counterclock = False, frame = 1)
```

3.

1	4	Tab
2		4
3		

1	if x == 4:	1	if x == 4 :	# ":"
2	print(x, y)	2	print(x , y)	# ","
3	x, y = y, x	3	x , y = y , x	# ","

2.1.5 Python



4

1	if a == 0:	1	if a = = 0:	#	=
2	x += 1	2	x + = 1	# +	=

4.

1

2

3

5.

1

◆ " "

2

◆ `mylist = ['Google', ' ', 1919, 2022]`

3

◆ `greed = re.findall(r'(\d+)', 'a23b' )`

4

3

◆ `""" """`

6.

◆ Python

1

import

2

```
1 >>>import sys, time, random, math          #
2 >>>from pandas import Series, DataFrame      #
    “;”
```

```
1 >>>a=2; b=3                                #
2 >>>print(a); print(b); print(a+b)           #
    2 3 5                                     #
```

2.1.5 Python



3

```
1 >>>def _sety(self, value): self.rect.y = value      #
2 >>>a = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]           #
3 >>>print([j for i in a for j in i])                 #
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

4 lambda

```
1 >>>f = lambda x, y:x+y                             # lambda
2 >>>print(f(1, 2))                                  # 1 2      lambda
3
```


7. Python

◆
◆ main()
◆ Python
◆ Python
if __name__ == '__main__':
 ◆ '__name__'
 ◆ if
 ◆
 ◆
 ◆

__ " "

8.

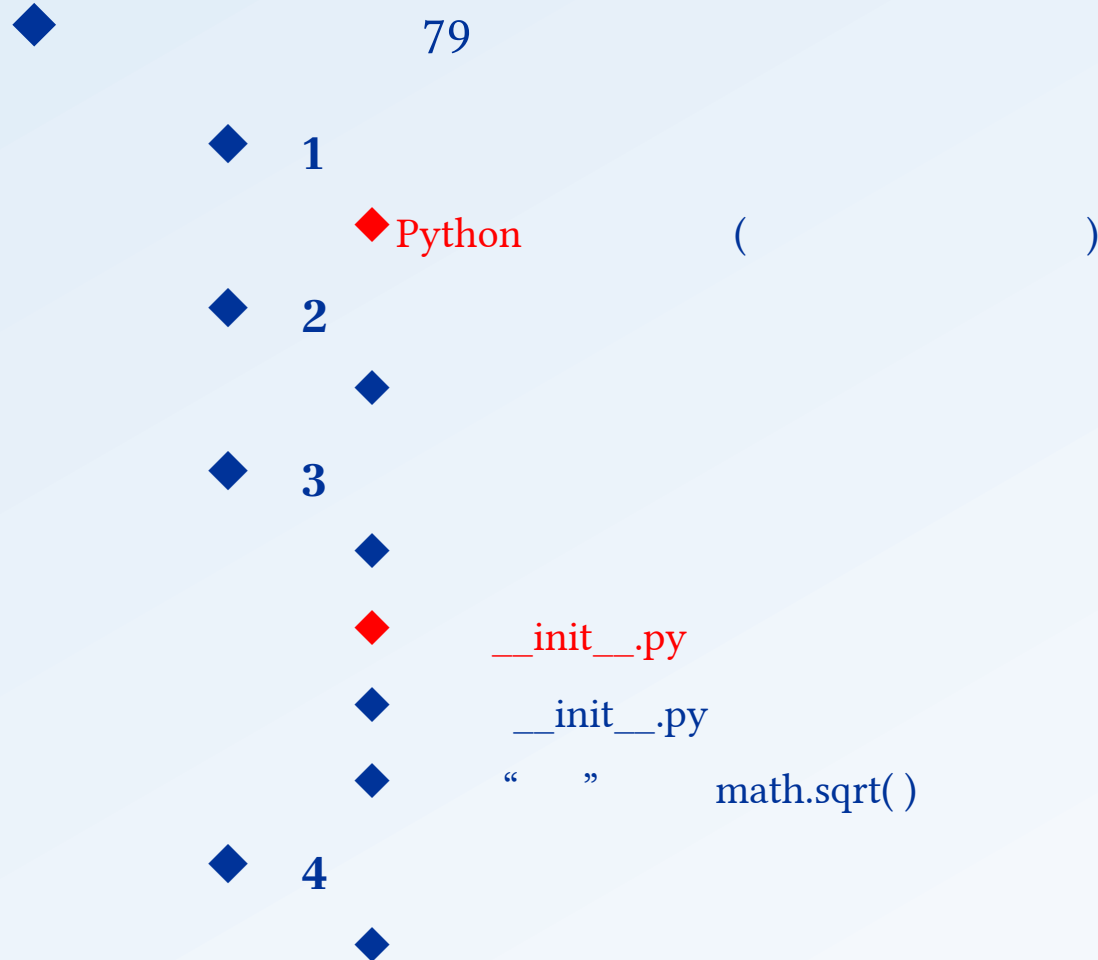
◆ Python



◆ if for while def class try with

2.1.5 Python





1.

1 int)

- ◆ 32 Python 231-1=2 147 483 647
- ◆ 64 Python 263-1=9 223 372 036 854 775 807
- ◆ Python
- ◆ Python

```

1 >>>123**100 # **
... # 209
2 >>>import sys #
3 >>>sys.maxsize #
2147483647
4 >>>sys.float_info.max #
1.7976931348623157e+308
5 >>>sys.float_info.min #
2.2250738585072014e-308

```

2.2.1



1.

2

float)



IEEE 754



CPU



ALU Arithmetic Logic Unit



FPU Floating Point Unit



```
1 >>>0.1+0.2==0.3
  False
2 >>>print(0.1+0.2)
  0.30000000000000004
```



2.



2.2.1



2.



0



-1



“ ” “ ”



“ ” 1

→											
	0	1	2	3	4	5	6	7	8	9	10
	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
								←			

2.2.1



1	>>>strDemo=' ', # strDemo
2	>>>strDemo[5:10] # 5 9
	' '
3	>>>strDemo[5:] # 5
	' '
4	>>>strDemo[-6:] # -6
	' '
5	>>>strDemo[-6::2] # -6 2
	' '
6	>>>strDemo[-1:-7:-1] # -1 -7
	' '
7	>>>strDemo[-1:-7:-2] # -1 -7 2
	' '

2.2.1



2.

3



```
1 >>>strDemo = '          '          #          strDemo
2 >>>strDemo = replace(strDemo, '          ')          #
Traceback (most recent call last):          #
3   File "<pyshell#1>", line 1, in <module>
4     strDemo = replace(strDemo, '          ')
NameError: name 'replace' is not defined
5 >>>x=strDemo.replace('          ','          ')          #          x
>>>x          #          x
'          '
>>>strDemo          #          strDemo
'          '
```

2.

4

◆ Python 4

1 “ ”

➤ C printf Python “ %”

➤ “ %s” Python situation “ ”

➤ “ ” “ %”

```
1 >>> situation=" "
2 >>> "%s, " % situation
' , '
,
```

```
1 >>> situation=" "
2 >>> attitude=" "
3 >>> length=11
4 >>> "%s, %, %d " % (situation, attitude, length)
' , , 11 '
```

2.

(4 □ □ □ □ □ □ □ □ □ □)

2 “ ”

➤ “ ”

%

➤ “ ”

format()

```

1 >>>situation="      "
2 >>>attitude="      "
3 >>>length=11
4 >>>"{}", {}, {} ".format(situation,attitude,length) #
  '      ,      , 11 '
5 >>> "{situation}, {attitude}, {length:x} " \      #
  .format(situation=situation,attitude=attitude,length=length)
  '      ,      , b '

```

➤

➤

Python 3

“ ”

“ % ”

2.

4

3

Python 3.6+

➤ Python3.6 Formatted string literal

➤ Python

```
1 >>>situation="    "
2 >>>attitude="    "
3 >>>length=11
4 >>>f"{situation}, {attitude}, {length}  "
> '    ,    , 11 '
```

```
1 >>>numA=2; numB=3
2 >>>f'2 3 {numA+numB} {2*(numA+numB)}. '
'2 3 5 10.'
```

2.

4

4



Python

Template



```
1 >>> situation="      "  
2 >>> attitude="      "  
3 >>> length=11  
4 >>> from string import Template  
5 >>> t=Template("$situation, $attitude, $length ")  
6 >>> t.substitute(situation=situation,attitude=attitude,length=length)  
    '      ,      , b '
```

2.2.1



2.



```
1 >>>password="7355608"
2 >>>class Error():
3     def __init__(self):
4         pass
5 >>>err=Error()
6 >>>userInput='{error.__init__.__globals__[password]}'
7 >>>userInput.format(error=err)
'7355608'
```



2.

5

1 join()

➤ join()

```
1 >>>demoSeq=[" ",""," ",""," "]
2 >>>"-".join(demoSeq)
' - - - '
```

2 split()

➤ split() join()

➤

```
1 >>>splitString="3!=1*2*3"
2 >>> splitString.split("*")
['3!=1', '2', '3']
```


2.

5

3 replace()

➤ replace()

➤

\xa0

\u3000

replace()

4 find()

```
1 >>> error="\u3000\u0000 !"
2 >>> error.replace(u"\u3000",u "").replace(u"\xa0",u "")
   '\u0000 !'
```

➤ find()

-1

```
1 >>> " ".find(" ")
   -1
2 >>> " ".find(" ")
   3
```

1.

◆ list Python



➤ len()

➤ max() min()

➤ sum()

2.2.2



1.

1



[]



```
=[] #
```

```
=list() #
```

```
=[ 1 2 ... n] #
```

```
[ ] #
```

```
[ ] #
```


2



3


```
# listDemo
# " "
# " "
```

2.2.2



1.

4



append() extend() insert()

1 append

.append()	#	1
.extend([])	#	
.insert(,)	#	

1	>>> listDemo=[" "," "," "," "]	#	listDemo
2	>>> listDemo.append(" ")	#	1
3	>>> listDemo	#	listDemo
	[' ',' ',' ',' ']	#	

2.2.2



1.

4

2 `extend()`

3 `insert()`

1	<code>>>> listDemo=[" ", " ", " ", " ", " "]</code>	<code>#</code>	<code>listDemo</code>
2	<code>>>> listDemo.extend(" ", " ", " ", " ")</code>	<code>#</code>	
3	<code>>>> listDemo</code>	<code>#</code>	<code>listDemo</code>
	<code>[' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ']</code>	<code>#</code>	

1	<code>>>> listDemo.insert(7, " ")</code>	<code>#</code>	
2	<code>>>> listDemo</code>	<code>#</code>	<code>listDemo</code>
	<code>[' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ']</code>	<code>#</code>	

2.2.2



1.

5



“ + ”

1	>>> concatenate=[" "]	#	concatenate
2	>>> concatenateTwo=[" "]	#	concatenateTwo
3	>>> concatenateOne+=concatenateTwo	#	concatenateOne
4	>>> concatenateOne		
	[' ',' ']		



“ * ”

1	>>> copyList1=[" "]	#	copyList1
2	>>> copyList2=[" "]	#	copyList2
3	>>> copyList1=copyList1+copyList2*3	#	copyList1 3 copyList2
4	>>> copyList1		
	[' ',' ',' ']		

2.2.2



1.

6



```
1 >>> lappingList=[copyList1, copyList2]
2 >>> lappingList
[[' ', ' ', ' ', ' ', ' '], [' ']]
```

7



Python



A

B

```
1 >>> copyList2=[" "] # copyList2
2 >>> copyList3=copyList2 #
3 >>> copyList2[0]=" " # copyList2 copyList3
4 >>> copyList3
[[' ']]
```


2.2.2



1.

8



[]=

1	>>> List1=[' ',' ',' ',' ',' ']	#	List1
2	>>> List1[2]=' '	#	2
3	>>>List1		
	[' ',' ',' ',' ',' ']		

2.2.2



1.

9

- ◆ pop() del
- ◆ clear()

```
1 >>> List1=[' ',' ',' ',' ',' ']  
2 >>> List1.pop(2)  
3 >>>List1  
  [' ',' ',' ',' ']  
4 >>>del List1[1]  
5 >>>List1  
  [' ',' ',' ']  
6 >>>List1.clear()  
7 >>>List1  
  []
```

2.



tuple



()

[]

 $= (\quad 1 \quad 2 \quad 3 \dots \quad n)$

1



2.

1



Python

```
1 >>> tupleDemo1=(1,) #
2 >>>tupleDemo2=2,3,4,5,6 #
3 >>> tupleDemo2 #
  (2, 3, 4, 5, 6)
4 >>>tupleDemo2[1] #
  3
```

2.2.2



2.

2



“ + ” “ * ”

```
1 >>> tupleNew=tupleDemo1+tupleDemo2      #
2 >>>tupleNew                               #      tupleNew
   (1, 2, 3, 4, 5, 6)
3 >>> tupleCopy=tupleDemo1*2               #
4 >>>tupleCopy
   (1, 1)
```

2



del

```
1 >>> del tupleNew                          #
2 >>>tupleNew
NameError: name 'tupleNew' is not defined
```



1.



{ : }



“ ” “ ”



2.2.3



1.

1



}

1	>>> poemDict={" " " " " " " " " :26}	#	
2	>>> poemDict	#	
	{' ' ' ' ' ' ' ' ' : 26}		
1	>>>dict1 = {' ' ' ' ' ' ' ' ' :1000}	#	
2	>>>dict2 = {' ' ' :20, ' ' ' :1000}	#	
3	>>>dict3 = {' ' ' ' ' ' ' ' ' :1000}	#	“ ” dict1 dict3
4	>>>dict4 = {' ' ' :35, ' ' ' :1000}	#	
5	>>>dict5 = {' ' ' :86, ' ' ' :86}	#	“ ”
6	>>>dict5[' '] = 88	#	dict5 ' ' :88
7	>>>dict6 = {(85, 95):' ' ' , 60:' ' ' }	#	(80, 90)
8	>>>dict7 = dict(x=10, y=20)	#	dict()



1.

2



3



1	>>> poemDict[" "]	#
	' '	

1	>>> poemDict[" "]=" "	#
2	>>> poemDict	
	{' ':' ', ' ':' ', ' ':' ': 26}	
3	>>>poemDict[' ']=' 675 “	#
	>>> poemDict	
	{' ':' ', ' ':' ', ' ':' ': 26, ' ':' ' 675 '}	

2.



set



“ ” “ ”



2.

1



	={} =set()	# #
1	>>>setA = {1, 2, 3, 2, 1, 5, 6, 3, 4, 7, 8}	# setA
2	>>>setA {1, 2, 3, 4, 5, 6, 7, 8}	# #
3	>>>s = {' ', ' ', ' ', ' ', ' ', ' '}	# s
4	>>>s {' ', ' ', ' '}	# #

2.

2



1	>>> numSeriesA={1,2,3,4,5}	#	numSeriesA
2	>>>numSeriesA.add(0)	#	
3	>>>numSeriesA	#	
	{0, 1, 2, 3, 4, 5}		
4	>>>set1 = set((" ", " ", " "))	#	
5	>>>set1.add(" ")	#	
6	>>>set1		
	{' ', ' ', ' '}	#	

2.

3

- ◆ remove() discard() pop()
- ◆ remove() discard() pop()

```
1  >>> numSeriesA={1,2,3,4,5}           #    numSeriesA
2  >>>numSeriesA.remove(1)               #    1
3  >>>numSeriesA.remove(1)
   Traceback (most recent call last):    #    1
4  >>>numSeriesA.discard(1)              # discard()
5  >>>numSeriesA.pop()                   # pop()
   2
6  >>>numSeriesA                         #
   {3, 4, 5}
```

2.2.3



2.

4



& | - > < ==

1	>>>s1 = {' ', ' ', ' ', ' ', ' ', ' ', ' '}	#	s1
2	>>>s2 = {' ', ' ', ' ', ' ', ' ', ' ', ' '}	#	s2
3	>>>s3 = s1 & s2	#	s1 s2
4	>>>s3	#	
	{' ', ' ', ' ', ' '}	#	
5	>>>s4 = s1 s2	#	s1 s2
6	>>>s4	#	
	{' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '}		
7	>>>s5 = s1 - s2	#	s1 s2
8	>>>s5	#	
	{' ', ' ', ' '}		
9	>>>s6 = s1 ^ s2	#	s1 s2
10	>>>s6	#	
	{' ', ' ', ' ', ' ', ' ', ' '}		

2.

4

◆ “<” “>”

◆ “==”

1	>>>s1 = {1,2,3,4,5}	#	s1
2	>>>s2 = {1,2,3}	#	s2
3	>>>s1>s2	#	s2 s1
	True		
4	>>>s1==s2	#	
	False		
5	>>>s3 = {1,2,3}	#	s3
6	>>>s2==s3	#	
	True		

- ◆ 3
- ◆
- ◆

“import matplotlib.pyplot”

- ◆ D:\Python38\Lib\site-packages\matplotlib\pyplot.py

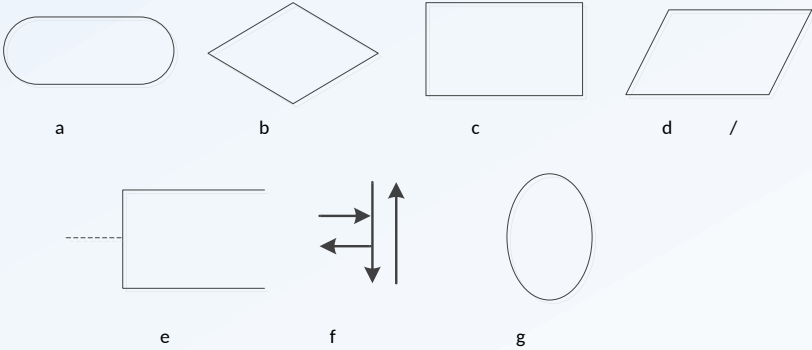
1	import	#
2	import as	#
3	from import	#
4	from import *	#

1

2



7

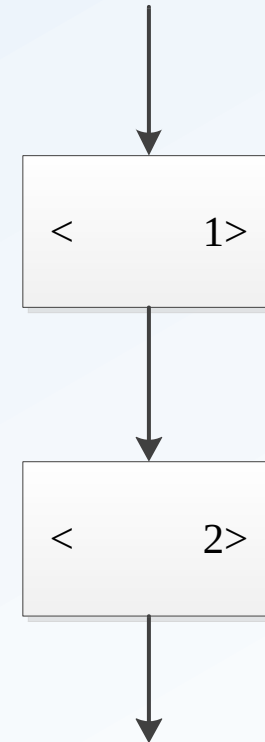


/

2.3.2



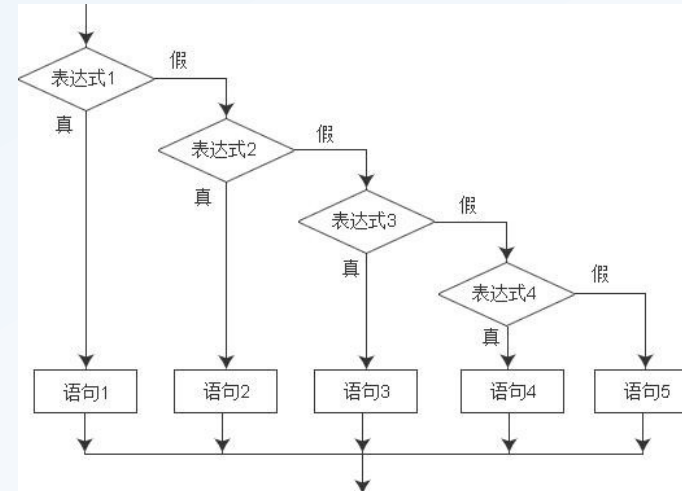
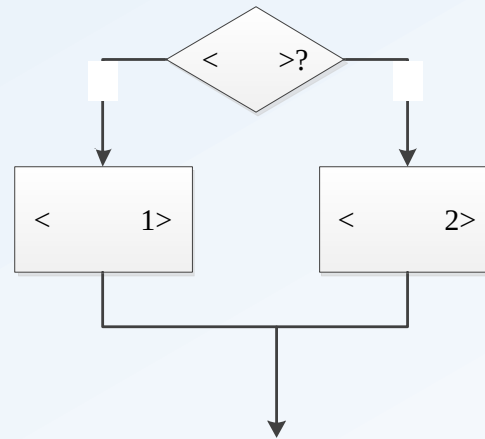
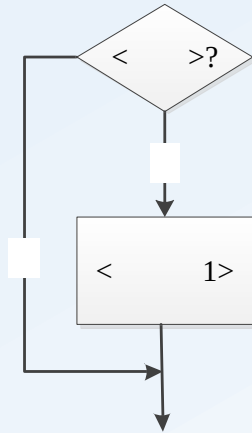
```
1 >>> numA=10
2 >>> numB=10
3 >>> numC=10
4 >>> average=(numA+numB+numC)/3
5 >>> average
10.0
```



2.3.3



- ◆
- ◆
- ◆ Python if



1.



:

1	if	#	"=="	"=="
2	1	#	True	1



if

True False



if

if

1	>>>numA=10	#	numA
2	>>> if numA == 10:	#	numA 10
3	print("numA 10!")		
	numA 10!		

2.

◆ Python if-else

◆ :

1	if	#	"=="	"="
2	1	#	=True	1 if 2
3	else	#		
4	2	#	=False	2 if 1

◆ 1 if 2 if

◆

< 1>if else < 2>

1	>>>numA=5
2	>>>print("numA 10 ") if numA == 10 else print("numA 10 ")
	numA 10

3.

◆ Python if-elif-else

◆ elif else if :

1	if	1:	
2		1	1
3	elif	2:	
4		2	2
5	elif	3:	
6		3	3
7	else:		
8			4

◆ if elif else

◆ if else elif if

◆ if elif else

◆ if True elif else

3.

2 3

1	num = int(input(' '))	# if
2	if num % 2 == 0:	# if 1 %
3	if num % 3 == 0:	# if 2
4	print(' 2 3 ')	
5	else:	# if 2
6	print(' 2 3 ')	# if 2
7	else:	# if 1
8	if num % 3 == 0:	# if 3
9	print(' 3 2 ')	
10	else:	# if 3
11	print(' 2 3 ')	# if 3 1
	>>>	#
	60	
	2 3	

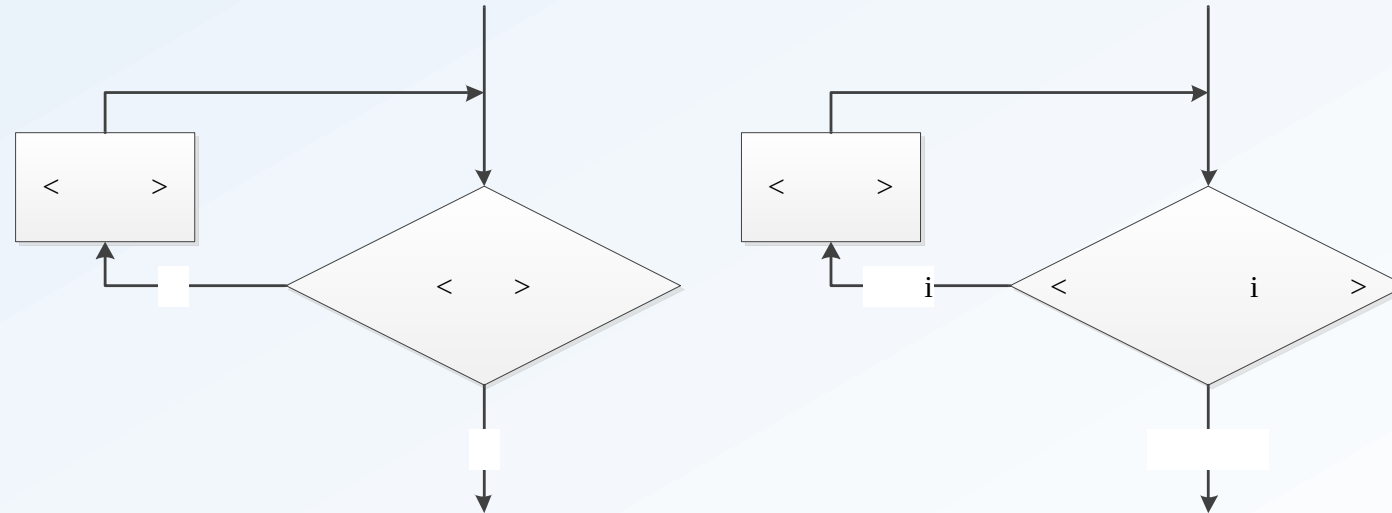
2.3.4





for

while



1. **for**◆ **for**

◆

◆

:

◆ **for**

1	for	in	:	#
2	<	>		

1	sum = 0	#	sum
2	for x in [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]:	#	
3	sum = sum + x	#	
4	print('1+2+3+...+10=', sum)		

>>>1+2+3+...+10= 55	#
-------------------------------------	----------

1. **for**

◆ `range()`

`range( )`

◆ `0 range(8) 8`

◆ `range(0, 8) 8`

◆ `1 range(8) range(0, 8, 1)`

◆ `range() [ ]`

2.3.4



100

10

1	high = 100	#	
2	total = 0	#	
3	for i in range(10):	#	range(10) 0-10
4	high /= 2	#	2
5	total += high	#	
6	print(' ', i+1, ' ', high)	#	i 0 +1
7	print(' [] ', total)		
>>>	1	50.0	#
	...		
	10	0.09765625	
	[]	99.90234375	

1. for

1	for	in		#	1
2	for	in	if	#	2

90-100

1	>>>scores = [95, 86, 75, 88, 91, 68, 90, 70]				#
2	>>>print(' ', [s for s in scores if s >=90 and s < 100])				#
	[95, 91, 90]				

2. while

◆ while

◆ :

1	while < >:	#	=True
2	< >	#	=False

◆ while 1-100

1	n = 100	#	
2	sum = 0	#	sum
3	counter = 1	#	
4	while counter <= n:	#	counter<=n True
5	sum = sum + counter	#	sum +counter sum
6	counter += 1	#	ounter<=n False
7	print('1 %d =%d' %(n, sum))	#	
>>>1 100 =5050		#	

2. while

◆ while else :

```
1 while< 1>: # =True 1
2 < 1>
   else< 2>: # while else
3 < 2>
```

◆

◆ while else

```
1 >>>strDemo,idx="demo",0
2 >>>while idx < len(strDemo):
3     print(" :"+strDemo[idx])
4     idx+=1
5 else:
6     print(" ")
```

```
>>> :d :e :m :o #
```

3. break continue

- ◆ break
- ◆ continue

1	>>> strDemo="demo"	
2	>>>for s in strDemo:	
3	if s=="m":	
4	break	
5	print(s,end="")	
	>>>de	#

1	>>> strDemo="demo"	
2	>>>for s in strDemo:	
3	if s=="m":	
4	continue	
5	print(s,end="")	
	>>>deo	#

2.3.4



1	for i in range(1, 10):	#	9
2	for j in range(1, i+1):	#	9
3	print('{x}={y}\t'.format(j, i, i*j), end='')	#	
4	print()	#	
>>>			
1x1=1			
1x2=2 2x2=4			
1x3=3 2x3=6 3x3=9			
1x4=4 2x4=8 3x4=12 4x4=16			
1x5=5 2x5=10 3x5=15 4x5=20 5x5=25			
1x6=6 2x6=12 3x6=18 4x6=24 5x6=30 6x6=36			
1x7=7 2x7=14 3x7=21 4x7=28 5x7=35 6x7=42 7x7=49			
1x8=8 2x8=16 3x8=24 4x8=32 5x8=40 6x8=48 7x8=56 8x8=64			
1x9=9 2x9=18 3x9=27 4x9=36 5x9=45 6x9=54 7x9=63 8x9=72 9x9=81			

2.3.4



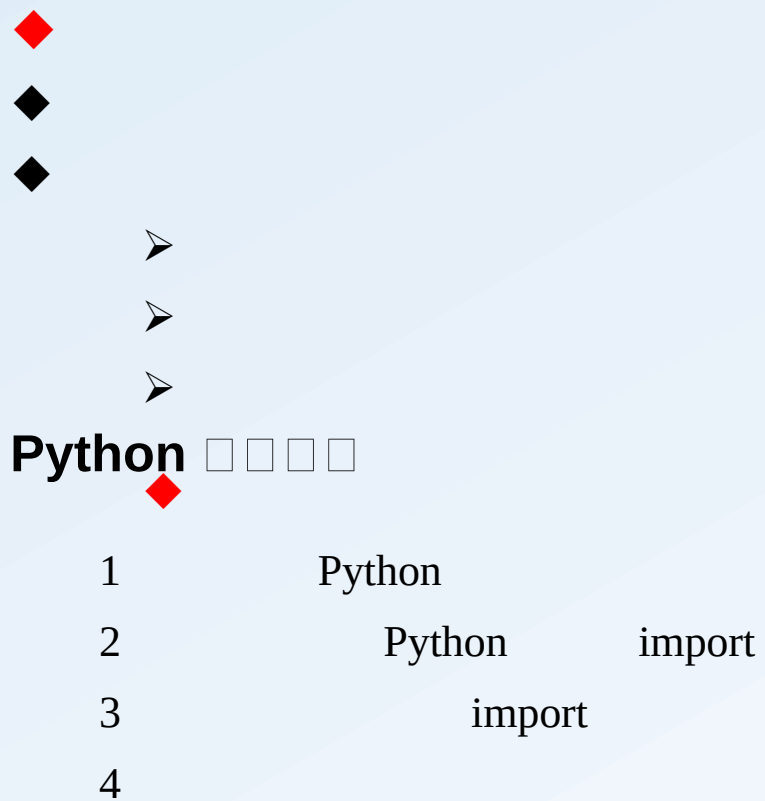
1-100

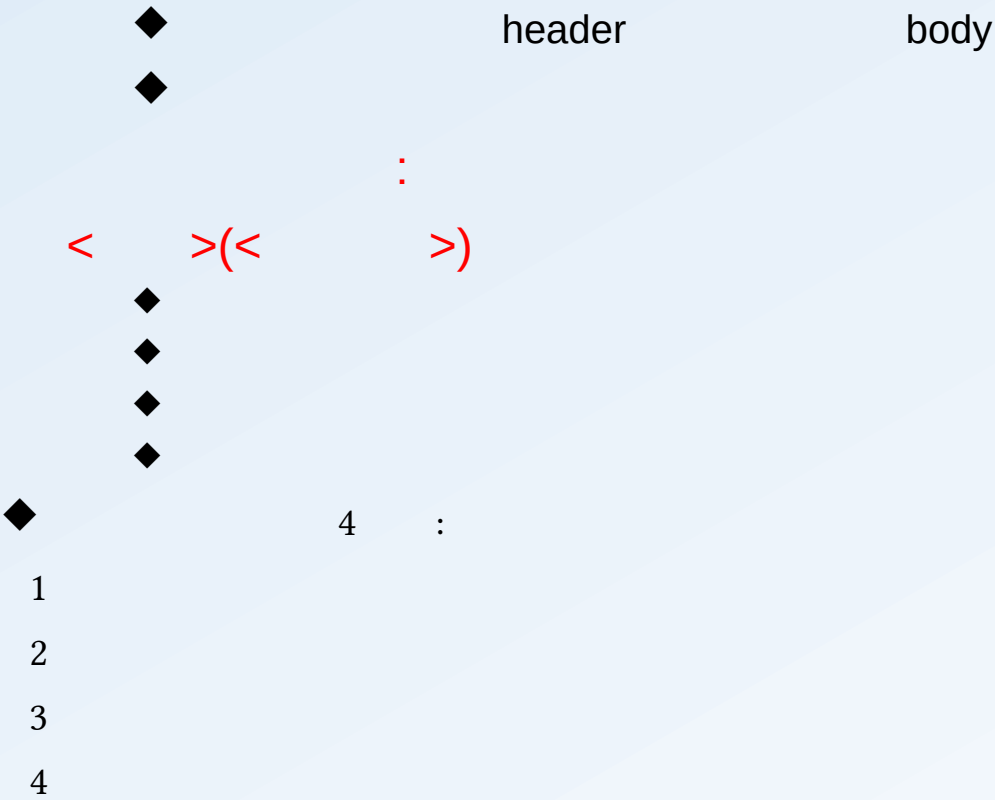


n $2 \sqrt{n}$

n

1	import math	#	-
2	for n in range(1, 100):	#	n 1-100
3	for j in range(2, round(math.sqrt(n))+1):	#	j 2 sqrt(n)
4	if n%j == 0:	#	n%j=0
5	break	#	
6	else:	#	
7	print(' ', n)	#	n%j≠0 n
>>> 1 2 3... #			





1.

◆ Python

◆

```
1 >>> def cal_token(token1,token2,token3):  
2     print(token1,token2,token3)  
3 >>> cal_token(1,2,3)  
1 2 3  
4 >>> cal_token(2,1,3)  
2 1 3
```

2.



```
1 >>> def cal_token(token1,token2,token3):  
2     print(token1,token2,token3)  
3 >>> cal_token(token2=2,token1=1,token3=3)  
1 2 3  
4 >>> cal_token(1,2,token3=3)  
1 2 3  
5 cal_token(1,token2=2,3) #SyntaxError: positional argument follows keyword argument
```



3.



```
1 >>> def cal_token(token1,token2,token3=3):  
2     print(token1,token2,token3)  
3 >>> cal_token(1,2)  
1 2 3  
4 >>> cal_token(1,2,6)  
1 2 6
```



None True False

4.



0 1 n



1 (*)



2 (**)

```
1 >>> def sum_square(*args):           # *args
2     sum=0
3     for n in args:
4         sum+=n**2
5     return sum
>>> sum_square(1,2,3)
14
```

2.4.2



1	>>>def do_something(name,age,gender=' ',*HandW,**score):	#	
2	print(' :%s, :%d, :%s'%(name,age,gender))	#	
3	print(' :',HandW)	#	
4	print(' :',score)	#	
5	>>>do_something(' ',60,' ',160,80, =20, =90)	#	
	>>>	#	
	: , :60, :		
	: (160, 80)		
	: {' ': 20, ' ': 90}		

5.



* *

```
1 >>> def call_tokens(*tokens,arg1,arg2): #
2     print(f"tokens :{tokens},arg1 :{arg1},arg2 :{arg2}")
3 >>> call_tokens(1,2,3,1,2) #
TypeError: call_tokens() missing 2 required keyword-only arguments: 'arg1' and 'arg2' #
4 >>> call_tokens(1,2,3,1,2) #
SyntaxError: positional argument follows keyword argument #
5 >>> call_tokens(1,2,3,arg1=1,arg2=2)
tokens :(1, 2, 3),arg1 :1,arg2 :2 #
```


6.



	*	**
1	>>> seq=[1,2,3]	#
2	>>> dic={" ":"",", ":" "}	#
3	>>> def functionA(*args,**kwargs):	
4	print(f"args {args}\nkwargs {kwargs}")	
5	>>> functionA(seq,dic)	#
	args ([1, 2, 3], {' ':'«»',' ':' '})	
	kwargs {}	#kwargs
6	>>> functionA(*seq,**dic)	#
	args (1, 2, 3)	#args
	kwargs {' ':'«»',' ':' '}	#kwargs



1



2



3



“

”

global

global

2.4.3



1	def test():	#	test()
2	txt = 'Python'	#	txt
3	print(txt)	#	txt
4	test()	#	test()
5	print('txt', txt)	#	txt
>>>		#	
Python		#	txt
NameError: name 'txt' is not defined		#	

global

1	def test1():	#	test1()
2	global txt	#	txt global
3	txt = 'Python'	#	
4	print(txt)	#	txt
5	test1()	#	test1()
6	print('txt', txt)	#	
>>>		#	
Python		#	
txt Python		#	

1.



def

return

def

return



Python

lambda

2.

= lambda :

◆ lambda



➤ lambda lambda

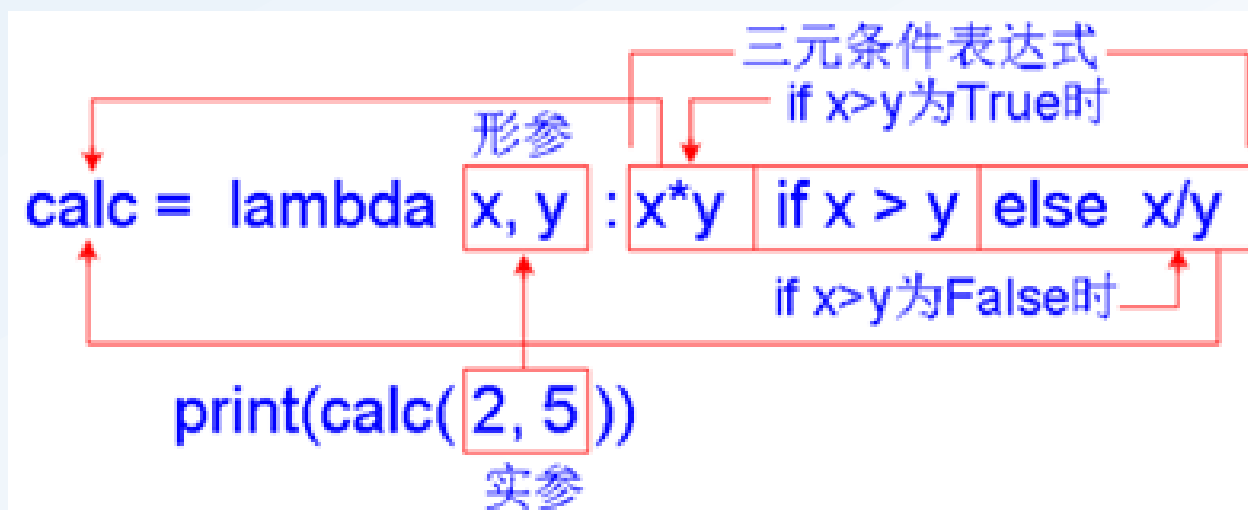
```
1 >>> pfunc = lambda x: x*x #  
2 >>> pfunc(3) # pfunc  
9
```



2.4.4

```
1 >>> calc = lambda x, y: x*y if x > y else x/y      #
2 >>> print(calc(2, 5))                             #    calc( )
0.4
```

- ◆ $x > y$ $x * y$ calc
- ◆ x / y calc



lambda

1	>>> poem=lambda x,y:x+y	#	lambda
2	>>> poem(" ", " ")	#	lambda
	,		

lambda key

1	>>> tuples=[(101,'a'),(59,'b'),(33,'c'),(267,'d')]	#	
2	>>> sorted(tuples,key=lambda x:x[0])	#	0
	[(33, 'c'), (59, 'b'), (101, 'a'), (267, 'd')]		

2.4.5



- ◆ .py .py Module
- ◆ ()
- ◆ Python import import math from math import pi

```
1 >>> from math import pi
2 >>> pi * 2 * 2
12.566370614359172
```

from math import

◆ \Python\Lib\site-packages import

➤ 1 IDLE

➤ 2 `print("Hello ")` # hello.py
➤ 3 `import hello.py`

```
>>>import hello
Hello
```

◆ \Python\Lib\site-packages

◆ .py

◆ Python __init__.py __init__.py

◆

1	import examples	#	examples	
2	import examples.exam	#	examples	exam
3	from examples import samp	#	samp	

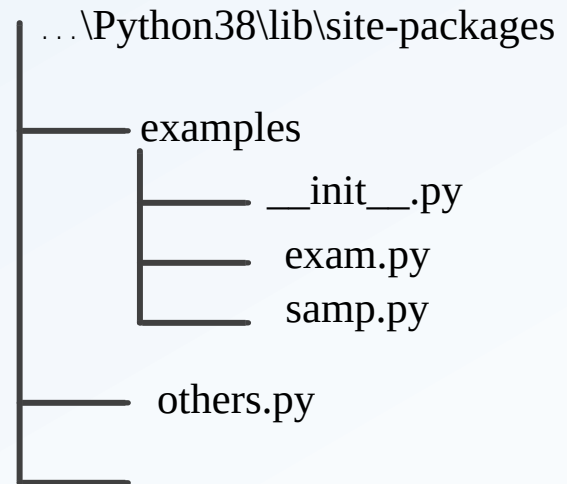
➤ Python __init__.py

➤ exam samp

 examples.exam exam

➤ from examples import samp samp

 samp



- 1 “D:\Python\Lib\site-packages” “demo”
- 2 demo `__init__.py`
- 3 4 `add.py` `sub.py` `mul.py` `div.py`

`add.py`

1	<code>def add(a, b):</code>
2	<code> return a+b</code>

`sub.py`

1	<code>def sub(a, b):</code>
2	<code> return a-b</code>

`mul.py`

1	<code>def mul(a, b):</code>
2	<code> return a*b</code>

`div.py`

1	<code>def div(a,b):</code>
2	<code> return a/b</code>

2.4.5



◆

```
1 >>>import demo.add #
2 >>>demo.add.add(2, 6) add      add
8
```

```
1 >>>import demo.mul #
2 >>>demo.mul.mul(4, 5)
20
```



/

1.

◆ open()

```
1      = open('      ',' ' )      #      1
2 with open('      ',' ' ) as      :      #      2
```

1



open()



```
2
3
4 " .txt"
5 "d:\\test\\02\\ .txt"
   mode "
      \n
```

r	FileNotFoundError
w	
a	
x	FileExistsError
r+	+
a+	
w+	
rb	b
wb	
ab	



```
1 >>>import os                # -
2 >>>os.getcwd()              #
  'D:\\Python38'
3 >>>os.chdir('d:\\test')     #
4 >>>os.getcwd()              #
  'd:\\test'
```

2.5.1





1	>>>f = open(r'./test/01/ .txt', 'r').read()	#	d:\
2	>>>f = open(r'./01/ .txt', 'r').read()	#	d:\test
3	>>>f = open(' .txt', 'r').read()	#	d:\test\01

2.5.1





```
d:\test\01\>>>f = open(r'd:\test\01\  .txt', 'r').read() #
```

2.5.1



◆ Windows \

◆ Linux UNIX /

d:\test\

1	>>>book1 = open('d:\\test\\02\\ .txt', 'r')	#	\\ 1 \
2	>>>book2 = open(r'd:\test\02\ .txt', 'r')	#	r ''
3	>>>book3 = open(R'd:/test/02/ .txt', 'r')	#	R
4	>>>book4 = open('d://test//02// .txt', 'r')	#	
5	>>>book5 = open('C://Users/Administrator/\	#	\
6	Desktop/ .txt', 'r')	#	
7	>>>book6 = open('./02/ .txt', 'r')	#	d:\test\01\
8	>>>book7 = open('.\02\\ .txt', 'r')	#	d:\test\01\
9	>>>book8 = open(' .txt', 'r')	#	d:\test\

2.

◆ close()

◆ Python

◆ flush

 .close()

#

 .flush()

#

2.5.2



1.

◆ Python read() readline() readlines()

◆ splitlines()

1 read([size])

◆ size size

◆ size

read() “ .txt”

```
1 >>>f = open(" .txt") # f
2 >>>s = f.read( ) #
3 >>>s #
' \n[ ] \n\n \n ' # \n
```

1.

1 read([size])

read() “ .txt”

1	>>>f = open(" .txt")	# f
2	>>>s = f.read()	#
3	>>>s=s.splitlines()	#
4	>>>s	#
	[' ;'[] ;',' ']	

2.5.2



1.

2 `readline([size])`

◆ `size` `size` `size`

◆ `size`

◆ `size`

`readline()` “ `.txt`”

1	>>>f = open(" .txt")	# f
2	>>>f.readline() ' \n'	#
3	>>>f.readline(5) '[] '	# 5 #

1.

3 readlines()

`readlines()` “ .txt”

1	>>>f = open(" .txt")	# f
2	>>>f.readlines()	#
	[' \n', '[] \n', '\n', ' ' \n', ' ']	#
	>>>f.close()	
3		#

2.5.2



1.

◆ Python write() writelines()

1 write()

.write(" ")

◆ write()

" .txt"

```
1 >>>str1 = '          \n          \n'          #          \n2 >>>f = open(' .txt', 'w')          #          \n3 >>>f.write(str1)          #          \n26          #          \n4 >>>f.close( )          #          \n5 >>> f=open(" .txt")          \n6 >>> f.read()          \n'          \n          \n'
```


2.5.2



1.

2 writelines()

.writelines(" ")

" .txt"

```
1 >>>s=[' \n', ' \n', ' \n'] # \n
>>>f = open(' .txt', 'a+')
2 >>>f.writelines(s) #
3 >>>f.close() #
4 >>> f=open(" .txt") #
5 >>> f.read()
6 ' \n \n \n \n
 \n'
```

- ◆ `file.seek(n)` `n` `0`
- ◆ `file.tell(n)`
- ◆ `for line in file`

1	<code>>>>f = open(' .txt')</code>	#
2	<code>>>>f.seek(3)</code>	# 3
	3	
3	<code>>>>print(f.tell())</code>	#
	3	
4	<code>>>> file=open(" .txt")</code>	
5	<code>>>> for index,content in enumerate(file):</code>	#
6	<code> print(f" {index} , :{content}")</code>	
	0 , :	
	1 , :[]	
	2 , :	
	3 , :	
	4 , :	



3. Data collection

Data collection



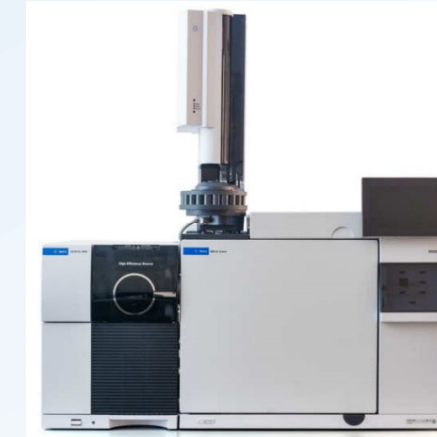
- Source of data
Instruments:



NMR



Raman



MS



UV-vis



Fluorescence



X-ray mapping

Data collection



- Source of data

Publishers:

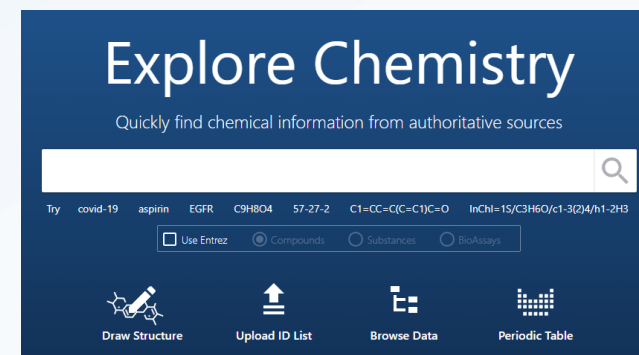
WILEY  Online Library



nature



Dataset:



scientific **data**

ZINC20

RESEARCH ARTICLE

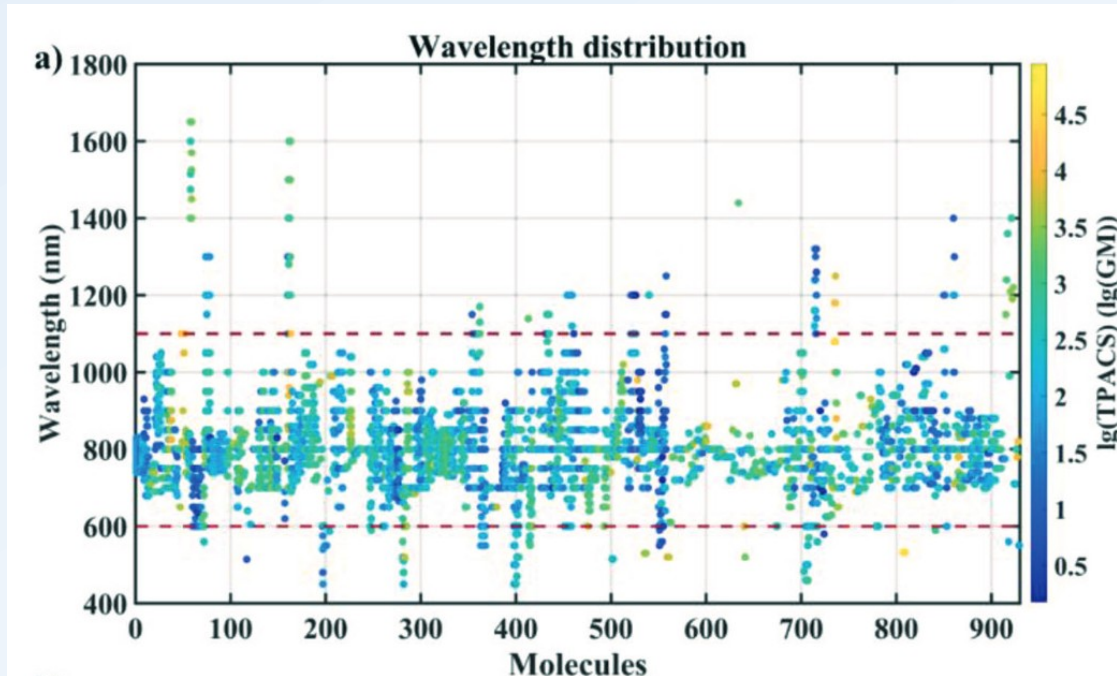
ADVANCED
SCIENCE

Open Access

www.advancedscience.com

Interpretable Machine Learning of Two-Photon Absorption

Yuming Su, Yiheng Dai, Yifan Zeng, Caiyun Wei, Yangtao Chen, Fuchun Ge, Peikun Zheng, Da Zhou,* Pavlo O. Dral,* and Cheng Wang*



Adv. Sci. 2023,10, 2204902

Some packages for this task:

CV

pyautogui

selenium

wget

Pdfminer

PyPDF2

pymupdf

.....

Data collection

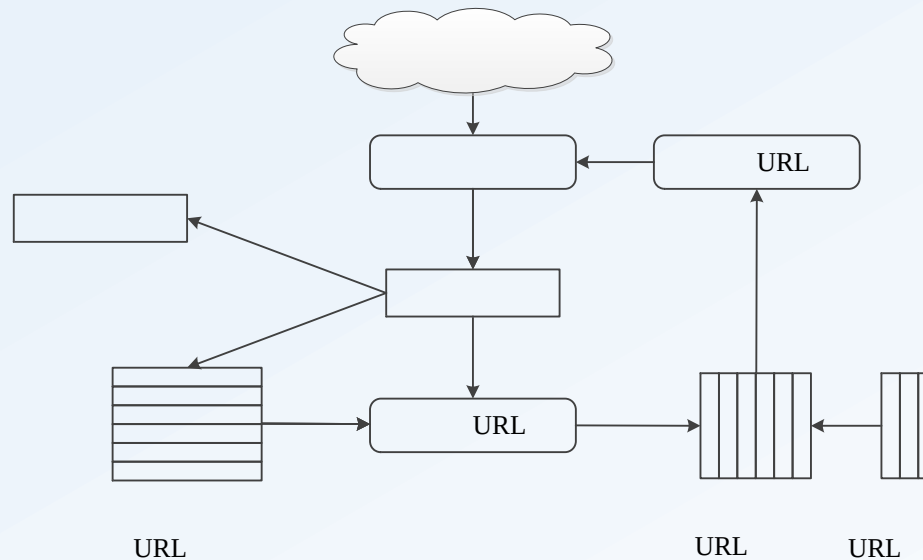


- **How to obtain these data?**

Software: ETL, Flume, Fluent, Logstash, Chukwa

- **Scalable Web Crawler**

A web crawler, crawler or web spider, is a computer program that's used to search and automatically index website content and other information over the internet.



Software: Google bot, ApacheNutch, CUN Wget,

Data collection



- **How to obtain these data?**

We can use Python to conduct scalable web crawler.

- ◆ **urllib**

- urllib Python HTTP HTTP

- **Urllib**

- urllib.request HTTP HTTP

- urllib.error urllib.request

- urllib.parse URL URL

- urllib.robotparser robots.txt robots.txt RobotFileParser can_fetch()

Data collection



◆ urllib

- `urllib.request.urlopen()` `urllib.request.urlopen()`
- `urllib.request.urlopen(url,data=None,[timeout,]*,cafile=None,capath=None,cadefault=False,context=None)`
 - `url` URL
 - `data` None GET `data` POST
 - `timeout`
 - `cafile capath cadefault` CA HTTP
 - `context` SSL

- How to obtain these data?

- ◆ urllib

1 urllib

```
1 import urllib.request                # request
2 response=urllib.request.urlopen('https://    # urllib.request    urlopen()    URL
   www.baidu.com')                    # read()
3 html=response.read().decode('UTF-8')    #
4 print(html)
```

```
===== RESTART: D:/Program Files/urllib1.py =====
<html>
<head>
    <script>
        location.replace(location.href.replace("https://", "http://"));
    </script>
</head>
<body>
    <noscript><meta http-equiv="refresh" content="0;url=http://www.baidu.com/"></noscript>
</body>
</html>
```

Data collection



◆ Requests

➤ Requests

Python

HTML

➤ urllib

HTTP

requests.requests()	
requests.get()	HTML HTTP GET
requests.head()	HTML HTTP HEAD
requests.post()	HTML POST HTTP POST
request.put()	HTML PUT HTTP PUT
requests.patch()	HTML HTTP PATCH
requests.delete()	HTML HTTP DELETE

◆ Requests

2 GET

```
1 import requests #
2 response = #
3 requests.get('https://www.baidu.com')
4 print('    :',response.status_code) #
5 print('url:',response.url) #
6 print('header:',response.headers) #
7 print('cookie:',response.cookies) # cookie
8 print('text:',response.text) #
9 print('content:',response.content) #
```

Data collection



◆ **BeautifulSoup**

- BeautifulSoup HTML XML Python Python
- BeautifulSoup Python DOM Document Object Model Unicode
UTF-8
- BeautifulSoup Python lxml html5lib
- BeautifulSoup BeautifulSoup lxml html5lib
Python
- lxml lxml

Python	BeautifulSoup(markup,'html.parser')	bs4
lxmlHTML	BeautifulSoup(markup,'lxml')	pip install lxml
lxmlXML	BeautifulSoup(markup,'xml')	pip install lxml
html5lib	BeautifulSoup(markup,'html5lib')	pip install html5lib

◆ BeautifulSoup

3 BeautifulSoup

```
1 import requests
2 from bs4 import BeautifulSoup          #
3 url = 'http://httpbin.org/get'
4 r = requests.get(url)
5 soup = BeautifulSoup(r.text, 'html.parser')    # BeautifulSoup
6 print(soup.prettify())                    #
```

```
{
  "args": {},
  "headers": {
    "Accept": "*/*",
    "Accept-Encoding": "gzip, deflate",
    "Host": "httpbin.org",
    "User-Agent": "python-requests/2.23.0",
    "X-Amzn-Trace-Id": "Root=1-62d7b957-00eelcce0d450dc3017f7251"
  },
  "origin": "183.227.100.21",
  "url": "http://httpbin.org/get"
}
```

Data collection



Homework

1. vibe coding ACS Wiley 403



4. Data preprocessing

Data preprocessing



Data preprocessing

- Cleaning
- Integration
- Conversion
- Reduction
- Masking

Data preprocessing



◆ NumPy Numerical Python

Python

◆ NumPy

,

◆ NumPy

N

ndarray

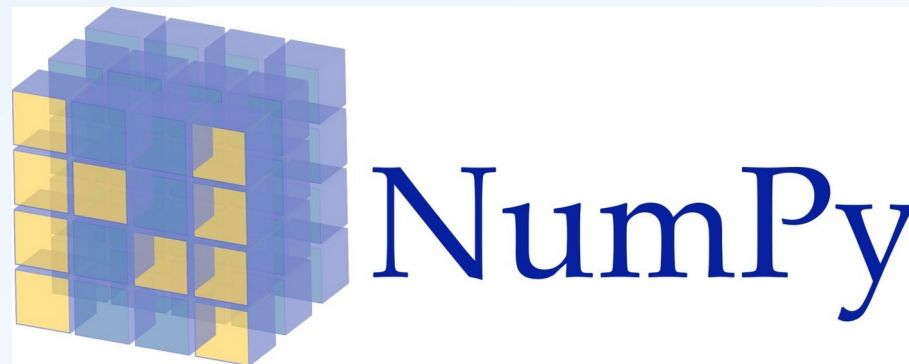
ndarray

ndarray

Python

0

ndarray



Nature **585**, 357–362 (2020)

Data preprocessing



◆ ndarray

◆ ndim

◆

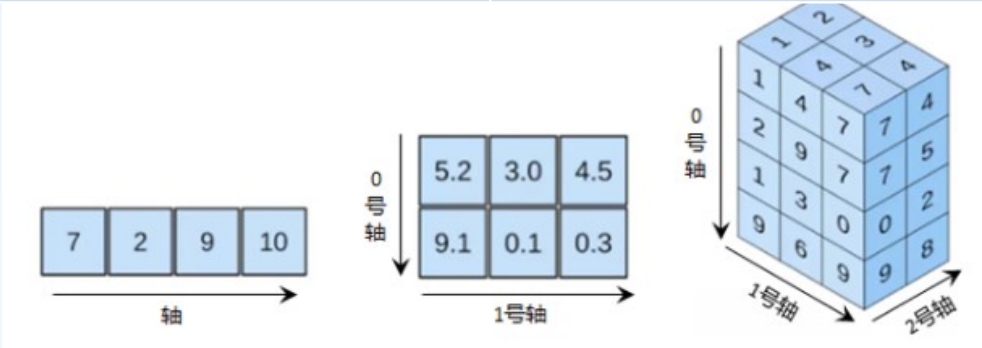
NumPy

0

1

1 2 0

ndim	
shape	
size	
dtype	
itemsize	



Data preprocessing



◆ ndarray

◆ shape

◆ dtype

(,)

bool	True False
int8 uint8	8
int16 uint16	16
int32 uint32	32
int64 uint64	64
float16	16
float32	32
float64	64
complex64	32
complex128	64

Data preprocessing



1.

◆ NumPy `numpy.array()`



◆ : Python NumPy Python “pip install numpy” NumPy
 NumPy NumPy “import numpy”

1	pip install numpy	# numpy
2	import numpy	# numpy

Data preprocessing



1

1	import numpy as np	#	numpy
2	a = np.array(range(0,9))	#	
3	print(' a {}'.format(a))	#	a
4	print(' a {}'.format(len(a)))	#	a a.size
5	print(' a {}'.format(a.shape))	#	a
6	print(' a {}'.format(type(a)))	#	a
7	print(' a {}'.format(a.ndim))	#	a
8	a[1]=9999	#	1 9999
9	a[2]=6666	#	2 6666
10	print(' a {}'.format(a))	#	2 6666

>>>	#
a [0 1 2 3 4 5 6 7 8]	
a 9	
a (9,)	
a <class 'numpy.ndarray'>	
a 1	
a [0 9999 6666 3 4 5 6 7 8]	

Data preprocessing



2.



NumPy

NumPy

2

```
1 import numpy as np                                # numpy
2 a = np.array([[1, 2], [3, 4]])                    #
3 print(' a {}'.format(a))                          # a
4 print(' a {}'.format(type(a)))                    # a
5 print(' a {}'.format(type(a[0][0])))              # a
6 print(' a {}'.format(a.ndim))                     # a
7 b = np.array([1, 2, 3, 4, 5], dtype=np.float64)   #
8 print(' b {}'.format(b))                          # b
9 print(' b {}'.format(type(b)))                    # b
10 print(' b {}'.format(b.ndim))                    # b
11 c= np.array([[1, 2, 3, 4, 5]])                   #
12 print(' c {}'.format(c))                          # c
>>>                                                # ( )
```

Data preprocessing



2

()

```
>>> #  
a [[1 2]  
[3 4]]  
a      <class 'numpy.ndarray'>  
a      <class 'numpy.int32'>  
a      2  
b [1. 2. 3. 4. 5.]  
b      <class 'numpy.ndarray'>  
b      1  
c [[1 2 3 4 5]]
```


Data preprocessing



◆ numpy.array()

- `zeros((m,n))` $m \quad n \quad 0$
- `ones((m,n))` $m \quad n \quad 1$
- `eye(m)` $\begin{matrix} & & 1 & & 0 & m & m \end{matrix}$
- `arange(m,n,t)` $\begin{matrix} & m & n & t \end{matrix}$
- `full((m,n),c)` $m \quad n \quad c$
- `empty()`

◆ `eye(m,k,dtype)`

$0 \quad "1" \quad 0 \quad k \quad "1" \quad 0 \quad k \quad "1"$

◆ `a = np.eye(3,k=1,dtype=int)`

`a = np.eye(3,k=-1,dtype=int)`

```
array([[0, 1, 0],
       [0, 0, 1],
       [0, 0, 0]])
```

```
array([[0, 0, 0],
       [1, 0, 0],
       [0, 1, 0]])
```

Data preprocessing



3 NumPy

1	<code>import numpy as np</code>	#	numpy
2	<code>a=np.zeros(5)</code>	#	5 0
3	<code>print(' a {}'.format(a))</code>	#	a
4	<code>b=np.ones(5)</code>	#	5 1
5	<code>print(' b {}'.format(b))</code>	#	b
6	<code>c=np.zeros((2,2))</code>	#	4 0
7	<code>print(' c {}'.format(c))</code>	#	c
8	<code>d=np.ones((2,2))</code>	#	4 0
9	<code>print(' d {}'.format(d))</code>	#	d
10	<code>e=np.eye(3)</code>	#	3x3
11	<code>print(' e {}'.format(e))</code>	#	e
12	<code>f=np.arange(0,10,2)</code>	#	0 10 2
13	<code>print(' f {}'.format(f))</code>	#	f
14	<code>g=np.full((4,4),6)</code>	#	4x4 6
15	<code>print(' g {}'.format(g))</code>	#	g
	<code>h=np.arange(1,30,5)</code>	#	[1,30) 5
	<code>print(' h {}'.format(h))</code>	#	g
	<code>>>></code>	#	()

Data preprocessing



3 NumPy

()

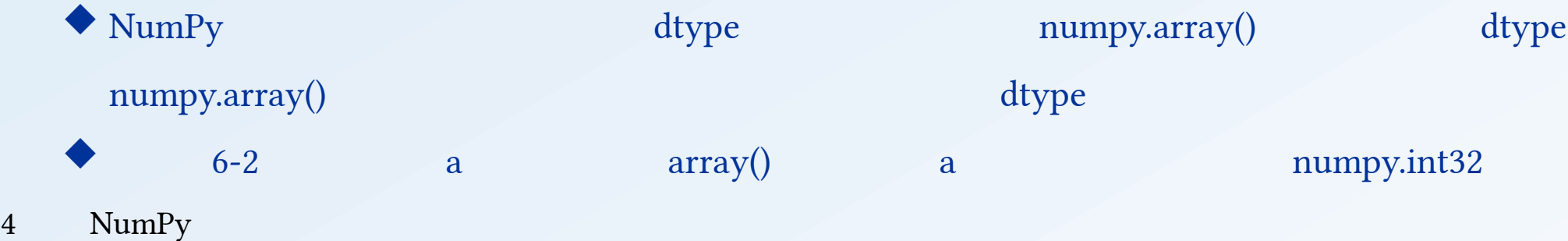
```
>>> #  
a [0. 0. 0. 0. 0.]  
b [1. 1. 1. 1. 1.]  
c [[0. 0.]  
[0. 0.]  
d [[1. 1.]  
[1. 1.]  
e [[1. 0. 0.]  
[0. 1. 0.]  
[0. 0. 1.]  
f [0 2 4 6 8]  
g [[6 6 6 6]  
[6 6 6 6]  
[6 6 6 6]  
h [ 1 6 11 16 21 26]
```

:arange()

Data preprocessing



3. NumPy



1	import numpy as np	#	numpy
2	a=np.array([1,2,3,4],dtype=np.int64)	#	a int64
3	print(' a {}'.format(a.dtype))	#	a
4	b=np.array([1,2,3,4],dtype=np.float64)	#	b float64
5	print(' b {}'.format(b.dtype))	#	b
>>> a int64 b float64		#	

Data preprocessing



5 NumPy

```
1 import numpy as np                                # numpy
2 a=np.array([1,2,3,4])                             #
3 print('  a      {}'.format(a.dtype))              # a
4 float_a=a.astype(np.float64)                     # a
5 print('          {}'.format(float_a.dtype))        #
>>>  a      int32      float64                      #
```

6 NumPy

```
1 import numpy as np                                # numpy
2 a=np.array([1.1,1.2,1.3,1.4])                     #
3 print('  a      {}'.format(a))                    # a
4 print('  a      {}'.format(a.dtype))               # a
5 b=a.astype(np.int64)                               # int64
6 print('      a      {}'.format(b))                 # a
7 print('  a      {}'.format(b.dtype))               # a
>>>  a      [1.1 1.2 1.3 1.4]  a      float64      #
      a      [1 1 1 1]      a      int64
```

Data preprocessing



4. NumPy

◆ NumPy



Data preprocessing



7 NumPy

```
1 import numpy as np                                # numpy
2 a = np.array([2,1,1,2])                          # a
3 b = np.array([2,1,1,2])                          # b
4 print('          a,b   :')                       #
5 c = a + b                                         # a,b
6 print('{}+{}={}'.format(a,b,c))                  # a,b
7 print('          a,b   :')                       #
8 c = a - b                                         # a,b
9 print '{}-{}={}'.format(a,b,c)                   # a,b
10 print('          a,b   :')                      #
11 c = a * b                                         # a,b
12 print('{}*{}={}'.format(a,b,c))                  # a,b
13 print('          a,b   :')                       #
14 c = a / b                                         # a,b
15 print('{} / {} = {}'.format(a,b,c))              # a,b
16 print('  a      :')                              #
17 print(-a)                                         # a
18 print('  a      :')                              #
19 print(a**2)                                       # a
20 print('  a      :')                              #
21 print(a^b)                                       # a
>>>                                              # ( )
```

Data preprocessing



7 NumPy

()

```
>>>                                     #  
    a,b  :  
[2 1 1 2]+[2 1 1 2]=[4 2 2 4]  
    a,b  :  
[2 1 1 2]-[2 1 1 2]=[0 0 0 0]  
    a,b  :  
[2 1 1 2]*[2 1 1 2]=[4 1 1 4]  
    a,b  :  
[2 1 1 2]/[2 1 1 2]=[1. 1. 1. 1.]  
    a    :  
[-2 -1 -1 -2]  
    a    :  
[4 1 1 4]  
    a    :  
[0 0 0 0]
```


Data preprocessing



4. NumPy



NumPy



sqrt()

log()

sin()



sum()

min()

max()

prod()

mean()

Data preprocessing



8 NumPy

```
1 import numpy as np                                # numpy
2 a=np.array([[4, 4], [4, 4]])                      # a
3 print(' a      {}'.format(a))                    # a
4 print(' a  sqrt() {}'.format(np.sqrt(a)))         # a
5 print(' a  sin()  {}'.format(np.sin(a)))          # a
6 print(' a  sum()  {}'.format(np.sum(a)))          # a
7 print(' a  min()  {}'.format(np.min(a)))          # a
8 print(' a  max()  {}'.format(np.max(a)))          # a
9 print(' a  prod() {}'.format(np.prod(a)))         # a
10 print(' a  mean() {}'.format(np.mean(a)))         # a
11 print('square()   {}'.format(np.square(a)))       #
12 print('power()    {}'.format(np.power(a,3)))      # n
13 print('power()    {}'.format(np.isnan(a)))        #
>>>                                                # ( )
```

Data preprocessing



8 NumPy

()

```
>>> #  
a      [[4 4]  
[4 4]]  
a      sqrt()  [[2. 2.]  
[2. 2.]]  
a      sin()   [[-0.7568025 -0.7568025]  
[-0.7568025 -0.7568025]]  
a      sum()   16  
a      min()   4  
a      max()   4  
a      prod()  256  
a      mean()  4.0  
square()      [[16 16]  
[16 16]]  
power()       [[64 64]  
[64 64]]  
power()       [[False False]  
[False False]]
```

Data preprocessing



5. NumPy



NumPy

0

0

1

9

1	import numpy as np	# numpy
2	a=np.array([[1,1,2,2],[3,3,4,4]])	# a
3	print(' a {}'.format(a))	# a
4	print(' a[1,3] {}'.format(a[1,3]))	# a 3
5	a[1,3]=666	# a[1,3] 666
6	print(' a {}'.format(a))	# a
7	b=a[:2,:2]	# 0,1 0 1
8	print(' b {}'.format(b))	# b
>>> a [[1 1 2 2] [3 3 4 4]]		#
a[1,3] 4		
a [[1 1 2 2]		
[3 3 4 666]]		
b [[1 1]		
[3 3]]		

Data preprocessing



6.

NumPy

10

```
1 import numpy as np                                # numpy
2 a=np.array([[1,2,3],[4,5,6]])                      # a
3 print('  a  {}'.format(a))                        # a
4 print('  0  1      1  0  ')                      #
5 print(a[0][1])                                     # a[0][1]
6 print(a[1][0])                                     # a[1][0]
7 print('  0  1      1  0  ')                      #
8 print(a[[0,1],[1,0]])                             #
>>>  a  [[1 2 3] [4 5 6]]                          #
      0  1      1  0      2  4
      0  1      1  0
[2 4]
```

Data preprocessing



7.



True False

11

1	import numpy as np	#	numpy
2	a=np.array([[1,2],[3,4]])	#	a
3	print(' a {}'.format(a))	#	a
4	bol=(a>1)	#	
5	print(' {}'.format(bol))	#	a[0][1]
6	print(' 1 {}'.format(a[bol]))	#	a[1][0]
7	print(' 1 {}'.format(a[a<3]))	#	
>>>		#	
a [[1 2]			
[3 4]]			
[[False True]			
[True True]]			
1 [2 3 4]			
1 [1 2]			

Data preprocessing



8.



示例

```
import numpy as np
array_1d = np.arange(1, 10)
print(array_1d)
# 访问索引为[2,5,8]的元素
print(array_1d[[2, 5, 8]])
```

[1 2 3 4 5 6 7 8 9]

[3 6 9]

Data preprocessing



8.



示例

```
array_2d = np.arange(1, 10).reshape((3, 3))  
print(array_2d)  
# 访问索引为[0, 2]的元素  
print(array_2d[[0, 2]])
```

[[1 2 3]
 [4 5 6]
 [7 8 9]]

[[1 2 3]
 [7 8 9]]

array_2d[[0, 2]]

1	2	3
4	5	6
7	8	9

操作示意图

Data preprocessing



8.



“ [] ”

“ [] ”

示例

```
# 使用两个花式索引访问元素  
print(array_2d[[0, 2], [1, 1]])
```

[2 8]

array_2d[[0, 2], [1, 1]]

1	2	3
4	5	6
7	8	9

操作示意图

Data preprocessing



()



[]

```
1 import numpy as np          # numpy
2 arr_2d=np.array([[1,2,3],[4,5,6],[7,8,9]])
3 print(arr_2d)               #
4 print(arr_2d[:2])           #
5 print(arr_2d[:2,0:1])       #
6 print(arr_2d[:2, 1])       #
>>>
```

```
[[1 2 3]
 [4 5 6]
 [7 8 9]]
[[1 2 3]
 [4 5 6]]
[[1]
 [4]]
[2 5]
```

arr_2d[:2]

1	2	3
4	5	6
7	8	9

arr_2d[:2, 0:1]

1	2	3
4	5	6
7	8	9

arr_2d[:2, 1]

1	2	3
4	5	6
7	8	9

Data preprocessing



Pandas

◆ Pandas Python

SQL Excel

◆ Pandas Series DataFrame

Pandas

CSV JSON

NumPy Pandas



```
1 pip install pandas # pandas
2 import pandas # pandas
```

Data preprocessing



1. Series

◆ Pandas Series

◆ Series

◆ Series

◆ Series

`pandas.Series( data, index, dtype, name, copy)`

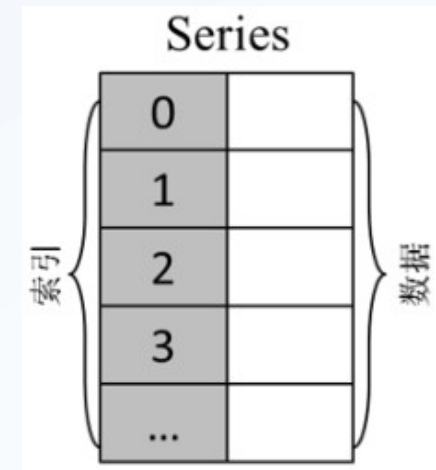
➤ data Series

➤ index 0

➤ dtype

➤ name Series name=None

➤ copy False



Data preprocessing



12

Series

```
1 import pandas as pd # pandas
2 a = [1, 2, 3] # a
3 ser = pd.Series(a) # a Series
4 print(ser) # ser
>>> #
0 1
1 2
2 3
dtype: int64
```

Series

Series

Data preprocessing



13

Series

```
1 import pandas as pd # pandas
2 print('      Series') #
3 tup=(1,2,3) #
4 s=pd.Series(tup) #      Series
5 print(s) # s
6 print('      Series') #
7 arr=np.array([1,2,3]) #
8 ys=pd.Series(arr) #      Series
9 print(ys) # ys
10 print('      Series') #
11 dic={"index0":1,"index1":2,"index2":3} #
12 ds=pd.Series(dic) #      Series
13 print(ds) # ds
14 print('      Series') #
15 bs=pd.Series(10) #      Series
16 print(bs) # bs
>>> # ( )
```

Data preprocessing



13

Series ()

```
>>> #
Series
0  1
1  2
2  3
dtype: int64
Series
0  1
1  2
2  3
dtype: int32
Series
index0  1
index1  2
index2  3
dtype: int64
Series
0  10
dtype: int64
```

Data preprocessing



Series

Series

14

Series

Series

```
1 import pandas as pd # pandas
2 a = [1, 2, 3] # a
3 print(' index,name dtype ') #
4 ser = pd.Series(data=a,index=['a','b','c'], #
5 name=' Series',dtype='float64') # name dtype
6 print(ser) # ser
>>> #
 index,name dtype
a 1.0
b 2.0
c 3.0
Name: Series, dtype: float64
```


Data preprocessing



15

Series

```
1 import pandas as pd                                # pandas
2 date_index=pd.to_datetime(['20220101','20220201','20220301']) #
3 date_ser=pd.Series([10,20,30],index=date_index)      # Series
4 print(date_ser)                                     #
>>>                                                  #
```

```
2022-01-01    10
2022-02-01    20
2022-03-01    30
dtype: int64
```

Data preprocessing



2. Series



“Series . ”

Series

Series

16 Series

```
1 import pandas as pd # pandas
2 a = [4,5,6] # a
3 print(' index,name dtype ') #
4 ser = pd.Series(data=a,index=['a','b','c'], #
5 name=' Series',dtype='float64') # name dtype
6 print(ser) # ser
7 print(' Series ') #
8 print('ser values:{}'.format(ser.values)) # ser
9 print('ser name:{}'.format(ser.name)) # ser name
10 print('ser index:{}'.format(ser.index)) # ser index
11 print('ser dtype:{}'.format(ser.dtype)) # ser
12 print(' mean() Series ') #
13 print(ser.mean()) # mean()
>>> # ( )
```

Data preprocessing



16 Series

()

```
>>> #  
      index,name  dtype  
a  4.0  
b  5.0  
c  6.0  
Name:      Series, dtype: float64  
      Series  
ser  values:[4. 5. 6.]  
ser  name:      Series  
ser  index:Index(['a', 'b', 'c'], dtype='object')  
ser  dtype:float64  
      mean()      Series  
5.0
```

Data preprocessing



3. DataFrame

◆ DataFrame

index

columns

Series

◆ DataFrame

◆ DataFrame

`pandas.DataFrame(data, index, columns, dtype, copy)`

◆ data DataFrame

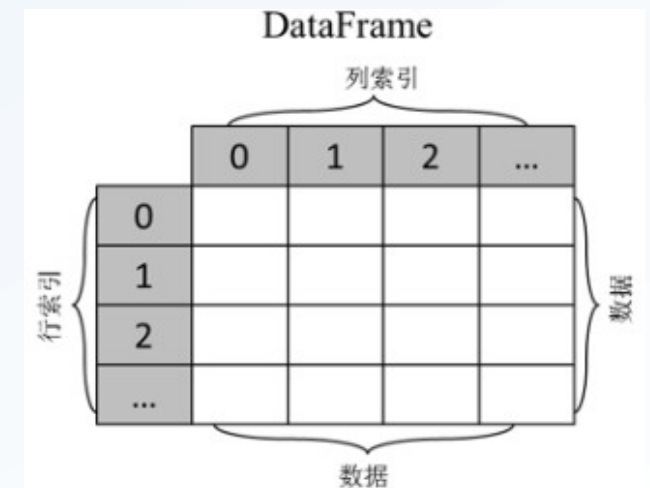
Series)

◆ index

◆ columns

◆ dtype

◆ copy False



Data preprocessing



17

DataFrame

```
1 import pandas as pd # pandas
2 print(' DataFrame') #
3 data1 = {'a': [1,2], 'b': [4,5], 'c': [7,8]} #
4 df1 = pd.DataFrame(data1) # DataFrame
5 print(df1) # df1

>>> DataFrame #
   a b c
0 1 4 7
1 2 5 8
```

18

Series

DataFrame

```
1 import pandas as pd # pandas
2 import numpy as np # numpy
3 print(' Series DataFrame') #
4 data2 = {'a': pd.Series(np.random.rand(3)), # Series
5          'b': pd.Series(np.random.rand(4)), # Series
6          'c': pd.Series(np.random.rand(5))} # Series
7 df2 = pd.DataFrame(data2) # DataFrame
8 print(df2) # df2

>>> Series DataFrame #
      a      b      c
0 0.014231 0.264698 0.620003
1 0.005264 0.199053 0.492979
2 0.812914 0.658674 0.764992
3      NaN 0.314128 0.188877
4      NaN      NaN 0.098748
```

Data preprocessing



19

DataFrame

```
1 import pandas as pd # pandas
2 import numpy as np # numpy
3 print(' DataFrame') #
4 df3=pd.DataFrame(np.random.randint(0,10,(3,5))) #
5 print(df3) # df3

>>> DataFrame #
   0 1 2 3 4
0 4 7 3 3 5
1 7 8 9 9 1
2 0 4 2 9 8
```

20

DataFrame

```
1 import pandas as pd # pandas
2 print(' DataFrame') #
3 data4 = [{'a':0,'b':1},{'a':2,'b':3}] #
4 df4 = pd.DataFrame(data4) # DataFrame
5 print(df4) # df4

>>> DataFrame #
   a b
0 0 1
1 2 3
```

Data preprocessing



21

DataFrame

```
1 import pandas as pd # pandas
2 import numpy as np # numpy
3 print('          DataFrame:') #
4 data5 = {'Xiaohua': {'Chinese': np.random.randint(60,100) , #
5                  'Math': np.random.randint(60,100),
6                  'English': np.random.randint(60,100)},
7         'Xiaoming': {'Chinese': np.random.randint(60,100), #
8                  'Math': np.random.randint(60,100),
9                  'English': np.random.randint(60,100)},
10        'Xiaoliang': {'Chinese': np.random.randint(60,100), #
11                  'Math': np.random.randint(60,100),
12                  'English': np.random.randint(60,100)},
13        }
14 df5 = pd.DataFrame(data5) # DataFrame
15 print(df5) # df5
```

```
>>>          DataFrame:
      Xiaohua Xiaoming Xiaoliang
Chinese      88       66       96
Math         73       70       61
English      68       73       89
```

Data preprocessing



4. DataFrame

22 DataFrame

```
1 import pandas as pd                                # pandas
2 df = pd.DataFrame({'col1':list(range(0,5)),          # DataFrame
3 'col2':range(5,10),'col3':range(10,15)})
4 print(" DataFrame ")                                #
5 print(df.iloc[0])                                   # df  0
6 print(" DataFrame ")                                #
7 print(df['col1'])                                    # df  'col1'
8 print(" df ")                                        #
9 print(type(df))                                     # df
10 print(" df.iloc[0] ")                               #
11 print(type(df.iloc[0]))                             # df.iloc[0]
12 print(" df['col1'] ")                               #
13 print(type(df['col1']))                             # df['col1']
>>>                                                  # ( )
```


Data preprocessing



22 DataFrame ()

```
>>> #  
    DataFrame  
col1    0  
col2    5  
col3   10  
Name: 0, dtype: int64  
    DataFrame  
0    0  
1    1  
2    2  
3    3  
4    4  
Name: col1, dtype: int64  
    df  
<class 'pandas.core.frame.DataFrame'>  
    df.iloc[0]  
<class 'pandas.core.series.Series'>  
    df['col1']  
<class 'pandas.core.series.Series'>
```

Data preprocessing



23 DataFrame

```
1 import pandas as pd # pandas
2 df = pd.DataFrame({'col1':list(range(0,3)), # DataFrame
3 'col2':range(3,6)})
4 df=df.rename(index={0:'a',1:'b',2:'c'}, # a b c
5 columns={'col1':'X','col2':'Y'}) # X Y
6 print(df) # df

>>>X Y #
a 0 3
b 1 4
c 2 5
```

24 DataFrame

```
1 import pandas as pd # pandas
2 df = pd.DataFrame({'col1':list(range(0,2)), # DataFrame
3 'col2':range(0,2)})
4 print(df.index) # df
5 print(df.columns) # df
6 print(df.values) # df
7 print(df.shape) # df
8 print(df.mean()) # df

>>>RangeIndex(start=0, stop=2, step=1) Index(['col1', 'col2'], dtype='object') #
[[0 0] [1 1]]
(2, 2)
col1 0.5 col2 0.5
dtype: float64
```

Data preprocessing



25 DataFrame

```
1 import pandas as pd                                # pandas
2 df = pd.DataFrame({'col1':list(range(0,2)),          # DataFrame
3 'col2':range(3,5),'col3':range(5,7)})
4 df['col4']=pd.Series(range(8,10))                   # df    "col4"
5 df1=pd.DataFrame({'col1':list(range(0,2)),          # df    df1
6 'col2':range(3,5),'col3':range(5,7),'col4':range(8,10)},
7 index=['a','b'])
8 df=df.append(df1)                                # df    df1
9 print("      df")
10 print(df)
11 print("    col3    ")
12 del df['col3']                                     # col3
13 print("    col3    df")
14 print(df)
15 print("      ")
16 df=df.drop(1)
17 print("      df")
18 print(df)
>>>                                                # ( )
```

Data preprocessing



25 DataFrame

()

```
>>>                                     #
      df
  col1 col2 col3 col4
0    0    3    5    8
1    1    4    6    9
a    0    3    5    8
b    1    4    6    9
      col3
      df
  col1 col2 col4
0    0    3    8
1    1    4    9
a    0    3    8
b    1    4    9
      df
  col1 col2 col4
0    0    3    8
a    0    3    8
b    1    4    9
```

Data preprocessing



5. DataFrame

◆ DataFrame df DataFrame

DataFrame	
df.head(N)	N
df.tail(M)	M
df[m: n]	m~n-1
df[df['>value'] df.query('>value')	
df.query('=[v1,v2,...]')	v1,v2,...
df.ix[: , 'colname']	colname
df.ix[row, col]	
df['col']	col Series

Data preprocessing



6. DataFrame

- ◆ DataFrame
- ◆ `df.duplicated(subset=None, keep='first'/'last'/'False')`
last False None first
- ◆ `df.drop_duplicates(subset=None, keep= 'first'/'last'/'False', inplace= False/True)`
inplace=False True df
- ◆ `df.fillna(value=None,method=None,axis=None,inplace=False, limit=None)`
NA NaN
- ◆ `df.drop(labels=None,axis=0, index=None,columns=None, inplace=False)`
DataFrame DataFrame
- ◆ `df.dropna(axis=0,how='any',thresh= None,subset=None, inplace=False)`
- ◆ `del df['col']`
df 'col'

Data preprocessing



6. DataFrame

- ◆ DataFrame
- ◆ `df.reindex(index=None, columns=None, fill_value='NaN')`
index columns
fill_value
- ◆ `df.replace(to_replace=None, value=None, inplace=False, limit=None, regex=False, method='pad')`
df value to_replace
- ◆ `df.merge(right, how='inner', on=None, left_on=None, right_on=None)` DataFrame
- ◆ `df.concat(objs, axis=0, join='outer', join_axes=None, ignore_index=False, keys=None)`
- ◆ `df.stak(level=-1, dropna=True)` df
- ◆ `df.unstak(level=-1, fill_value=None)` df

Data preprocessing



7. pandas

◆ [] loc iloc at iat Series DataFrame

1. []

◆ []

◆ []

◆ Series DataFrame “[]”

2. loc iloc

◆ .loc[] .iloc[]

◆ “loc[]” “iloc[]” DataFrame “loc[]” “iloc[]”

Data preprocessing



7. pandas

◆ [] loc iloc at iat Series DataFrame

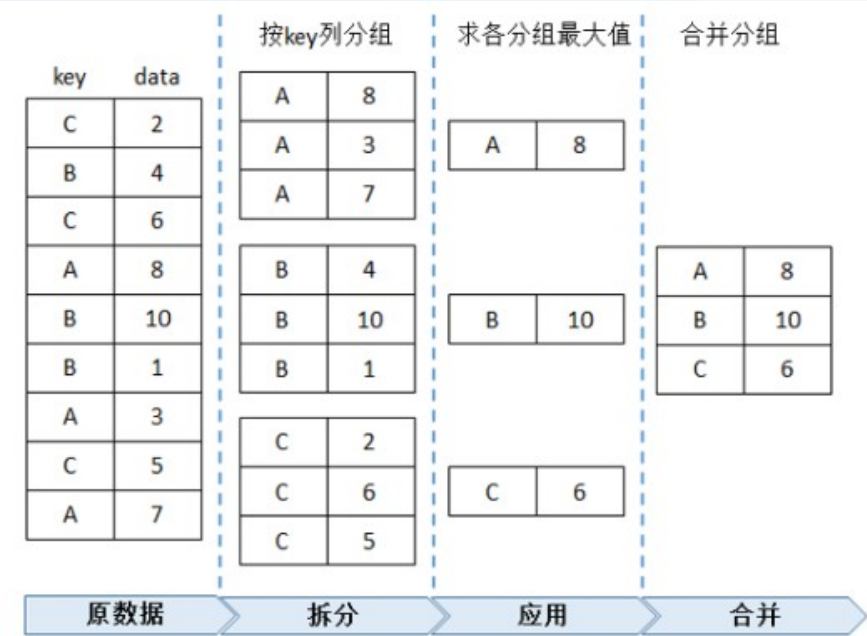
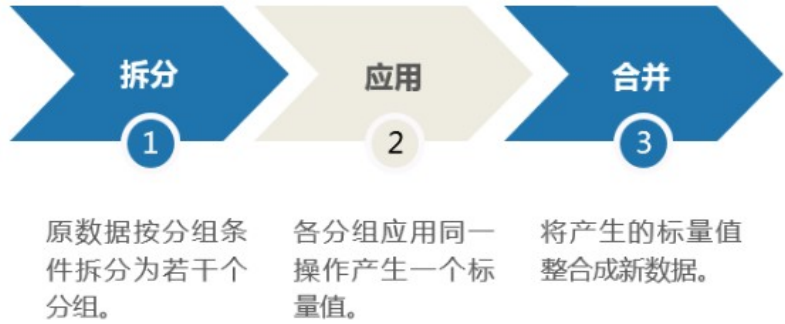
3. at iat

◆ DataFrame

◆ .at[,] .iat[,]

◆ “at[,]” ” iat[,]”

Data preprocessing



原数据首先经过拆分这一过程，按key列划分为3个组：值为A的数据归为一组，值为B的数据归为一组，值为C的数据归为一组，然后经过应用这一过程，对每个分组执行求最大值的操作，依次产出每个分组的最大值：8、10、6，最后经过合并这一过程，将各分组中有最大值的一行数据整合到一起。

Data preprocessing



1. groupby



groupby()



groupby



`DataFrame.groupby(by=None,axis=0,level=None,as_index=True,sort=True, squeeze = False,group_keys=True)`

◆ by

◆ axis axis=0 axis=1

◆ level None

◆ as_index True

◆ sort True

◆ squeeze

◆ Group_keys True

Data preprocessing



26 groupby

“ ”

```
1 import pandas as pd # pandas
2 df=pd.DataFrame({'': ['A','B','A','B'], # DataFrame
3                 '': [' ', ' ', ' ', ' '],
4                 '': [90,80,64,71],
5                 '': [94,81,62,51],
6                 '': [98,82,64,61],
7                 '': [' ', ' ', ' ', ' ']}
8                })
9 grouped = df.groupby(' ') #df “ ”
10 print(" df") #
11 print(df) # df
12 print(" ") #
13 print(grouped.get_group(' ' ' ')) # “ ”

>>> df #

0 A 90 94 98
1 B 80 81 82
2 A 64 62 64
3 B 71 51 61

0 A 90 94 98
1 B 80 81 82
```

Data preprocessing



27 groupby

```
1 import pandas as pd # pandas
2 df=pd.DataFrame({'A':['A','B','A','B'], # DataFrame
3                  'B':[' ',' ',' ',' '],
4                  'C':[90,80,64,71],
5                  'D':[94,81,62,51],
6                  'E':[98,82,64,61],
7                  'F':[' ',' ',' ',' ']}
8                })
9 print(' A B ') #
10 print(df.groupby(['A']).mean()) #
11 print("-----") #
12 print(' A B ') #
13 print(df.groupby(['A'])['C','D','E'].max()) #
14 print("-----") #
15 print(' A B ') #
16 print(df.groupby(['A'])['C','D','E'].min()) #
17 print("-----") #
18 print(' A B ') #
19 print(df.groupby(['A']).median()) #
>>> # ( )
```

Data preprocessing



27 groupby

()

```
>>> A B #  
  
A 77.0 78.0 81.0  
B 75.5 66.0 71.5  
-----  
A B  
  
A 90  
B 80  
-----  
A B  
  
A 64  
B 61  
-----  
A B  
  
A 77.0 78.0 81.0  
B 75.5 66.0 71.5
```

Data preprocessing



2.

1

- ◆ Pandas filter()
- ◆ `filter(items=None, like=None, regex=None, axis=None)`
 - ◆ `items` items
 - ◆ `like` like
 - ◆ `regex`
 - ◆ `axis` axis=0 axis=1

Data preprocessing



2.

2



◆ `transform(func,axis=0)`



`func`



`axis`

0

`func` 1

`func`

Data preprocessing



28

```
1 import pandas as pd                                # pandas
2 df=pd.DataFrame({'category':['A','B','A','B'],      # DataFrame
3                  'value':['1','2','3','4'],
4                  'score':[90,80,64,71],
5                  'rank':[94,81,62,51],
6                  'weight':[98,82,64,61],
7                  'label':['1','2','3','4']
8                  })
9 print("Category: A, Score: 80")                    #
10 en=df.groupby('category').filter(lambda x:(x['score']>80)) # en 80
11 print(en)                                           #
12 print('Category: A')                               #
13 print(df.groupby('category')['value'].head())      #
14 print("Category: A, Score: -10")                   #
15 print(df.groupby('category')['value'].transform(lambda # -10
16 x:x-10))
>>>                                                  # ( )
```

Data preprocessing



28

()

```
>>>          80          #
0 A      90 94 98
1 B      80 81 82

0      98
1      82
2      64
3      61
      -10

0 88
1 72
2 54
3 51
```

Data preprocessing



29

```
1 import pandas as pd # pandas
2 import numpy as np # numpy
3 df=pd.DataFrame([[1,2,3,4,5,6],[1,2,3,4,5,6], # DataFrame
4 [7,8,9,10,11,12],[7,8,9,10,11,12]],
5 columns=['a','b','c','d','e','f'])
6 print(" df") #
7 print(df) # df
8 print(" df1") #
9 df1=df[2:4][['d','f']] # df 2 3 d f
10 print(df1) # df1

>>> df
  a b c d e f
0 1 2 3 4 5 6
1 1 2 3 4 5 6
2 7 8 9 10 11 12
3 7 8 9 10 11 12
df1
d f
2 10 12
3 10 12
```

Data preprocessing





Python

NumPy

Pandas

Pandas

Series

DataFrame

函数/方法	说明
merge()	根据一个或多个键连接两组数据
merge_ordered()	通过可选的填充值/插值连接两组有序的数据（如时间序列）
merge_asof()	根据匹配最近的键连接两个DataFrame类对象（必须按键排序）
join()	根据行索引连接多组数据
concat()	沿着一条轴方向堆叠多组数据
append()	向数据末尾追加若干行数据
combine_first()	使用一个对象填充另一个对象中相同位置的缺失值



Data preprocessing



◆ `DataFrame.merge(left,right,how='inner',on=None,left_on=None,right_on=None, left_index=False,right_index=False,sort=False,indicator=False)`

◆ left, right Series DataFrame

◆ how inner() left right outer

➤ how inner left right

➤ left left

➤ right right

➤ outer left right

➤ on left right

➤ sort True

Data preprocessing



NaN

3 5

key

4 5

key

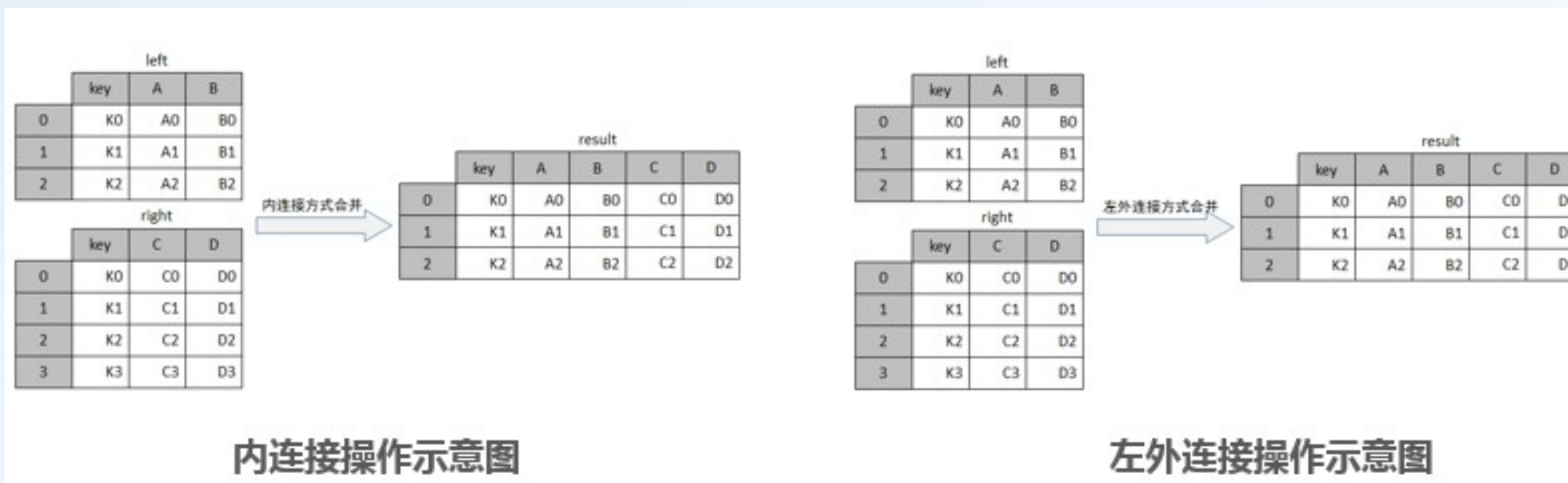
A B

3

C D

4

A B



Data preprocessing



◆ `pandas.concat(objs,axis=0,join='outer',join_axes=None, ignore_index=False, keys=None)`

➤ `join` `inner` `outer(     )`

➤ `Inner` `NaN`

➤ `outer` `NaN`

➤ `ignore_index` `True` `False (     )` `True`

Data preprocessing



- ◆ left right
NaN
- ◆ left right
left 3
NaN

left			
	key	A	B
0	K0	A0	B0
1	K1	A1	B1
2	K2	A2	B2

right			
	key	C	D
0	K0	C0	D0
1	K1	C1	D1
2	K2	C2	D2
3	K3	C3	D3

沿行方向合并

result					
	key	A	B	C	D
0	K0	A0	B0	NaN	NaN
1	K1	A1	B1	NaN	NaN
2	K2	A2	B2	NaN	NaN
0	K0	NaN	NaN	C0	D0
1	K1	NaN	NaN	C1	D1
2	K2	NaN	NaN	C2	D2
3	K3	NaN	NaN	C3	D3

采用外连接方式沿行方向合并示意图

left			
	key	A	B
0	K0	A0	B0
1	K1	A1	B1
2	K2	A2	B2

right			
	key	C	D
0	K0	C0	D0
1	K1	C1	D1
2	K2	C2	D2
3	K3	C3	D3

沿列方向合并

result						
	key	A	B	key	C	D
0	K0	A0	B0	K0	C0	D0
1	K1	A1	B1	K1	C1	D1
2	K2	A2	B2	K2	C2	D2
3	NaN	NaN	NaN	K3	C3	D3

采用外连接方式沿列方向合并示意图

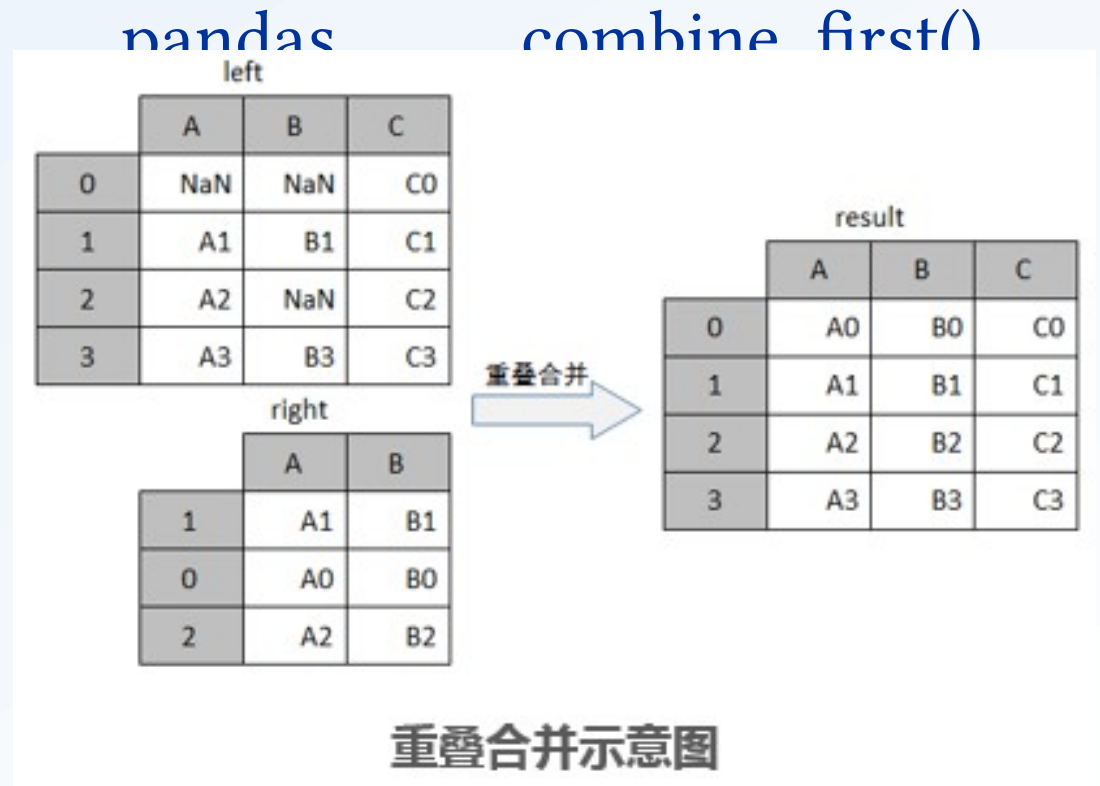
Data preprocessing



◆ `combine_first(other)`

◆ other

Series DataFrame



Data preprocessing



1. NumPy

◆ NumPy

append() concatenate()

1 **append()**

`np.append(arr, values, axis=None)`

- arr values
- values arr
- axis axis=0 axis=1

2 **concatenate()**

`concatenate((a1,a2,...), axis)`

- (a1,a2,...)
- axis 0

Data preprocessing



30 np.append() np.concatenate() NumPy

```
1 import numpy as np                                # numpy
2 print("np.append() ")                             #
3 print("    axis ")                                #
4 a=np.append([[1,2]],[[4,5],[7,8]])                #
5 print(a)                                           # a
6 print("    axis =0")                              # values
7 b=np.append([[1,2]],[[4,5],[7,8]],axis=0)          #
8 print(b)                                           # b
9 print("    axis =1")                              # values
10 c=np.append([[1,2],[1,2]],[[4,5],[7,8]],axis=1)    #
11 print(c)                                           # c
12 print("np.concatenate() ")                       #
13 c=np.array([1,2])                                 # c
14 d=np.array([5,6])                                 # d
15 e=np.concatenate((c,d),axis=0)                   # c d
16 print(e)                                           # e
>>>                                                # ( )
```

Data preprocessing



30 np.append() np.concatenate() NumPy ()

```
>>> np.append()                                     #
      axis
[1 2 4 5 7 8]
      axis =0
[[1 2]
 [4 5]
 [7 8]]
      axis =1
[[1 2 4 5]
 [1 2 7 8]]
np.concatenate()
[1 2 5 6]
```

Data preprocessing



2. Pandas

- ◆ Pandas concat() join() merge() append() combine()
- ◆ `pd.concat(objs,axis=0,join='outer'/'inner', join_axes=None)`

objs	Series	DataFrame	join	join_axes
------	--------	-----------	------	-----------
- ◆ `pd.merge(left,right,how='inner',on=None, left_on =None,right_on=None, left_index=False, right_index=False, sort=True)`

	left	DataFrame	right	DataFrame	how	inner	outer	left	right
inner	on	left_on	DataFrame	right_on	DataFrame		left_index	DataFrame	
	right_index	DataFrame	sort		True				

Data preprocessing



2. Pandas

- ◆ Pandas concat() join() merge() append() combine()
- ◆ pd.append(other,ignore_index=False) other DataFrame Series
ignore_index False True index
- ◆ df0.join(df1,how='inner',on=None) how inner outer left right
inner on
- ◆ df0.combine(df1,fun) fun DataFrame

Data preprocessing



31 Pandas

```
1 import pandas as pd # pandas
2 a=pd.DataFrame([1,2,3]) # DataFrame a
3 b=pd.DataFrame([6,6,6],index=['a','b','c']) # DataFrame b
4 print("append ") #
5 print(a.append(b,ignore_index=True)) # a b
6 print("merge ") #
7 df1=pd.DataFrame({'key':list('abc'), # DataFrame df1
8 'data1':range(3)})
9 df2=pd.DataFrame({'key':list('abcd'), # DataFrame df2
10 'data2':range(4)})
11 print(" how='inner'") #
12 print(pd.merge(df1,df2,how='inner')) # key
13 print(" left_on right_on") #
14 print(pd.merge(df1,df2,how='inner', # key
15 left_on='key',right_on='key')) # left_on right_on key
16 print(" left_index right_index") #
17 print(pd.merge(df1,df2,how='inner', # df1 data1
18 left_index=True,right_index=True)) # df2
>>> # ( )
```

Data preprocessing



31 Pandas

()

```
>>> append #
0
0 1
1 2
2 3
3 6
4 6
5 6
merge
  how='inner'
  key data1 data2
0 a 0 0
1 b 1 1
2 c 2 2
  left_on right_on
  key data1 data2
0 a 0 0
1 b 1 1
2 c 2 2
  left_index right_index
  key_x data1 key_y data2
0 a 0 a 0
1 b 1 b 1
2 c 2 c 2
```


Data preprocessing



32 combine()

```
1 import pandas as pd                                # pandas
2 import numpy as np                                  # numpy
3 x = pd.DataFrame({"A":[3,4],"B":[1,4]})             # DataFrame x
4 y = pd.DataFrame({"A":[1,2],"B":[5,6]})             # DataFrame y
5 x.combine(y,lambda a,b:np.where(a>b,a,b))          # combine

>>>
A B
0 3 5
1 4 6
```

Data preprocessing



Pandas

DataFrame

DataFrame

Series

Pandas

1.



stack() unstack()



stack()

Series

df.stack(level=-1,dropna=True) level

-1

0

dropna

True



unstack()

stack()

DataFrame

df.unstack(level=-1,fill_value=None)

level

stack()

fill_value

NaN

Data preprocessing



33

```
1 import pandas as pd                                # pandas
2 df=pd.DataFrame({'col1':['A','B','C'],              # DataFrame
3                  'col2':['A+','A+','B+']})
4 print("df")                                         #
5 print(df)                                           # df
6 print(" ")                                         #
7 df1=df.stack()                                     #
8 print(df1)                                         # df1
9 print(" ")                                         #
10 df2=df1.unstack()                                  #
11 print(df2)                                         # df2
>>>                                                # ( )
```

Data preprocessing



33

()

```
>>> df                                     #
0      A  A+
1      B  A+
2      C  B+

0
      A
      A+
1
      B
      A+
2
      C
      B+
dtype: object

0      A  A+
1      B  A+
2      C  B+
```

Data preprocessing



2.

◆ pivot()

DataFrame

◆ DataFrame.pivot(index=None,columns=None,values=None)

➤ index DataFrame

DataFrame

➤ columns DataFrame

DataFrame

➤ values DataFrame

Data preprocessing



34

```
1 import pandas as pd # pandas
2 df=pd.DataFrame({'date':['2021-01-01','2021-01-01','2021-01-01','2022-03-15',
3 '2022-03-15','2022-03-15'],
4 'product':['laptop','watch','smartphone','laptop','watch','smartphone']})
print(df)
df.pivot(index='date',columns='product',values='price')
```

商品名称	手提电脑	手表	智能手机
出售日期			
2021年1月1日	5899元	2200元	3999元
2022年3月15日	6099元	2116元	4099元

Data preprocessing



2.

◆ `melt()` `pivot()` `DataFrame`

◆ `DataFrame.melt(id_vars=None,value_vars=None,var_name=None,value_name='value',
col_level=None,ignore_index=True)`

➤ `id_vars`

➤ `value_vars`

➤ `value_name`

➤ `ignore_index` `True`

Data preprocessing



Pandas 3 Python None

NaN Not a Number NaT

1. None

◆ Pandas None None Python Object Python

35 None

1	import numpy as np	# numpy
2	import pandas as pd	# pandas
3	df=np.array([None,2])	# None
4	print(df)	# df
>>>[None 2]		#

Data preprocessing



2. NaN

◆ NaN

36 NaN

```
1 import numpy as np # numpy
2 import pandas as pd # pandas
3 df=np.array([np.nan,2,3]) # NaN
4 print(" df:") #
5 print(df) # df
6 print(" df ") #
7 print(type(df)) # df
8 print(" df ") #
9 for i in df: # df
10     print(" {} {}".format(i,type(i)))
11     print(" df +1.0 df ") #
12     for i in df: #df +1.0
13         i=i+1.0
14     print(" {} {}".format(i,type(i)))
15
16 >>> # ( )
```

Data preprocessing



36 NaN

()

```
>>> df #  
[nan 2. 3.]  
df  
<class 'numpy.ndarray'>  
df  
nan      <class 'numpy.float64'>  
2.0      <class 'numpy.float64'>  
3.0      <class 'numpy.float64'>  
df      +1.0      df  
nan      <class 'numpy.float64'>  
3.0      <class 'numpy.float64'>  
4.0      <class 'numpy.float64'>
```

Data preprocessing



3. NaT

◆ NaT

Pandas

np.nan

37 NaT

```
1 import numpy as np # numpy
2 import pandas as pd # pandas
3 df = pd.DataFrame({"name": ['Alfred', 'Batman', 'Catwoman'], # NaT
                     "toy": [np.nan, 'Batmobile', 'Bullwhip'],
                     "born": [pd.NaT, pd.Timestamp("1940-04-25"), pd.NaT]})
4 print(df) # df
>>> #
```

	name	toy	born
0	Alfred	NaN	NaT
1	Batman	Batmobile	1940-04-25
2	Catwoman	Bullwhip	NaT

Data preprocessing



4.

◆ Pandas

isnull() notnull() isna() notna()

dropna()

fillna()

interpolate()

38 isnull()

```
1 import pandas as pd                                # pandas
2 df = pd.read_csv('E:\\test\\06\\data\\data_processing.csv') #
3 print("  df:")                                       #
4 print(df)                                           # df
5 print('  df "FREQUENCY" ')                         #
6 print(df['FREQUENCY'].isnull())                     #
>>>                                                  # ( )
```

Data preprocessing



38 isnull() ()

```
>>> df: #
      ID ACCOUNT NAME FREQUENCY
0 100001000.0 104.0          3.0
1 100002000.0 197.0          3.0
2 100003000.0  NaN          NaN
3 100004000.0 201.0          1.0
4      NaN 203.0          3.0
5 100006000.0 207.0          NaN
6 100007000.0  NaN          2.0
7 100008000.0 213.0          1.0
8 100009000.0 215.0          NaN
df "FREQUENCY"
0 False
1 False
2  True
3 False
4 False
5  True
6 False
7 False
8  True
Name: FREQUENCY, dtype: bool
```

Data preprocessing



4.

◆ isnull() notnull() isna() notna()

True ; notnull() notna()

isnull() isna()

False

方法	说明
isnull()	若返回的值为True，说明存在缺失值
notnull()	若返回的值为False说明存在缺失值
isna()	若返回的值为True说明存在缺失值
notna()	若返回的值为False说明存在缺失值

Data preprocessing



◆ dropna()

df.dropna(axis=0, how='any', thresh=None, subset=None, inplace=False)

- axis 0 axis 1
- how any NaN how='all' NaN
- thresh
- subset
- inplace True False

删除缺失值前					删除缺失值后				
	A	B	C	D		A	B	C	D
0	1.0	3	5	7.0	0	1.0	3	5	7.0
1	2.0	4	6	5.0	1	2.0	4	6	5.0
2	NaN	4	7	NaN					
3	4.0	5	8	NaN					

Data preprocessing



39 dropna()

```
1 import pandas as pd                                # pandas
2 df = pd.read_csv('E:\\test\\06\\data\\data_processing.csv') #
3 df1 = df.dropna()                                   #
4 print(df1)                                           # df1

>>> #
      ID ACCOUNT NAME  FREQUENCY
0  100001000.0   104.0         3.0
1  100002000.0   197.0         3.0
3  100004000.0   201.0         1.0
7  100008000.0   213.0         1.0
```


Data preprocessing



40 dropna() ID

1	import pandas as pd	#	pandas
2	df = pd.read_csv('E:\\test\\06\\data\\data_processing.csv')	#	
3	df.dropna(subset=['ID'], inplace =True)	#	
4	print(df)	#	df
>>>ID ACCOUNT NAME FREQUENCY			
0	100001000.0	104.0	3.0
1	100002000.0	197.0	3.0
2	100003000.0	NaN	NaN
3	100004000.0	201.0	1.0
5	100006000.0	207.0	NaN
6	100007000.0	NaN	2.0
7	100008000.0	213.0	1.0
8	100009000.0	215.0	NaN

Data preprocessing



◆ `fillna()`

◆ `fillna()`

`df.fillna(value=None,method=None,axis=None,inplace=False,limit=None)`

- value
- method
 - None
 - ffill pad
 - bfill backfill
 - nearest
- axis
 - 0
 - axis 1
- inplace:
 - False
- limit
 - None

Data preprocessing



41 fillna() 666 df

1	import pandas as pd	#	pandas
2	df = pd.read_csv('E:\\test\\06\\data\\data_processing.csv')	#	
3	df.fillna(666, inplace = True)	#	df
4	print(df)	#	df

>>>	ID	ACCOUNT	NAME	FREQUENCY	#
0	100001	1000.0	104.0	3.0	
1	100002	2000.0	197.0	3.0	
2	100003	3000.0	666.0	666.0	
3	100004	4000.0	201.0	1.0	
4	666.0	203.0		3.0	
5	100006	6000.0	207.0	666.0	
6	100007	7000.0	666.0	2.0	
7	100008	8000.0	213.0	1.0	
8	100009	9000.0	215.0	666.0	

Data preprocessing



◆ Pandas mean() median() mode()

42

“ACCOUNT”

```
1 import pandas as pd                                # pandas
2 df = pd.read_csv('E:\\test\\06\\data\\data_processing.csv')  #
3 x = df["ACCOUNT"].mean()                               # "ACCOUNT"
4 df["ACCOUNT"].fillna(x, inplace = True)                #
5 print(df)                                              # df
```

>>>	ID	ACCOUNT NAME	FREQUENCY	#
0	100001000.0	104.000000	3.0	
1	100002000.0	197.000000	3.0	
2	100003000.0	191.428571	NaN	
3	100004000.0	201.000000	1.0	
4	NaN	203.000000	3.0	
5	100006000.0	207.000000	NaN	
6	100007000.0	191.428571	2.0	
7	100008000.0	213.000000	1.0	
8	100009000.0	215.000000	NaN	

Data preprocessing



43

“ACCOUNT”

1	import pandas as pd	#	pandas
2	df = pd.read_csv('E:\\test\\06\\data\\data_processing.csv')	#	
3	x = df["ACCOUNT"].median()	#	
4	df["ACCOUNT"].fillna(x, inplace = True)	#	
5	print(df)	#	df
>>>	ID ACCOUNT NAME FREQUENCY	#	
0	100001000.0 104.0 3.0		
1	100002000.0 197.0 3.0		
2	100003000.0 203.0 NaN		
3	100004000.0 201.0 1.0		
4	NaN 203.0 3.0		
5	100006000.0 207.0 NaN		
6	100007000.0 203.0 2.0		
7	100008000.0 213.0 1.0		
8	100009000.0 215.0 NaN		

Data preprocessing



44

“ACCOUNT”

```
1 import pandas as pd # pandas
2 df = pd.read_csv('E:\\test\\06\\data\\data_processing.csv') #
3 x = df["ACCOUNT"].mode() #
4 df["ACCOUNT"].fillna(x, inplace = True) #
5 print(df) # df
```



```
>>> ID ACCOUNT NAME FREQUENCY #
0 100001000.0 104.0 3.0
1 100002000.0 197.0 3.0
2 100003000.0 201.0 NaN
3 100004000.0 201.0 1.0
4 NaN 203.0 3.0
5 100006000.0 207.0 NaN
6 100007000.0 215.0 2.0
7 100008000.0 213.0 1.0
8 100009000.0 215.0 NaN
```

Data preprocessing



◆ interpolate()

◆ interpolate()

**df.interpolate(method='linear',axis=0,inplace=False,limit=None,limit_direction=None,
limit_area=None, downcast=None)**

- method linear() time index values nearest barycentric 6 linear
 time index values nearest barycentric
- axis 0 axis 1
- inplace: False
- limit None
- limit_direction

Data preprocessing



45 interpolate

```
1 import numpy as np                    # numpy
2 import pandas as pd                  # pandas
3 data1=[[1,3,5,7],[2,4,6,8],[np.nan,3,6,np.nan],[2,np.nan,5,8]]
4 df=pd.DataFrame(data1)               # DataFrame
5 df.interpolate(method='linear')      #
6 print(df)                           # df
>>>                                  #
```

	0	1	2	3
0	1.0	3.0	5	7.0
1	2.0	4.0	6	8.0
2	NaN	3.0	6	NaN
3	2.0	NaN	5	8.0

	0	1	2	3
0	1.0	3.0	5	7.0
1	2.0	4.0	6	8.0
2	2.0	3.0	6	8.0
3	2.0	3.0	5	8.0

Data preprocessing



1.



pandas : plot() boxplot() plot0 Series DataFrame
boxplot() DataFrame

◆ DataFrame.boxplot(column=None, by=None, ax=None, fontsize=None, rot=0, grid=True, figsize=None, layout=None, return_type=None, backend=None)

➤ rot

➤ grid

➤ return_type axes dict both

Data preprocessing



1.

46

```
1 import pandas as pd # pandas
2 import numpy as np # numpy
3 df = pd.read_csv('E:\\test\\06\\data\\data_outliers.csv') #
4 mean1 = df['tip'].quantile(q=0.25) #
5 mean2 = df['tip'].quantile(q=0.75) #
6 mean3 = mean2-mean1 #
7 topnum2 = mean2+1.5*mean3 #
8 bottomnum2 = mean2-1.5*mean3 #
9 print(" df 5 ") #
10 print(df.head()) # df 5
11 print(" ",topnum2,bottomnum2) #
12 print(" ",any(df['tip']>topnum2)) #
13 print(" ",any(df['tip']<bottomnum2)) #
>>> # ( )
```

Data preprocessing



46

()

```
>>> df 5 #
total_bill tip sex smoker day time size
0 16.99 1.01 Female No Sun Dinner 2
1 10.34 1.66 Male No Sun Dinner 3
2 21.01 3.50 Male No Sun Dinner 3
3 23.68 3.31 Male No Sun Dinner 2
4 24.59 3.61 Female No Sun Dinner 4
5.90625 1.21875
True
True
```

Data preprocessing



2.

47

```
1 import pandas as pd # pandas
2 import numpy as np # numpy
3 df = pd.read_csv('E:\\test\\06\\data\\data_outliers.csv') #
4 tipmean=df['tip'].mean() #
5 tipstd = df['tip'].std() #
6 topnum1 =tipmean+2*tipstd #
7 bottomnum1 = tipmean-2*tipstd #
8 print(" ",topnum1,bottomnum1) #
9 print(" ",any(tips['tip']>topnum1)) #
10 print(" ",any(tips['tip']<bottomnum1)) #
>>> 5.765555066526954 0.23100231052222542 #
True
False
```

Data preprocessing



3.



3

drop() replace()



drop()

◆ `DataFrame.drop(labels=None,axis=0,index=None,columns=None,level=None, inplace=False, error='raise')`

➤ labels

➤ axis 0 index 1 columns

➤ index

➤ columns

Data preprocessing



3.

◆ `replace()`

◆ `DataFrame.replace(to_replace=None, value=None, inplace=False, limit=None, regex=False, method='pad')`

➤ `to_replace`

➤ `value` `None`

➤ `method` `'pad'/'ffill'` `'bfill'`

Data preprocessing



3.

48

```
1 import pandas as pd # pandas
2 import numpy as np # numpy
3 df = pd.read_csv('E:\\test\\06\\data\\data_outliers.csv') #
4 print(" df ") #
5 print(df.shape) # df
6 tipmean=df['tip'].mean() #
7 tipstd = df['tip'].std() #
8 topnum1 =tipmean+2*tipstd #
9 bottomnum1 = tipmean-2*tipstd #
10 for index,row in df.iterrows(): # df
11     if row['tip']<bottomnum1: #
12         df.drop(index,inplace=True) #
13     if row['tip']>topnum1: #
14         df.drop(index,inplace=True) #
15 print(" df ") #
16 print(df.shape) # df

>>> df (244, 7) #
      df (234, 7)
```

Data preprocessing



49

```
1 import pandas as pd # pandas
2 import numpy as np # numpy
3 df = pd.read_csv('E:\\test\\06\\data\\data_outliers.csv') #
4 print(" df :") #
5 print(df.shape) # df
6 print("---") #
7 tipmean=df['tip'].mean() #
8 tipstd = df['tip'].std() #
9 topnum1 =tipmean+2*tipstd #
10 bottomnum1 = tipmean-2*tipstd #
11 print(" {}".format(bottomnum1)) #
12 print(" {}".format(topnum1)) #
13 min1=df['tip'][df['tip']>bottomnum1].min() #df"tip"
14 max1=df['tip'][df['tip']<topnum1].max() #df"tip"
15 print(" index:") #
16 for index,row in df.iterrows(): # df
17     if row['tip']<bottomnum1: #
18         print("low") # index
19     if row['tip']>topnum1: #
20         print(index) # index
21 print(" 23 47 " tip ") #
22 print("index=23,"tip" {}) #
23 .format(df.iloc[23].tip) #
24 print("index=47,"tip" {}) #
25 .format(df.iloc[47].tip) #
26 df.loc[df['tip']>topnum1,'tip']=max1 #
27 df.loc[df['tip']<bottomnum1,'tip']=min1 #
28 print(" df ") #
29 print(df.shape) # df
30 print(" index=23 47"tip" ") #
31 print("index=23,"tip" {}) #
32 .format(df.iloc[23].tip) #
33 print("index=47,"tip" {}) #
34 .format(df.iloc[47].tip) #
>>> # ( )
```


Data preprocessing



49

()

```
>>> df : #
(244, 7)
      0.23100231052222542
      5.765555066526954
      index:
23
47
59
88
141
170
183
212
214
239
      23 47 "tip"
index=23,"tip"      7.58
index=47,"tip"      6.0
      df
(244, 7)
      index=23 47"tip"
index=23,"tip"      5.65
index=47,"tip"      5.65
```

Data preprocessing





◆ Pandas

`df.duplicated()` `df.drop_duplicates()` `df.duplicated()`

`df.drop_duplicates()`

◆ `DataFrame.duplicated(subset=None,keep= 'first' )`

➤ subset

➤ keep

`first( )` `last` `False` `first` `last` `False`

➤ `duplicated()`

`Series` `True` `True`

Data preprocessing





pandas drop_duplicates()

◆ DataFrame.drop_duplicates(subset=None, keep= 'first ', inplace=False, ignore_index=False)

➤ subset

➤ keep first() last False first last
False

➤ inplace False

➤ ignore_index Flase

Data preprocessing



50 duplicated() drop_duplicates()

```
1 import pandas as pd                                # pandas
2 p1 = {                                              #
3     "name": [' ', ' ', ' ', ' ', ' ', ' '],
4     "age": [17, 18, 18, 20]
5 }
6 df = pd.DataFrame(p1)                              # DataFrame
7 print(" df ")                                       #
8 print(df.duplicated())                             # df
9 print(" df ")                                       #
10 df.drop_duplicates(inplace = True)                 # df
11 print(df)                                          # df
>>>                                                # ( )
```

Data preprocessing



50 duplicated() drop_duplicates() ()

```
>>> df
#
0 False
1 False
2 True
3 False
dtype: bool
df
name age
0 17
1 18
3 20
```

Data preprocessing





◆ Pandas

15 5 1

1 Timestamp

DatetimeIndex

2 Period

PeriodIndex

3 TimeDelta Duration

Data preprocessing



◆ Pandas

51 datetime()

1	from datetime import datetime	# datetime
2	t1=datetime(2022,10,20)	# datetime
3	print(t1)	# t1
>>>2022-10-20 00:00:00		#

Data preprocessing



52

Series

```
1 import numpy as np                                # numpy
2 from datetime import datetime                      # datetime
3 from pandas import Series,DataFrame               # Series,DataFrame
4 date_list=[                                       #
5     datetime(2021,11,1),
6     datetime(2021,11,10),
7     datetime(2021,11,12),
8     datetime(2021,11,15),
9     datetime(2021,12,21)
10 ]
11 s1=Series(np.random.rand(5),index=date_list)      # Series
12 print(s1)                                         # s1

>>>2021-11-01    0.247211
2021-11-10    0.837908
2021-11-12    0.221722
2021-11-15    0.624606
2021-12-21    0.841500
dtype: float64
```


Data preprocessing



53 datetime

```
1 import numpy as np                                # numpy
2 from datetime import datetime                      # datetime
3 from pandas import Series, DataFrame              # Series, DataFrame
4 date_list=[datetime(2021,12,1),datetime(2021,12,1) #
5 ,datetime(2021,11,12)]
6 s1=Series(np.random.rand(3),index=date_list)      # Series
7 print(" s1  0  ")                                #
8 print("s1  0  {}".format(s1[0]))                 #s1  0
9 print("s1      2021.12.1      \n{}".format(s1['2021.12.1']) # s1['2021.12.1']
10 .format(s1['2021.12.1']))
11 print(s1['20211201'])                             # s1['2021121']
12 print(" 2021 11  ")                              #
13 print(s1['2021-11'])                             # s1['2021-11']
14 print(" 2021  ")                                  #
15 print(s1['2021'])                                # s1['2021']
>>>                                                # ( )
```

Data preprocessing



53 datetime ()

```
>>> s1 0 #
s1 0 0.07289238972672285
s1 2021.12.1
2021-12-01 0.072892
2021-12-01 0.436728
dtype: float64
2021-12-01 0.072892
2021-12-01 0.436728
dtype: float64
2021 11
2021-11-12 0.595734
dtype: float64
2021
2021-12-01 0.072892
2021-12-01 0.436728
2021-11-12 0.595734
dtype: float64
```

Data preprocessing



Pandas

date_range()

DatetimeIndex

`pandas.date_range(start=None end=None periods=None freq=None)`

➤ start None

➤ end None

➤ periods None start end None

➤ freq “ 6H” 6

“ D”

Data preprocessing



54 2021-11-01 5

1	import pandas	#	pandas
2	d1=pd.date_range('2021-11-01', periods=5)	#	2021-11-01 5
3	print(d1)	#	d1
>>>DatetimeIndex(['2021-11-01', '2021-11-02', '2021-11-03', '2021-11-04', '2021-11-05'], dtype='datetime64[ns]', freq='D')		#	

55 2021-11-01 2021-11-05

1	import pandas	#	pandas
2	d1=pd.date_range(start='2021-11-01'	#	2021-11-01 2021-11-05
3	,end='2021-11-05')		
4	print(d1)	#	d1
>>>DatetimeIndex(['2021-11-01', '2021-11-02', '2021-11-03', '2021-11-04', '2021-11-05'], dtype='datetime64[ns]', freq='D')		#	

Data preprocessing



56

2021-11-01 6H

```
1 import pandas as pd                                # pandas
2 d1=pd.date_range(start='2021-11-01' ,              # 2021-11-01
3 periods=4,freq='6H')                               6H
4 print(d1)                                           # d1

>>> DatetimeIndex(['2021-11-01 00:00:00', '2021-11-01 06:00:00',
                    '2021-11-01 12:00:00', '2021-11-01 18:00:00'],
                    dtype='datetime64[ns]', freq='6H')
```

Data preprocessing



◆ Pandas

Period

Period

Period

57

Period

```
1 import pandas as pd                                # pandas
2 p = pd.Period(2021, freq='D')                      # Period
3 print("  p")                                       #
4 print(p)                                           # p
5 print("  p+2")                                     # p  +2
6 print(p+2)                                         # p+2
7 print("  p-2")                                     # p  -2
8 print(p-2)                                         # p-2

>>> p  2021-01-01                                #
      p+2  2021-01-03
      p-2
```

Data preprocessing



58 Period

```
1 import pandas as pd                                # pandas
2 p = pd.Period(2010, freq='Y')                       # Period
3 print(pd.Period('2021',freq='Y') - p)              #
>>> <11 * YearEnds: month=12>                       #
```

- ◆ Period 6-55 Period Y 11
- ◆ Period period_range()

Data preprocessing



59 2021-01

5

1	import pandas as pd	#	pandas	
2	q = pd.period_range("2021-01", periods=5, freq="Q")	#	Q	5
3	print(" ")	#		
4	print(q)	#	q	
5	y = pd.period_range("2021-01", periods=5, freq="Y")	#	Y	5
6	print(" ")	#		
7	print(y)	#	y	
8	m = pd.period_range("2021-01", periods=5, freq="m")	#	M	5
9	print(" ")	#		
10	print(m)	#	m	

```
>>>
PeriodIndex(['2021Q1', '2021Q2', '2021Q3', '2021Q4', '2022Q1'],
dtype='period[Q-DEC]')

PeriodIndex(['2021', '2022', '2023', '2024', '2025'], dtype='period[A-DEC]')

PeriodIndex(['2021-01', '2021-02', '2021-03', '2021-04', '2021-05'],
dtype='period[M]')
```


Data preprocessing





◆ Python

Pandas

Data preprocessing



◆ Python

- cat()
- split()
- get()
- join()
- contains()
- replace()
- repeat()
- pad()
- center()
- ljust()
- rjust()
- zfill()

0

Pandas

- count()
- startswith()
- endswith()
- findall()
- match()
- extract()
- len()
- strip()
- rstrip()
- lstrip()
- partition()
- rpartition()

Pandas

DataFrame

Data preprocessing



- wrap()
 - slice()
 - slice_replace()
 - rfind()
 - rindex()
 - swapcase()
 - isalnum()
 - isdigit()
 - islower()
 - istitle()
 - isdecimal()
- lower()
 - upper()
 - find()
 - index()
 - capitalize()
 - normalize()
 - isalpha()
 - isspace()
 - isupper()
 - isnumeric()

Data preprocessing



60 df “ d”

1	import pandas as pd	# pandas
2	df=pd.DataFrame(['1d','dce','abd','1'],columns=['val'])	# DataFrame
3	print(df[df['val'].str.contains('d')])	# 'd'
>>> val		#
0	1d	
1	dce	
2	abd	

61 df

1	import pandas as pd	# pandas
2	df=pd.DataFrame(['1','123','abd','123'],	# DataFrame
3	columns=['v1'])	#
4	print(df['v1'].str.isdecimal())	# df
>>>0 True		#
1	True	
2	False	
3	True	
Name: v1, dtype: bool		

Data preprocessing



62

str1

2

```
1 import pandas as pd # pandas
2 str1 = pd.Series([96, 98, 94]) # Series
3 index_ = [' ', ' ', ' '] # Index
4 str1.index = index_ # index
5 print(str1) # str1
6 result = str1.repeat(repeats = 2) # 2
7 print(result) # result

>>>
96
98
94
dtype: int64
96
96
98
98
94
94
dtype: int64
```

Data preprocessing



1. NLTK

◆ NLTK Natural Language Toolkit

Python

NLTK 3.0 documentation

[NEXT](#) | [MODULES](#) | [INDEX](#)

Natural Language Toolkit

NLTK is a leading platform for building Python programs to work with human language data. It provides easy-to-use interfaces to [over 50 corpora and lexical resources](#) such as WordNet, along with a suite of text processing libraries for classification, tokenization, stemming, tagging, parsing, and semantic reasoning, wrappers for industrial-strength NLP libraries, and an active [discussion forum](#).

Thanks to a hands-on guide introducing programming fundamentals alongside topics in computational linguistics, plus comprehensive API documentation, NLTK is suitable for linguists, engineers, students, educators, researchers, and industry users alike. NLTK is available for Windows, Mac OS X, and Linux. Best of all, NLTK is a free, open source, community-driven project.

NLTK has been called "a wonderful tool for teaching, and working in, computational linguistics using Python," and "an amazing library to play with natural language."

[Natural Language Processing with Python](#) provides a practical introduction to programming for language processing. Written by the creators of NLTK, it guides the reader through the fundamentals of writing Python programs, working with corpora, categorizing text, analyzing linguistic structure, and more. The book is being updated for Python 3 and NLTK 3. (The original Python 2 version is still available at http://nltk.org/book_1ed.)

Some simple things you can do with NLTK

Tokenize and tag some text:

```
>>> import nltk
>>> sentence = """At eight o'clock on Thursday morning
... Arthur didn't feel very good."""
>>> tokens = nltk.word_tokenize(sentence)
>>> tokens
['At', 'eight', 'o'clock', 'on', 'Thursday', 'morning',
 'Arthur', 'didn', 't', 'feel', 'very', 'good', '.']
>>> tagged = nltk.pos_tag(tokens)
>>> tagged[0:6]
[('At', 'IN'), ('eight', 'CD'), ('o'clock', 'JJ'), ('on', 'IN'),
 ('Thursday', 'NNP'), ('morning', 'NN')]

Identify named entities:
```

```
>>> entities = nltk.chunk.ne_chunk(tagged)
>>> entities
Tree('S', [Tree('NP', [Tree('IN', 'At'), Tree('CD', 'eight'), Tree('JJ', 'o'clock')])])
```

TABLE OF CONTENTS

- NLTK News
- Installing NLTK
- Installing NLTK Data
- Contribute to NLTK
- FAQ
- Wiki
- API
- HOWTO

SEARCH

Data preprocessing



63 NLTK

```
1 import nltk # nltk
2 text = nltk.word_tokenize( "I love you 3000 times" ) #
3 print (text) #
>>> ['I', 'love', 'you', '3000', 'times'] #
```

64 NLTK

```
1 import nltk # nltk
2 all_words = nltk.FreqDist(w.lower() for w in #
3 nltk.word_tokenize( "I'm Iron Man man" )) #
4 for key in all_words: #
5 print(key, all_words[key]) #
>>>man 2 #
i 1
'm 1
iron 1
```

2. jieba

◆ jieba

◆ jieba 3 :

1

2

3

Data preprocessing



2. jieba

◆ jieba

<code>jieba.cut(s)</code>	
<code>jieba.cut(s,cut_all=True)</code>	s
<code>jieba.cut_for_search(s)</code>	
<code>jieba.lcut(s)</code>	
<code>jieba.lcut(s,cut_all=True)</code>	
<code>jieba.lcut_for_search(s)</code>	
<code>jieba.add_word(W)</code>	W
<code>jieba.del_word(W)</code>	W

Data preprocessing



65 jieba

```
1 import jieba                                # jieba
2 str="                                         #"
3 print(jieba.lcut(str))                       #

>>>                                           #

[' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ']
```

Data preprocessing



66 jieba

```
1 import jieba # jieba
2 def stopwordslist(): #
3     stopwords=[line.strip() for line
4 in open('E:\\test\\06\\data\\cn_stopwords.txt ',
5 encoding='UTF-8').readlines()]
6     return stopwords
7 def seg_depart(sentence): #
8     sentence_depart = jieba.cut(sentence.strip()) #
9     stopwords = stopwordslist() #
10    outstr = '' # ostr
11    for word in sentence_depart: #
12        if word not in stopwords:
13            if word != '\t':
14                outstr += word
15                outstr += " "
16    return ostr
17 str=" " #
18 str1=seg_depart(str) #
19 print(str1) # str1
>>> #
```

Data preprocessing



67 jieba

```
1 import jieba.posseg as psg # jieba.posseg
2 str="《》" #
3 psg.lcut(str) #

>>>[pair(' ', 'n'), pair('《', 'x'), pair(' ', 'ns'), pair('》', 'x'), pair(' ', 'v'), pair(' ', #
'ns'), pair(' ', 'n'), pair(' ', 'n'), pair(' ', 'n')]
```

◆ jieba

a	
c	
n	
x	
ns	
v	

Data preprocessing



3. sklearn

◆ sklearn Python

numpy

◆ K DBSCAN

Data preprocessing



68 sklearn

```
1 from sklearn import datasets #
2 from sklearn.linear_model import LinearRegression #
3 loaded_data=datasets.load_boston() #
4 data_X=loaded_data.data #
5 data_y=loaded_data.target #
6 model=LinearRegression() #
7 model.fit(data_X,data_y) #
8 print("      4      ") #
9 print(model.predict(data_X[:4,:])) # 4
10 print("      4      ") #
11 print(data_y[:4]) # 4

>>> #
      4
[30.00384338 25.02556238 30.56759672 28.60703649]
      4
[24. 21.6 34.7 33.4]
```

4. Gensim

◆ Gensim
doc2vec

TF-IDF LSA LDA word2vec

69 Gensim

```
1 from gensim import corpora # gensim
2 texts = [['human', 'computer', 'human', 'computer', 'nice']] #
3 dictionary = corpora.Dictionary(texts) #
4 corpus = [dictionary.doc2bow(text) for text in texts] #
5 print(corpus) #
>>> #
[[ (0, 2), (1, 2), (2, 1) ]]
```

Data preprocessing



Python

➤ .	1	\n	)	{m}	m
➤ []	[]			{m,}	m
➤ \d				{m,n}	m n
➤ \D				^	
➤ \s	Tab			\$	
➤ \S				\b	
➤ \w				\B	
➤ \W					
➤ *	0			(ab)	
➤ +:	1	1		\num	num
➤ ?	1	0		(?P<name>)	

Data preprocessing



◆ Python re 4 match() search() findall() compile()
70

```
1 import re # re
2 a = re.match('...', 'Test Test') #
3 print(a.group()) #
>>> Tes #
```

◆ re.match() group() span()
re.match() None

Data preprocessing



71

1	import re	#	re
2	a = re.match('\d\d','666qq')	#	
3	print(a.group())	#	
>>>66		#	

72

1	import re	#	re
2	a = re.match('12[234]','124qqqqq')	#	122 123 124
3	print(a.group())	#	
>>>124		#	

73

“ a”

1	import re	#	re
2	print(re.match('a+','aaaqwqewq').group())	#	a a 1
3	print(re.match('a+','aabbads').group())	#	a a 1
>>>aaa		#	
aa			

Data preprocessing



74

“qa{n}” n

1	import re	#	re	
2	print(re.match('qa{4}','qaaaaazz').group())	#	qaaaa	a
3	print(re.match('qa{3}','qaabbb'))	#	qaa	
>>>qaaaa		#		
None				

75

“d”

1	import re	#	re	
2	print(re.match('.*d\$','4sdafqd'))	#	d	
3	print(re.match('.*d\$','4saaaadc'))	#	d	none
>>>4sdafqd		#		
None				

Data preprocessing



76

11 1 3 5 6 7 8 9

1	import re	# re
2	pattern = re.compile(r"1[356789]\d{9}")	#
3	strs = ' 15467692841'	# 1
4	result = pattern.findall(strs)	# 1
5	strs1 = ' 12467692841'	# 2
6	result1 = pattern.findall(strs1)	# 2
7	print(" {}".format(result))	#
8	print(" {}".format(result1))	# 2 2
>>> [15467692841]		#
[]		

77

@

1	import re	# re
2	sre="[a-zA-Z0-9_-]+@[a-zA-Z0-9_-]+(?:\.[a-zA-Z0-9_-]+)"	#
3	pattern = re.compile(sre)	# sre
4	strs = ' xiaojinqw@163.com'	#
5	result = pattern.findall(strs)	#
6	print(result)	#
>>> [xiaojinqw@163.com]		#

Data preprocessing



正则表达式 - 教程

正则表达式(Regular Expression)是一种文本模式，包括普通字符（例如，a 到 z 之间的字母）和特殊字符（称为“元字符”），可以用来描述和匹配字符串的特定模式。

正则表达式是一种用于模式匹配和搜索文本的工具。

正则表达式提供了一种灵活且强大的方式来查找、替换、验证和提取文本数据。

正则表达式可以应用于各种编程语言和文本处理工具中，如 JavaScript、Python、Java、Perl 等。

入门视频：[正则表达式入门教程](#)。

实例

以下实例从字符串 str 中找出数字：

实例

从字符串 str 中提取数字部分的内容(匹配一次)：

```
var str = "abc123def";  
var patt1 = /[0-9]+/;  
document.write(str.match(patt1));
```

以下标记的文本是获得的匹配的表达式：

123

尝试一下 »

测试工具

修饰符： g

[0-9]+

匹配文本：

abc123def

<https://www.runoob.com/regexp/regexp-tutorial.html>



How can I help you today?

Data preprocessing





5

1

2

3

4

5

Data preprocessing



78

```
1 import re # re
2 import jieba # jieba
3 from gensim import corpora, models # gensim
4 from sklearn.feature_extraction.text import TfidfVectorizer # sklearn
5 str=' %$$ @ # ()++*dfsafdsaww' #
6 print(" str:") #
7 print(str) # str
8 def process_te(data): #
9     str1=re.sub('\W', '', data)
10     return str1
11 print(" str: ") #
12 str_q=process_te(str) #
13 print(str_q) #
14 print(" ") #
15 def is_chinese(char1): #
16     if char1>=u'\u4e00' and char1<= u'\u9fa5':
17         return True
18     else:
19         return False
20 def format_s(content): #
21     str1=""
22     for i in content:
23         if is_chinese(i):
24             str1=str1+i
25     return str1
26 str_c=format_s(str_q) #
27 print(str_c) #
28 print(" ") #
29 def fenc(data): #
30     word1=jieba.cut(data)
31     return list(word1)
32 fen_word=fenc(str_c) #
33 print(fen_word) # " " " " " " "
34 print(" ") #
35 file_userdict = ' E:\\test\\06\\data\\userdict.txt '
```

Data preprocessing



78

()

```
36 jieba.load_userdict(file_userdict) #
37 fen_word1=fenc(str_c) #
38 print(fen_word1) #
39 fen_word1_str = ".join(fen_word1) #
40 print(" ") #
41 def stopwordslist(): #
42 stopwords = [line.strip() for line in open('E:\\test\\06\\data\\cn_stopwords.txt',encoding='UTF-8').readlines()]
43 return stopwords
44 def seg_depart(sentence): #
45 sentence_depart = jieba.cut(sentence.strip()) #
46 stopwords = stopwordslist() #
47 outstr = [] # outstr
48 for word in sentence_depart: #
49 if word not in stopwords:
50 if word != '\t':
51 outstr.append(word)
52 return outstr
53 fen_word2=seg_depart(fen_word1_str) #
54 print(fen_word2) #
55 print(" ") #
56 tfidf_vec = TfidfVectorizer() # TfidfVectorizer
57 tfidf_matrix = tfidf_vec.fit_transform(fen_word2) #
58 print(" ") #
59 print(tfidf_vec.get_feature_names()) #
60 print(" id ") #
61 print(tfidf_vec.vocabulary_) # id
62 print(" ") #
63 print(tfidf_matrix.toarray()) # , id
```

>>>

#

()

[illegible]

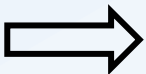


5. Data visualization

Data visualization



序号	工作中心	物料名称	需求量(个)	可生产时间(M)	触发时间
CP01	标准立式双速圆锥电动机	海研变频器	607	15M	2018/1/14 8:00
CP02	标准立式双速圆锥电动机	海研变频器	607	30M	2018/1/14 8:00
CP03	标准立式双速圆锥电动机	海研变频器	607	45M	2018/1/14 8:00
CP04	标准立式双速圆锥电动机	海研变频器	607	45M	2018/1/14 8:00
CP05	标准立式双速圆锥电动机	海研变频器	607	50M	2018/1/14 8:00
CP06	标准立式双速圆锥电动机	海研变频器	607	54M	2018/1/14 8:00
CP07	标准立式双速圆锥电动机	海研变频器	607	65M	2018/1/14 8:00
CP08	标准立式双速圆锥电动机	海研变频器	607	69M	2018/1/14 8:00
CP09	标准立式双速圆锥电动机	海研变频器	607	70M	2018/1/14 8:00
CP10	标准立式双速圆锥电动机	海研变频器	607	80M	2018/1/14 8:00
CP11	标准立式双速圆锥电动机	海研变频器	607	90M	2018/1/14 8:00
CP12	标准立式双速圆锥电动机	海研变频器	607	93M	2018/1/14 8:00



Data visualization





?



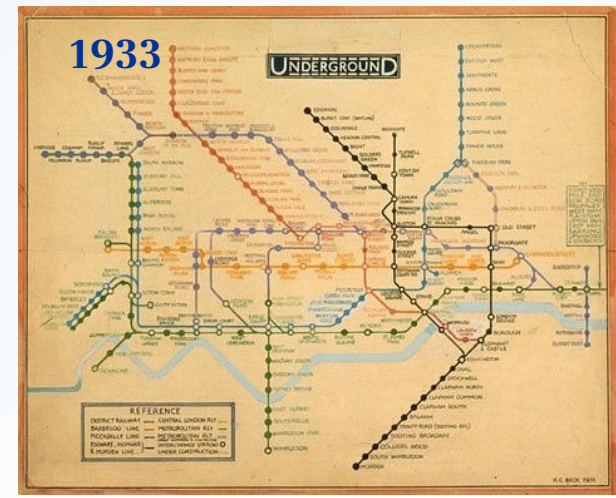
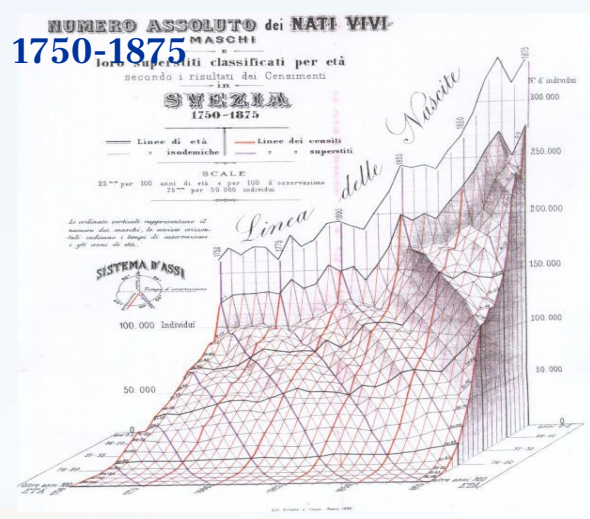
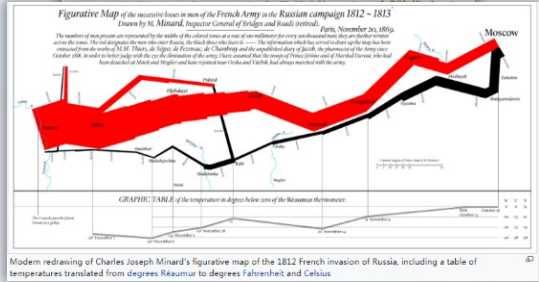
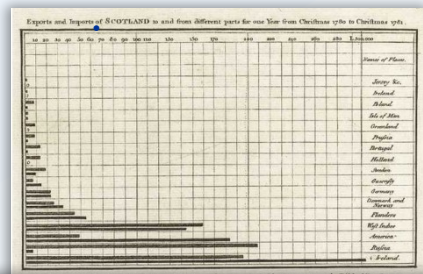
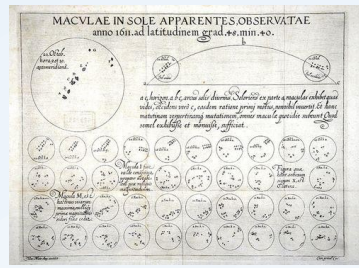
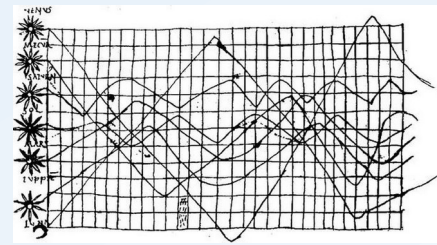
--- . <<>>

Data visualization



- 17
- 17
- 18
- 19
- 19
- 20
- 1950 — 1974
- 1975 — 2011
- 2012 —

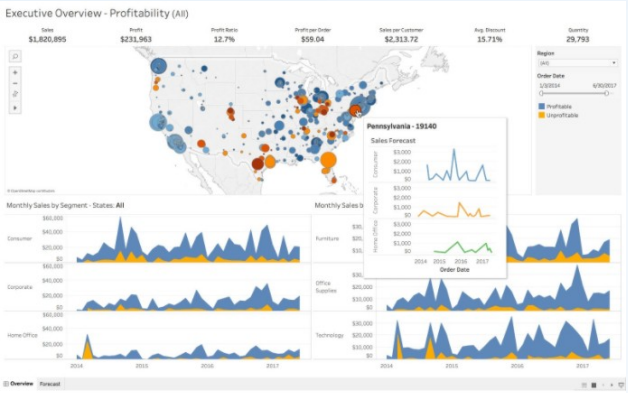
10



Data visualization



(2012 -)



Tableau

Data visualization



“ ” “ ”



1



60 000

2



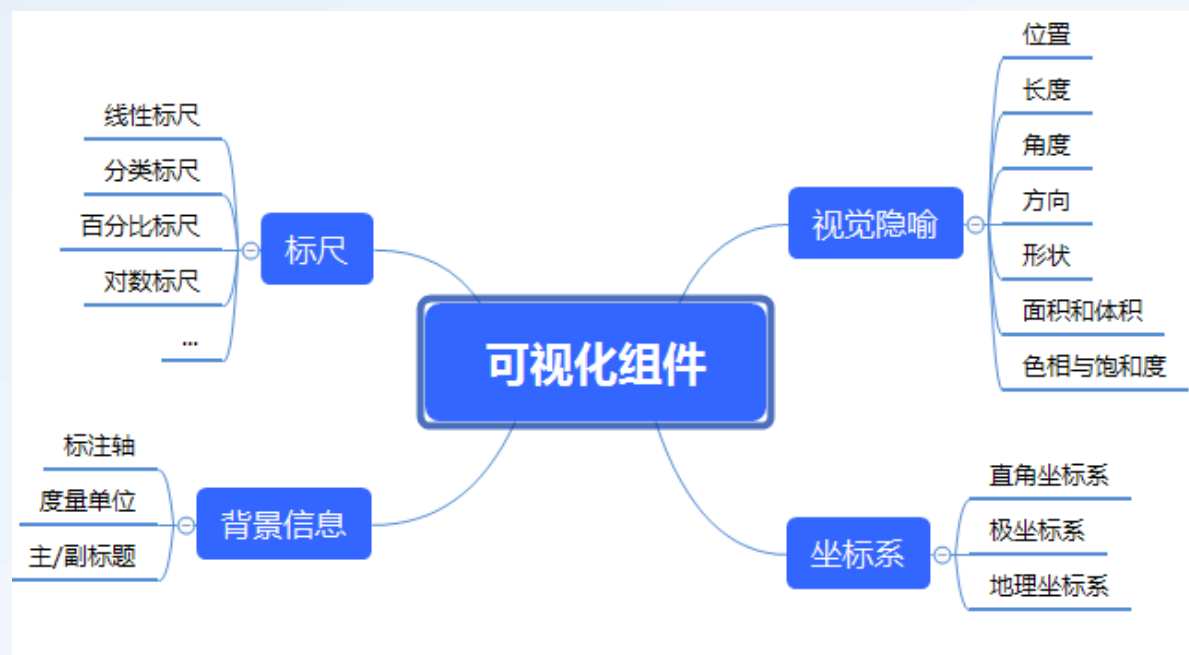
3



Data visualization



1.



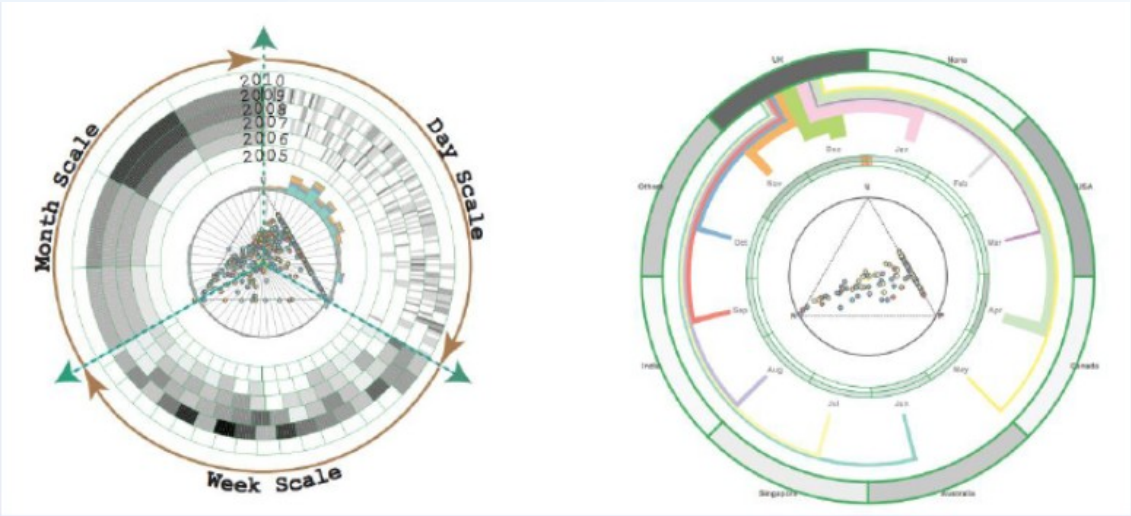
Data visualization



1

◆ (Metaphor)

◆ / ()



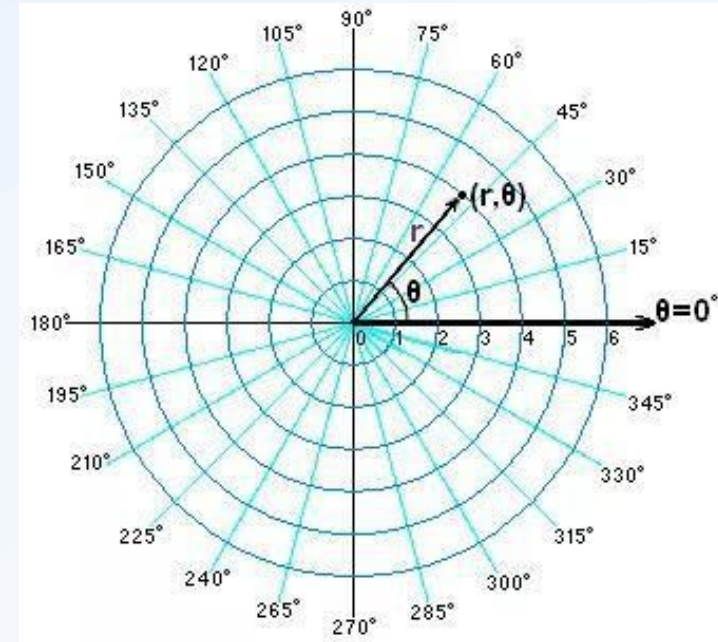
Data visualization



2



, ()



Data visualization



3



Data visualization



4



5W



Data visualization



2.



Data visualization

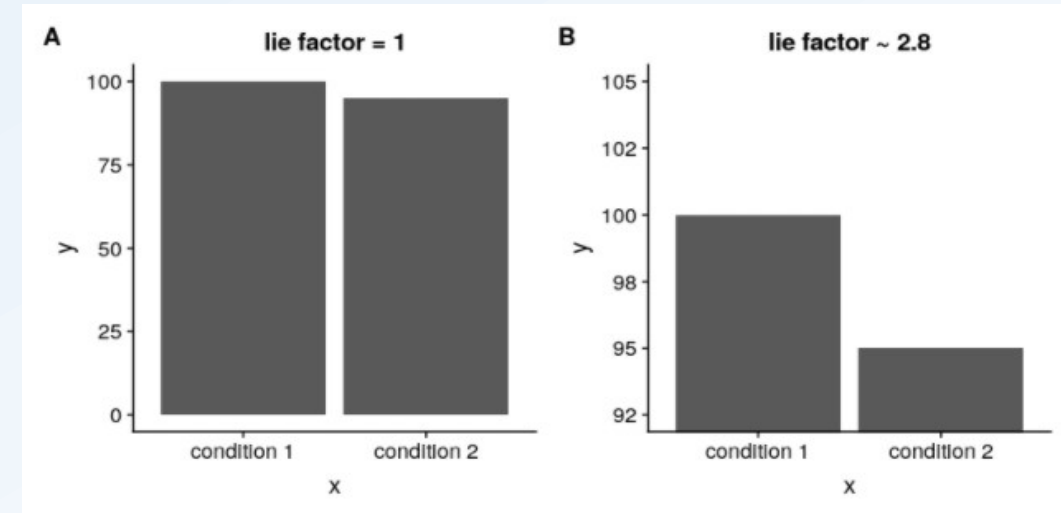


2.

(1



.



$$LF = \frac{\text{数据所对应的图形元素的相对变化量}}{\text{数据的真实变化量}}$$

LF

Lie Factor

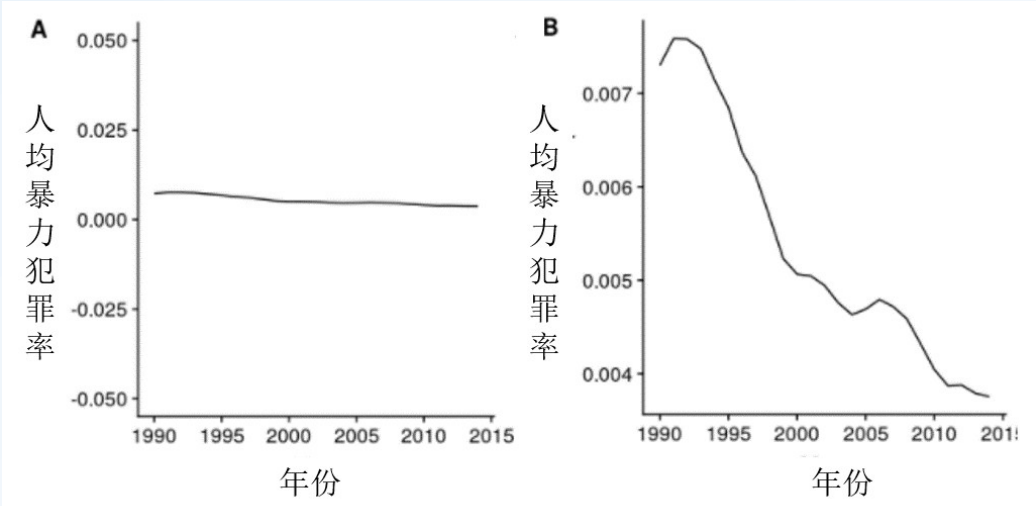
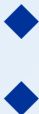
LF=1

LF

[0.95, 1.05]

2.

1



A

B

2.

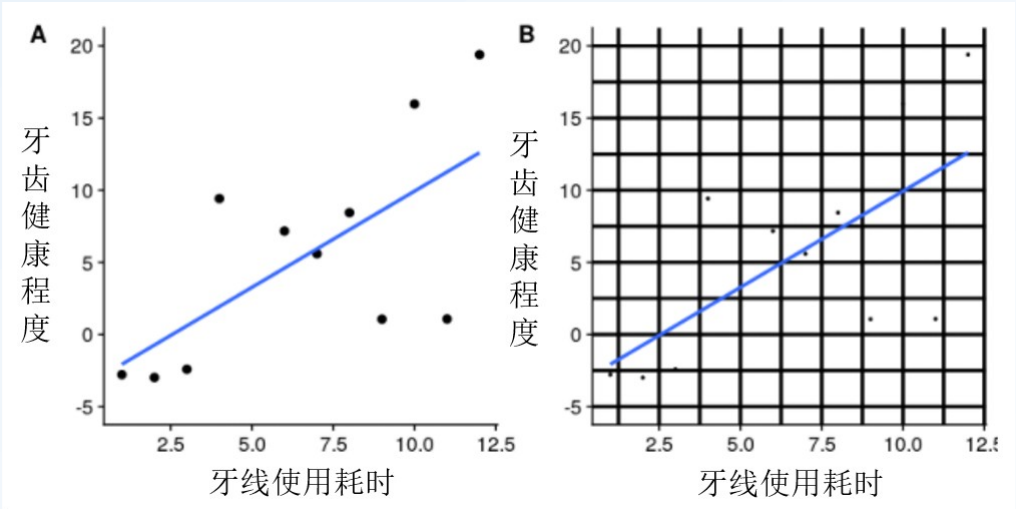
2



1983



“ ”



数据墨水

$$\text{data - ink ratio} = \frac{\text{数据墨水}}{\text{绘制可视化图形使用的所有墨水}}$$

Data visualization

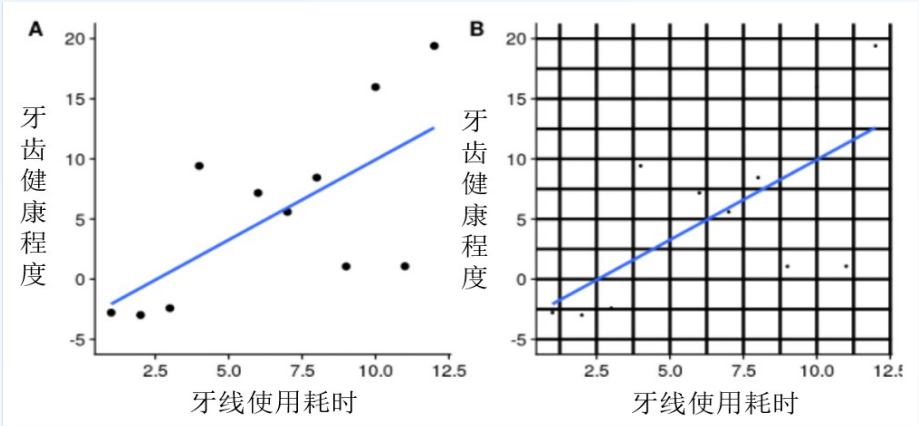


2.

2



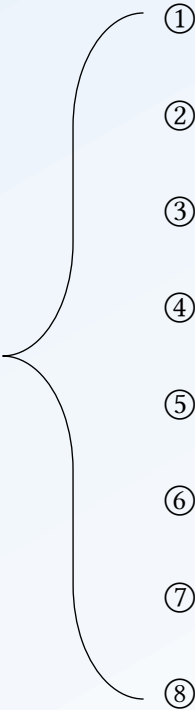
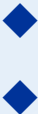
2





2.

2



Data visualization



2.

3

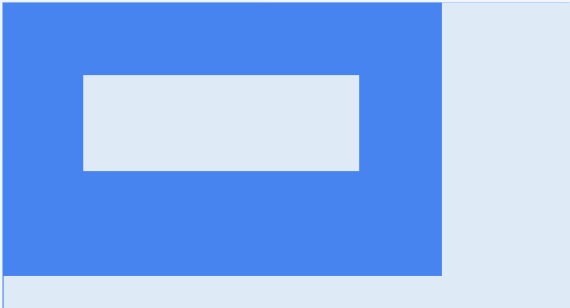
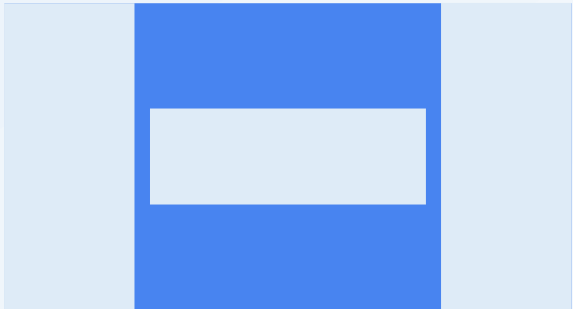


+ /





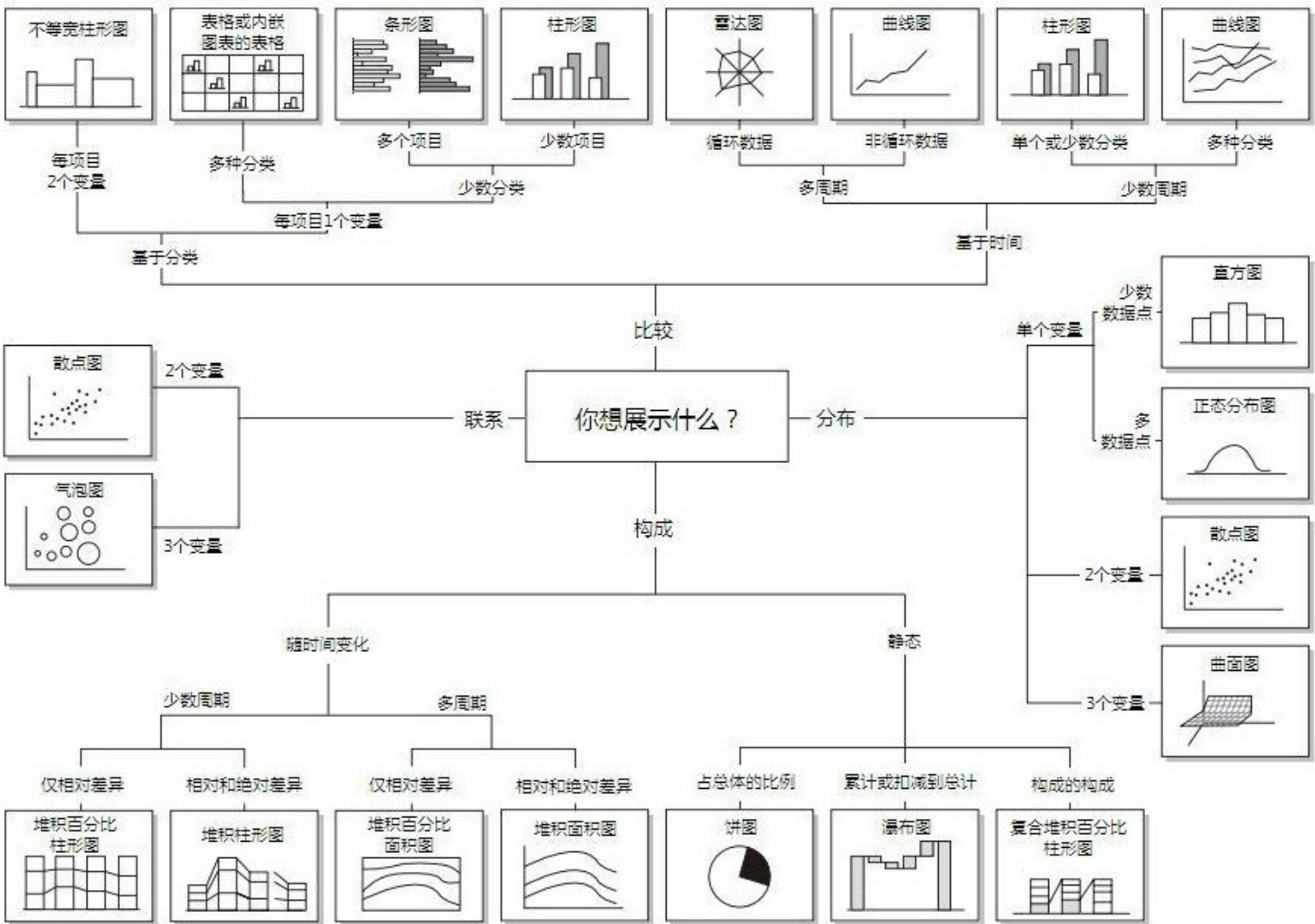
3.



Data visualization



图表建议—思维指南



Data visualization



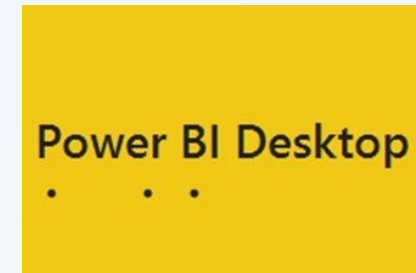
1.



Excel



Tableau



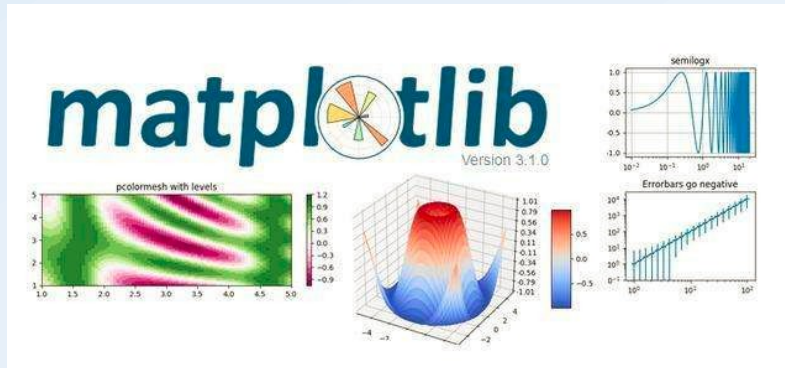
Power BI

Power BI
(BI)

Data visualization



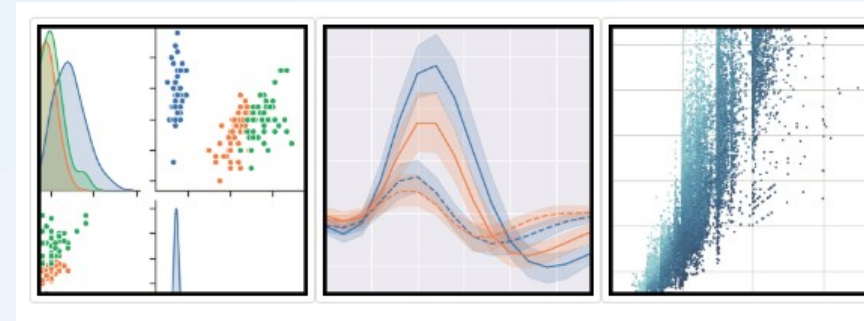
2.



Matplotlib Python

Numpy

Matlab



Seaborn

matplotlib

python

numpy pandas

scipy

statsmodels



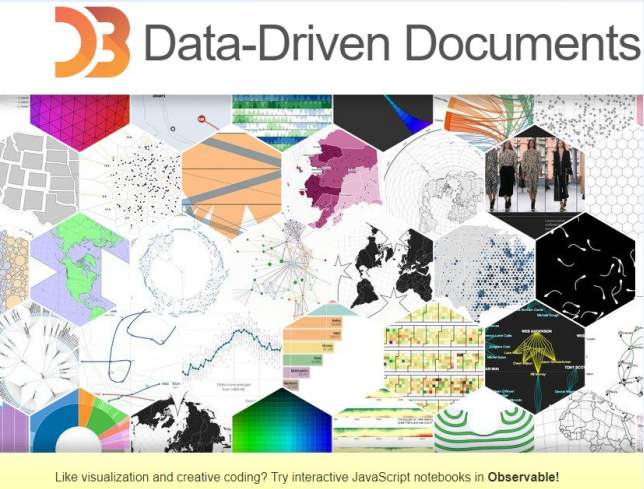
R

R

Data visualization



3. Web



D3 Data-Driven Documents D3.js
JavaScript D3

Web
DOM

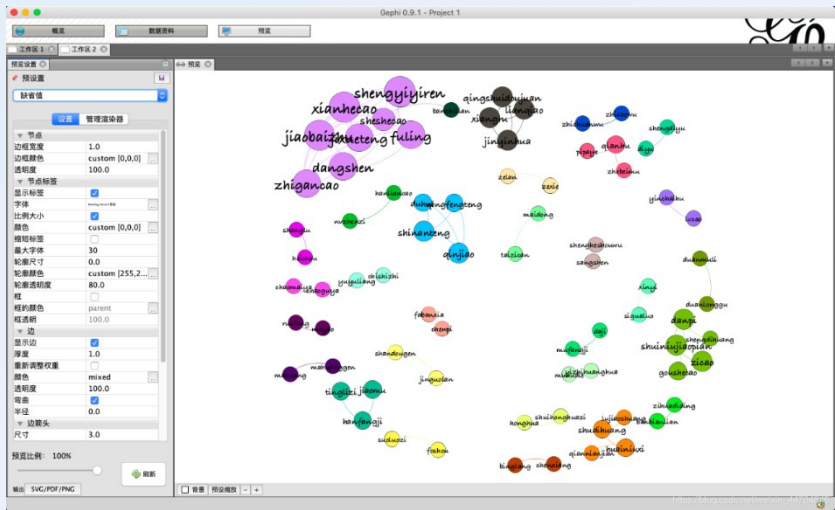


ECharts JavaScript

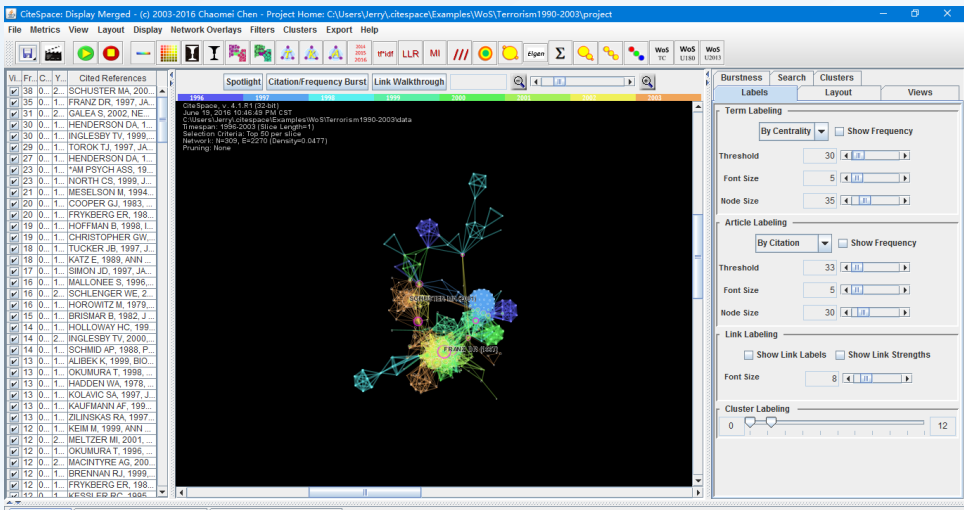
Data visualization



4.



Gephi



CiteSpace

“ ”

“ ”

<https://www.bilibili.com/video/BV12J411Y7EY/>

Data visualization



D3.JS

D3 <https://d3js.org/>

D3 : <https://github.com/d3/d3/wiki>

:

: <https://juejin.im/post/5aa0d5716fb9a028da7c1a4b#heading-0>

:

: 《D3js:

Echarts

: <https://echarts.apache.org/handbook/zh/get-started/>

: <https://echarts.apache.org/examples/zh/index.html>

<https://echarts.apache.org/zh/events.html>

Issue : <https://github.com/apache/incubator-echarts/issues>

Data visualization



Tableau

Tableau : <https://www.tableau.com/zh-cn>

W3Cschool Tableau : <https://www.w3cschool.cn/tableau/>

: <https://public.tableau.com/en-us/s/galleryTableau>

: <https://community.tableau.com/welcome>

: 《 Tableau Your Data 》《 Communicating Data with Tableau 》

Power BI

Power BI <https://powerbi.microsoft.com/zh-cn/>

Power BI <http://www.chinapowerbi.com/>

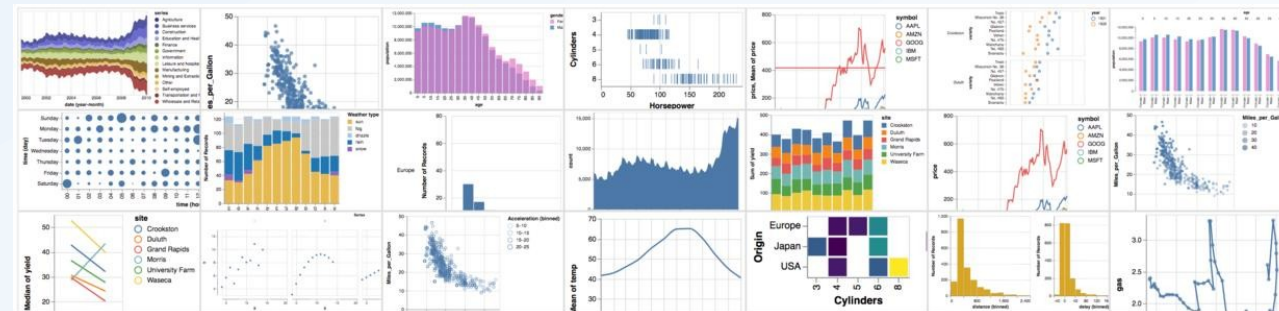
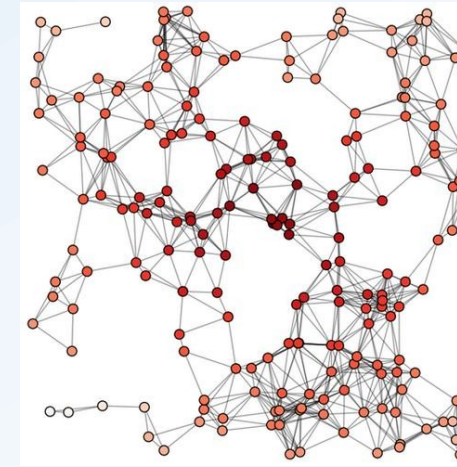
: <https://www.zhihu.com/question/49823620>

Data visualization



◆ Python

- ▶ Matplotlib
- ▶ Seaborn
- ▶ WorldCloud
- ▶ Networkx



◆ Matplotlib

. Matplotlib

◆ Matplotlib Python
NumPy

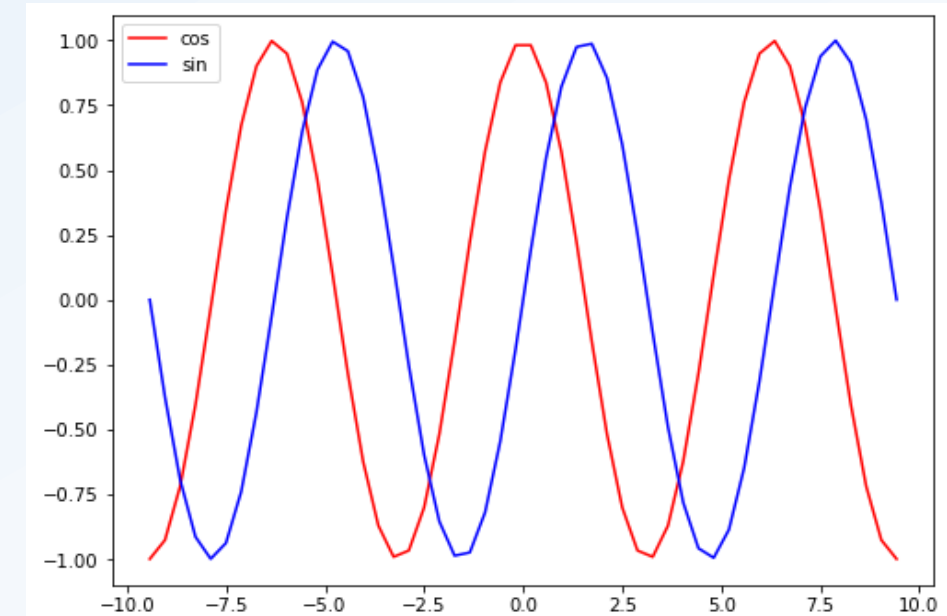
◆ Matplotlib
Numpy Pandas

◆ Matplotlib
`pip install matplotlib`

The matplotlib logo features the word "matplotlib" in a bold, blue, sans-serif font. The letter "o" is replaced by a circular icon containing a stylized pie chart with five segments in orange, yellow, green, and blue, and a black crosshair.

. Matplotlib

```
1 import numpy as np # -
2 import matplotlib.pyplot as plt # -
3 x = np.linspace(-3*np.pi,3*np.pi)
4 plt.figure(figsize=(8,6)) #
5 plt.plot(x,np.cos(x),color="red",label="cos") #
6 plt.plot(x,np.sin(x),color="blue",label="sin") #
7 plt.legend() #
8 plt.savefig("E:\\test\\07\\img\\G08.png") #
9 #
```



2. Matplotlib

1

```
1 import numpy as np                    # -
2 import matplotlib.pyplot as plt       # -
3 plt.rcParams['font.sans-serif'] = ['SimHei'] #
4 plt.rcParams['axes.unicode_minus'] = False #
5 x = np.linspace(-3*np.pi,3*np.pi,num=100) #
6 fig = plt.figure(figsize=(8,6))       #
7 plt.plot(x,np.cos(x),color="red",label="cos") #
8 plt.plot(x,np.sin(x),color="blue",label="sin") #
9 plt.xlabel('X',fontproperties='simhei',fontsize=14) # X    simhei
10 plt.ylabel('Y',fontproperties='simhei',fontsize=14) # Y    14
11 plt.legend()                           #
12 plt.show()                             #
```



2. Matplotlib

1

◆ Matplotlib

1

- ① “ fontproperties= ” 9~10
- ② rcParams['font.sans-serif'] 3
- plt.rcParams['font.sans-serif' = ['SimHei'] 'SimHei'

2

rcParams['axes.unicode_minus'] False Unicode ASCII

2	import matplotlib.pyplot as plt	# 导入第三方库 - 绘图
3	plt.rcParams['font.sans-serif'] = ['SimHei']	# 解决中文显示乱码的问题
4	plt.rcParams['axes.unicode_minus'] = False	# 解决负号显示的问题

2. Matplotlib

1

5	<code>x = np.linspace(-3*np.pi,3*np.pi,num=100)</code>	# 生成数据列表
---	--	----------

- ◆ 程序第5行，主要功能是创建数据列表，使用**`np.linspace`**将生成一组区间在 $[-3\pi, 3\pi]$ 内均匀分布的**100**个数，其中首个位置参数填入区间起始值，第二个参数填入区间结束值，**`np.pi`**表示 π ；参数**`num`**表示要生成的样本数量，默认是**50**，其值必须是**非负数**。



2. Matplotlib

1

6	<code>fig = plt.figure(figsize=(8,6))</code>	# 创建图形窗口区域
7	<code>plt.plot(x,np.cos(x),color="red",label="cos")</code>	# 绘制余弦曲线
8	<code>plt.plot(x,np.sin(x) ,color="blue",label="sin")</code>	# 绘制正弦曲线
9	<code>plt.xlabel('X轴',fontproperties='simhei',fontsize=14)</code>	# X轴坐标, simhei表示黑体
10	<code>plt.ylabel('Y轴',fontproperties='simhei',fontsize=14)</code>	# Y轴坐标, 14表示字体大小
11	<code>plt.legend()</code>	# 添加图例
12	<code>plt.show()</code>	# 显示绘制图片

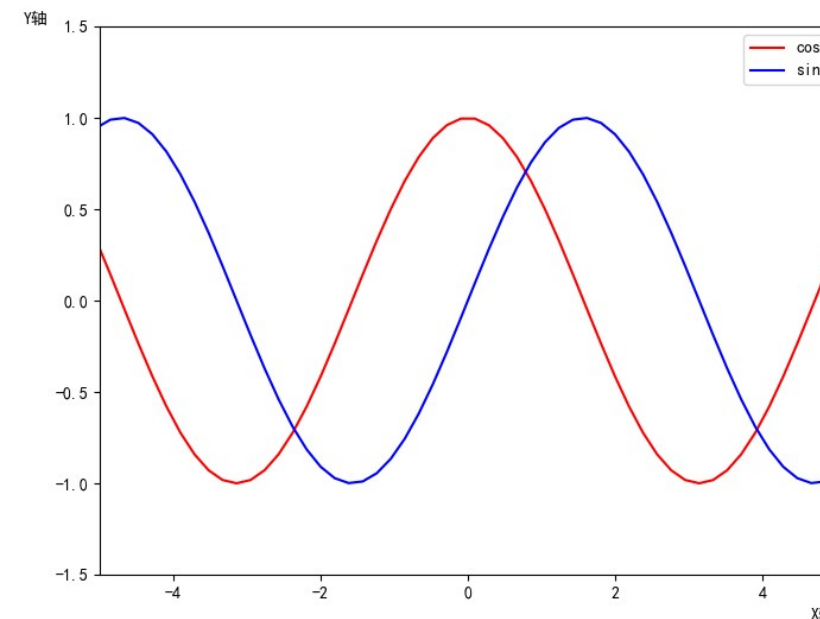
- ◆ 7~8 `plt.plot()` Matplotlib `color`
`label="xxx"` `plt.legend()` `loc`
- ◆ 9~10 `xlabel` `ylabel` `plt` `fontproperties`
`fontsize`

2. Matplotlib

2

◆ 9~12

```
1 import numpy as np                                # -
2 import matplotlib.pyplot as plt                    # -
... .....
9 plt.xlabel('X ', fontsize=14, loc='right')          # X    simhei
10 plt.ylabel('Y ', fontsize=14, loc='top', rotation=0) # Y    14
11 plt.xlim(xmin=-5, xmax=5)                          # X
12 plt.ylim(ymin=-1.5, ymax=1.5)                       # Y
13 plt.legend()                                         #
14 plt.show()                                          #
```





2. Matplotlib

2

9	plt.xlabel('X轴', fontsize=14, loc='right')	# X轴坐标, simhei表示黑体
10	plt.ylabel('Y轴', fontsize=14, loc='top', rotation=0)	# Y轴坐标, 14表示字体大小

◆	9~10	X	Y	loc	xlabel	left
	center	right		ylabel	top	center
	rotation		Y	bottom		
			90			

11	plt.xlim(xmin=-5, xmax=5)	# 添加X轴刻度
12	plt.ylim(ymin=-1.5, ymax=1.5)	# 添加Y轴刻度

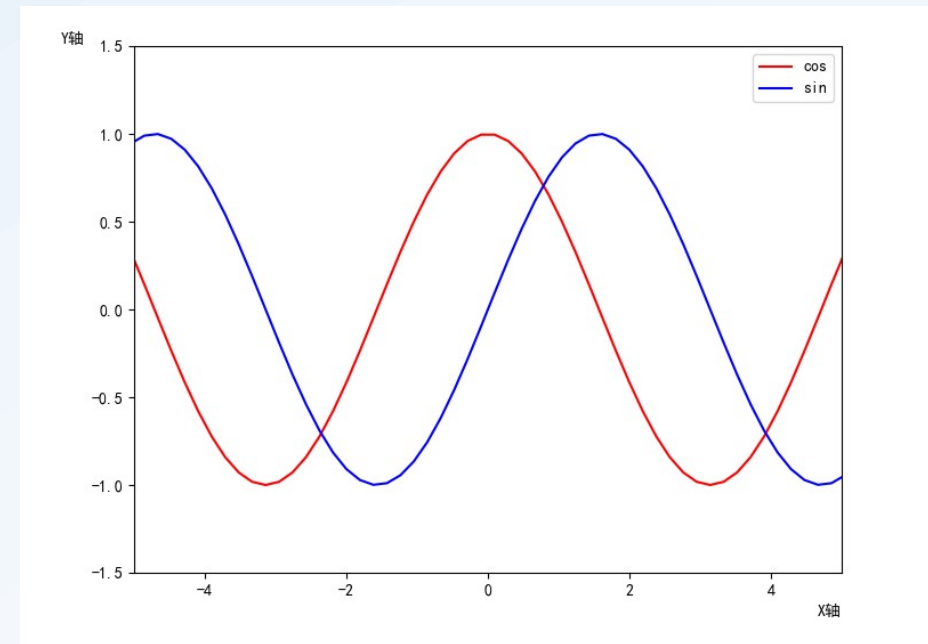
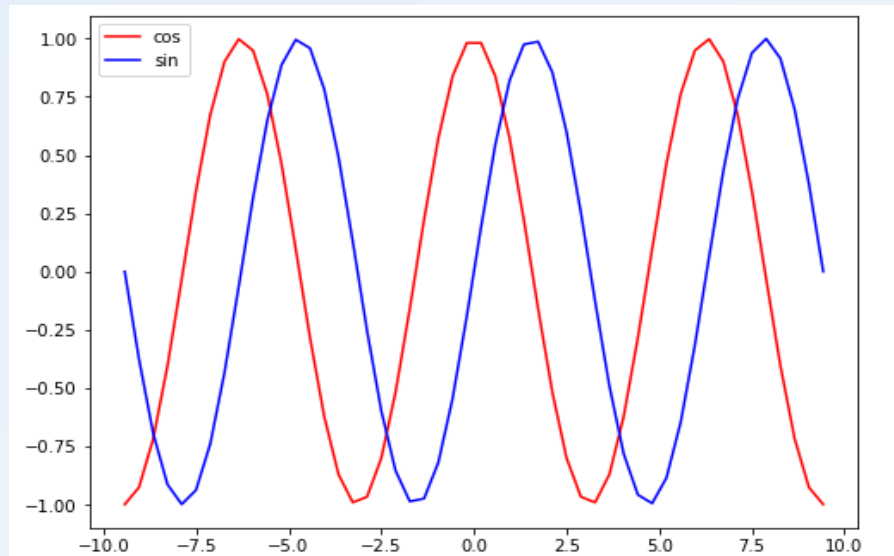
◆	11~12	xlim	ylim	plt	plt.xlim()	xmin	X	xmax	X
		plt.ylim()	ymin	Y	ymax	Y			

Data visualization



2. Matplotlib

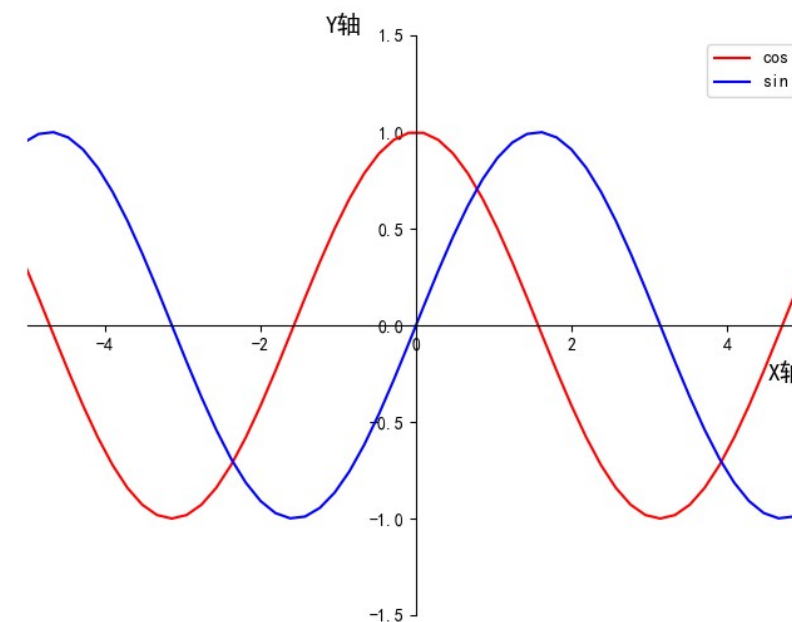
2



2. Matplotlib

3

```
1 import numpy as np          # -
... .....                   ...
13 ax = plt.gca()             #
14 ax.spines['right'].set_color('none') #
15 ax.spines['top'].set_color('none') #
16 ax.xaxis.set_ticks_position('bottom') # X
17 ax.spines['bottom'].set_position(('data',0)) #
18 ax.yaxis.set_ticks_position('left') # Y
19 ax.spines['left'].set_position(('data',0)) #
20 plt.legend()                #
21 plt.savefig("E:\\test\\07\\img\\G804.png") #
22 plt.show()                  #
```



3

13	<code>ax = plt.gca()</code>	# 获取当前坐标轴信息
14	<code>ax.spines['right'].set_color('none')</code>	# 不设置右侧边框颜色
15	<code>ax.spines['top'].set_color('none')</code>	# 不设置上侧边框颜色
16	<code>ax.xaxis.set_ticks_position('bottom')</code>	# 设置X坐标轴位置为下侧边
17	<code>ax.spines['bottom'].set_position(('data',0))</code>	# 设置下侧边起始位置
18	<code>ax.yaxis.set_ticks_position('left')</code>	# 设置Y坐标轴位置为左侧边
19	<code>ax.spines['left'].set_position(('data',0))</code>	# 设置左侧边起始位置

ax.xaxis.set_ticks_position()		ax.yaxis.set_ticks_position()			X	left	right	
both	default	none	Y	top	bottom	both	default	none



2. Matplotlib

3

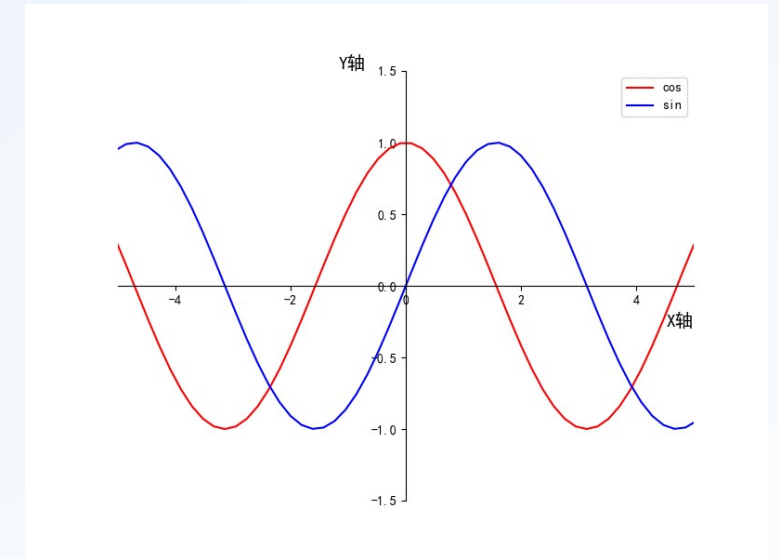
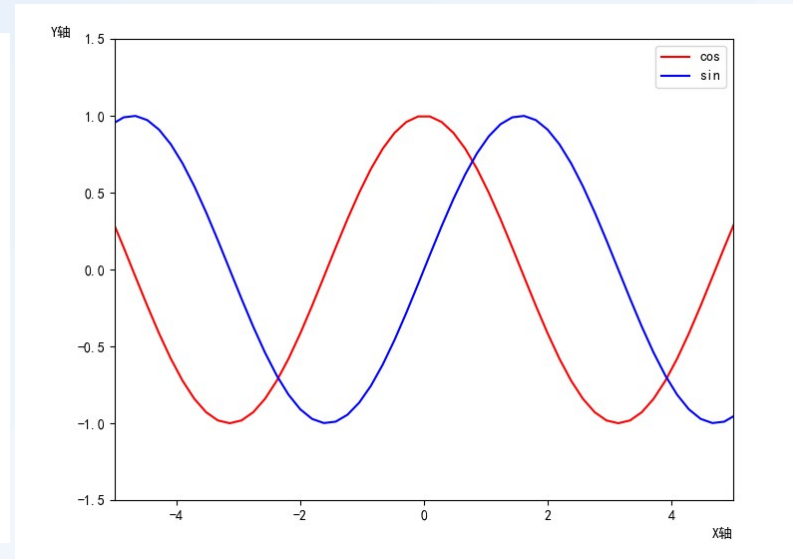
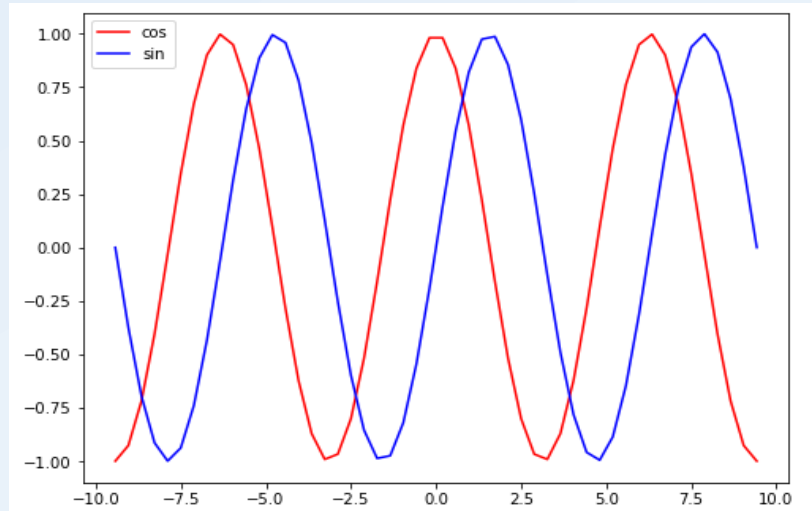
20	<code>plt.legend()</code>	# 添加图例
21	<code>plt.savefig("E:\\test\\07\\img\\G804.png")</code>	# 保存图形到指定路径
22	<code>plt.show()</code>	# 显示绘制图形

◆ 21 `plt.savefig("xxx")` “xxx”

◆ `plt.savefig()` `plt.show()`

2. Matplotlib

3

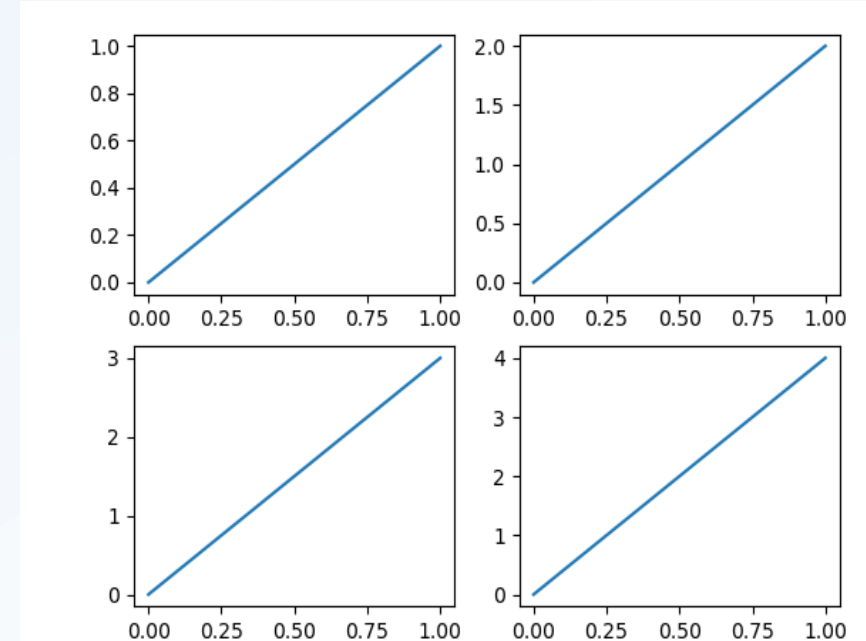


3. Matplotlib

1 subplot

◆ `subplot(numRows,numCols,plotNum)`

1	<code>import matplotlib.pyplot as plt</code>	#	-
2	<code>plt.figure()</code>	#	
3	<code>plt.subplot(2,2,1)</code>	#	1,1
4	<code>plt.plot([0,1],[0,1])</code>	#	
5	<code>plt.subplot(2,2,2)</code>	#	1,2
6	<code>plt.plot([0,1],[0,2])</code>	#	
7	<code>plt.subplot(2,2,3)</code>	#	2,1
8	<code>plt.plot([0,1],[0,3])</code>	#	
9	<code>plt.subplot(2,2,4)</code>	#	2,2
10	<code>plt.plot([0,1],[0,4])</code>	#	
11	<code>plt.show()</code>	#	





3. Matplotlib

1 subplot

3	<code>plt.subplot(2,2,1)</code>	# 创建坐标为 (1,1) 的子图
4	<code>plt.plot([0,1],[0,1])</code>	# 绘图

◆ 3 `subplot(numRows, numCols, plotNum)` numRows numCols 2
2×2 plotNum 1 1 1 1

7	<code>plt.subplot(223)</code>	# 创建坐标为 (2,1) 的子图
8	<code>plt.plot([0,1],[0,3])</code>	# 绘图

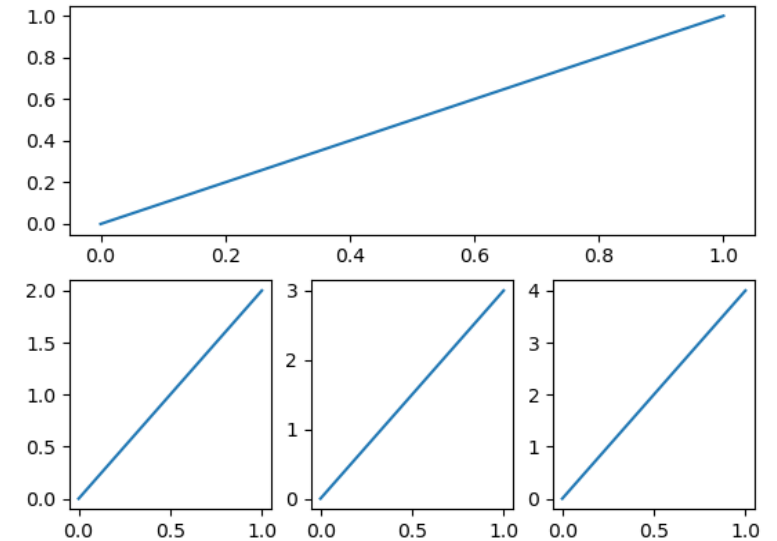
◆ 7 `subplot(numRows, numCols, plotNum)` numRows numCols plotNum
10 `subplot(223)` `subplot(2,2,3)` numRows numCols 2
plotNum 3 3 2 1

3. Matplotlib

2

subplot

1	<code>import matplotlib.pyplot as plt</code>	#	-
2	<code>plt.figure()</code>	#	
3	<code>plt.subplot(2,1,1)</code>	#	1,1 1,3
4	<code>plt.plot([0,1],[0,1])</code>	#	
5	<code>plt.subplot(2,3,4)</code>	#	2,1
6	<code>plt.plot([0,1],[0,2])</code>	#	
7	<code>plt.subplot(2,3,5)</code>	#	2,2
8	<code>plt.plot([0,1],[0,3])</code>	#	
9	<code>plt.subplot(2,3,6)</code>	#	2,3
10	<code>plt.plot([0,1],[0,4])</code>	#	
11	<code>plt.show()</code>	#	



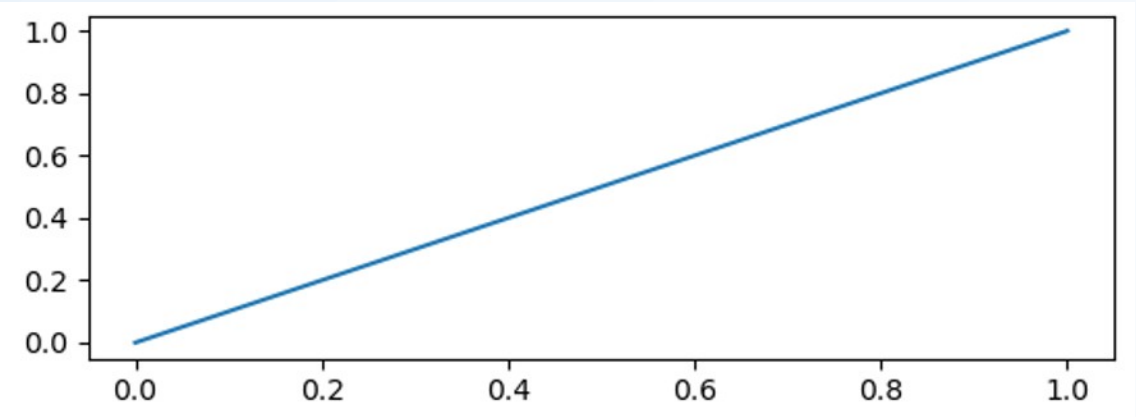


3. Matplotlib

2 subplot

3	<code>plt.subplot(2,1,1)</code>	# 创建坐标 (1,1) 至 (1,3) 的子图
4	<code>plt.plot([0,1],[0,1])</code>	# 绘图

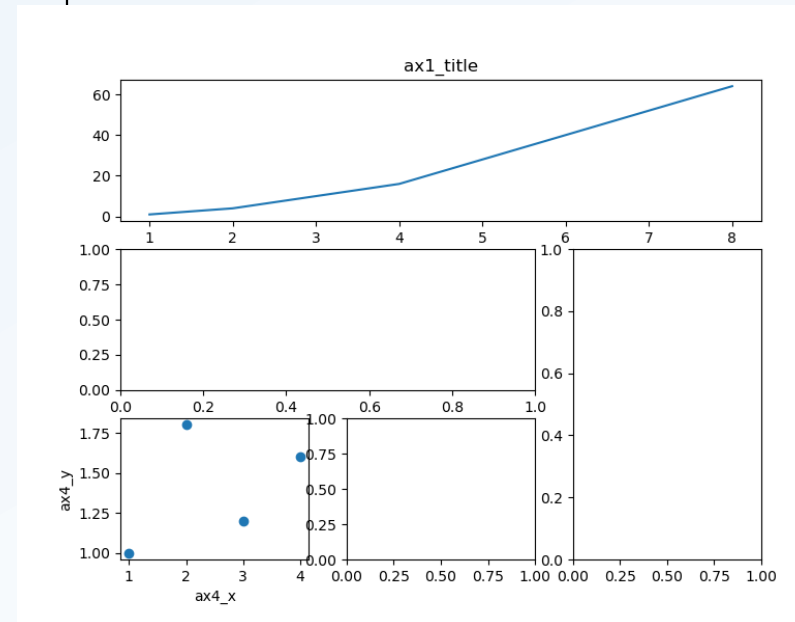
◆ 3 subplot() numRows numCols 2 1 2×1 plotNum 1
1 1 1 1,1 1,2 1,3 3



3. Matplotlib

3 subplot2grid

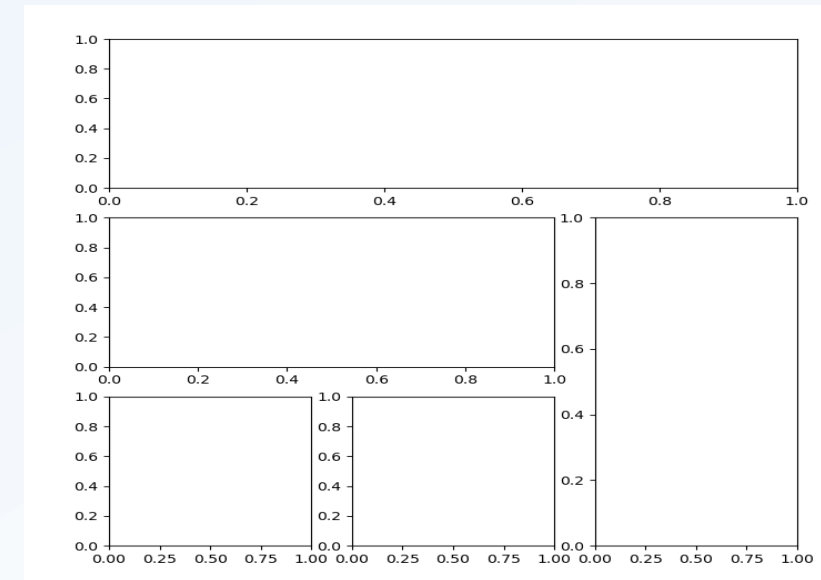
```
1 import matplotlib.pyplot as plt # -
2 plt.figure(figsize=(8,6)) #
3 ax1=plt.subplot2grid((3,3),(0,0),rowspan=1,colspan=3) # ax1
4 ax1.plot([1,2,4,8],[1,4,16,64]) # ax1
5 ax1.set_title('ax1_title') #
6 ax2=plt.subplot2grid((3,3),(1,0),rowspan=1,colspan=2) # ax2
7 ax3=plt.subplot2grid((3,3),(1,2),rowspan=2,colspan=1) # ax3
8 ax4=plt.subplot2grid((3,3),(2,0)) # ax4
9 ax5=plt.subplot2grid((3,3),(2,1)) # ax5
10 ax4.scatter([1,2,3,4],[1,1.8,1.2,1.6]) # ax4
11 ax4.set_xlabel('ax4_x') # ax4 X
12 ax4.set_ylabel('ax4_y') # ax4 Y
13 plt.show() #
```



3. Matplotlib

3 gridspec

```
1 import matplotlib.pyplot as plt           # -
2 from matplotlib import gridspec          # gridspe
3 plt.figure(figsize=(8,8))                #
4 gs=gridspec.GridSpec(3,3)                # 3 3
5 ax1=plt.subplot(gs[0,:])                  # ax1
6 ax2=plt.subplot(gs[1,:2])                 # ax2
7 ax3=plt.subplot(gs[1,:2])                 # ax3
8 ax4=plt.subplot(gs[-1,0])                 # ax4
9 ax5=plt.subplot(gs[-1,-2])                # ax5
10 plt.show()                               #
```



Data visualization



◆ Seaborn

Seaborn

◆ Seaborn

Matplotlib

Seaborn

Matplotlib

Matplotlib

Seaborn

API

◆ Seaborn

Pandas

Numpy



seaborn

◆ Seaborn

Matplotlib

Seaborn

Matplotlib

Data visualization



1. Seaborn

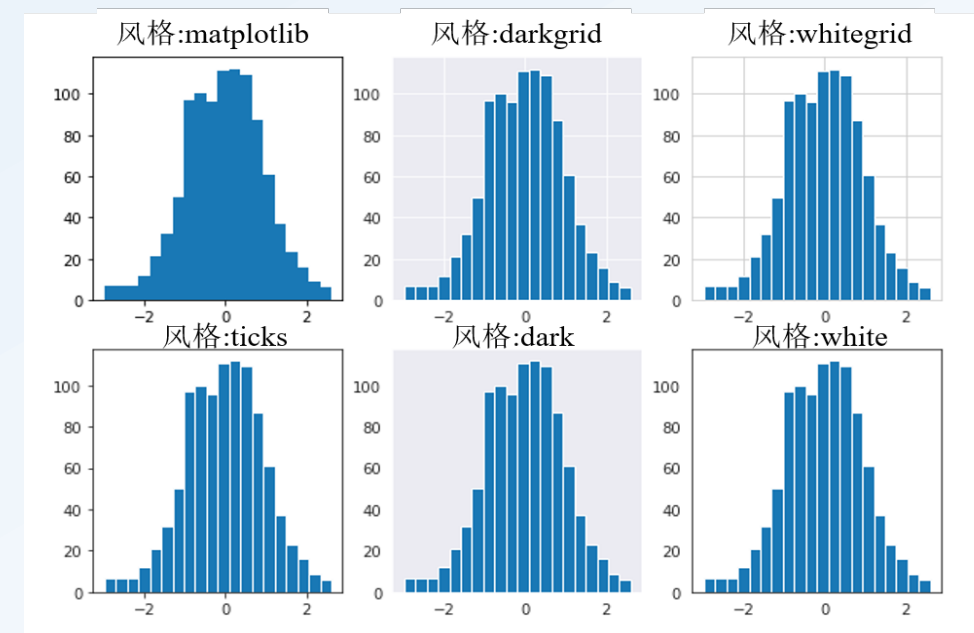
◆ style

- darkgrid
- Matplotlib
- Whitegrid
- Dark
- White
- ticks



- 1 set
- 2 set_style
- 3 axes_style

axes



1. Seaborn

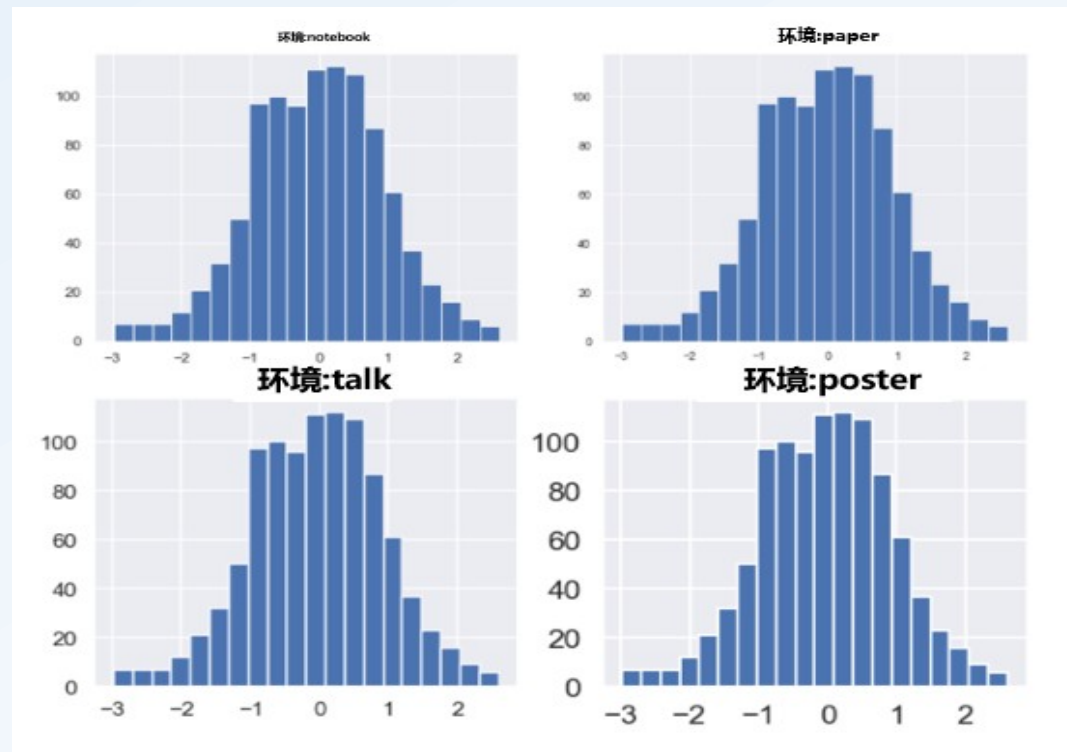
◆ context

- notebook
- Paper
- Talk
- poster



- 1 set
 - 2 set_context
 - 3 plotting_context
- with

axes



Data visualization



1. Seaborn



◆ color_palette	RGB	8
	n_color	
◆ hls_palette	Hue	Luminance
Saturation	8	n_color

```
sns.palplot(sns.hls_palette(n_colors=8))
```



```
sns.palplot(sns.color_palette(n_colors=8))
```



2. Seaborn



- ◆ iris
- ◆ titanic
- ◆

```
1 import pandas as pd
2 iris=pd.read_csv("E:\\test\\07\\data\\iris.csv")
3 titanic=pd.read_csv("E:\\test\\07\\data\\titanic.csv")
```

Data visualization



3.



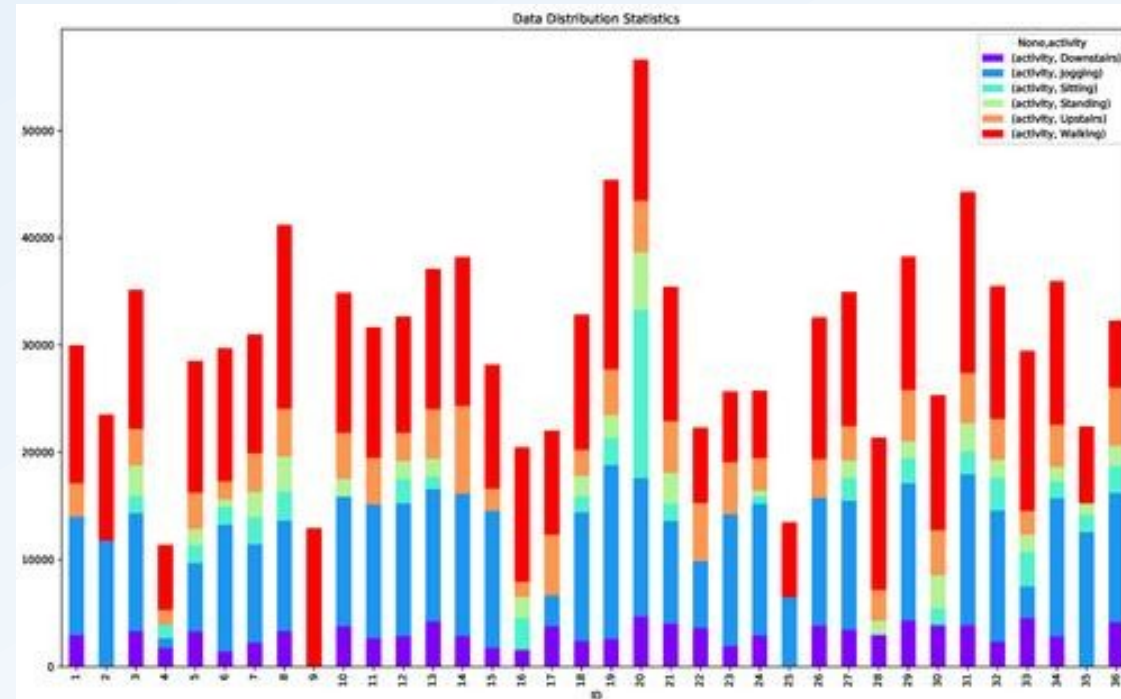
displot



kdeplot



rugplot rug



3.

1 `displot`

◆ `distplot` `kdeplot` `rugplot`

◆ `seaborn.displot(a,bins=None,hist=True,kde=True,rug=False,fit=None, hist_kws= None, kde_kws=None,rug_kws=None,fit_kws=None, color=None, vertical=False, norm_hist=False, axlabel=None,label=None,ax=None)`

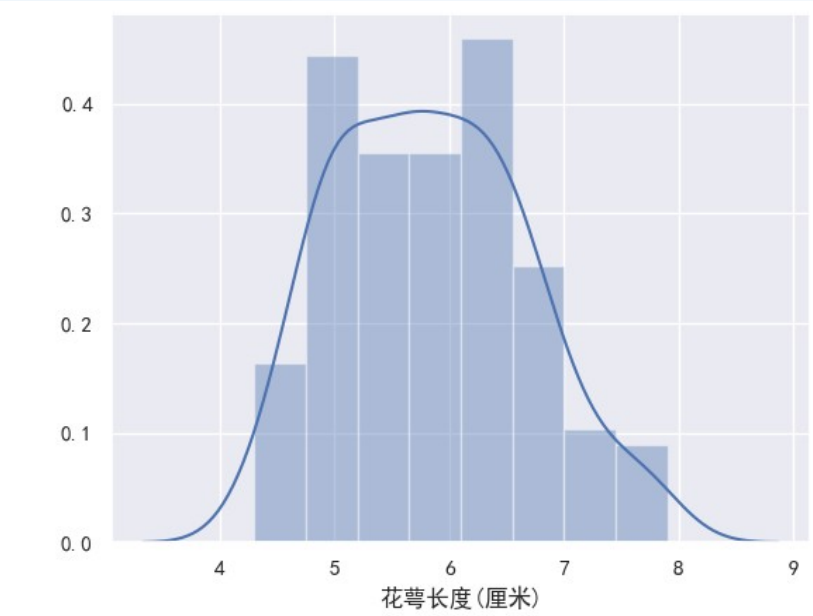
- `a` `Series`
- `bins` `None`
- `hist` `True`
- `kde` `True`
- `rug` `False`
- `color` `"r"`
- `vertical` `False`
- `norm_hist` `True` `False` `False`



3.

1 **displot**

```
1 import pandas as pd                                # -
2 import seaborn as sns                              # -
3 import matplotlib.pyplot as plt                    # -
4 sns.set_style('darkgrid')                          # Seaborn
5 sns.set(font='SimHei')                             #
6 iris=pd.read_csv("E:\\test\\07\\data\\iris.csv")    #
7 sns.distplot(iris["sepal_length"],axlabel="      (  )") #
8 plt.show()                                          #
```



3.

2

kdeplot

◆ kdeplot distplot kdeplot kde kdeplot

◆ `seaborn.kdeplot(data,data2=None,shade=False,vertical=False,kernel='gau' , bw='scott',
gridsize=100,cut=3,clip=None,legend=True,cumulative=False,color=None, shade_lowest=True, cbar=False,
cbar_ax=None, cbar_kws=None, ax=None)`

- data
- data2 data 2 None
- shade True False
- vertical
- kernel gau 2
- legend

3.

2

kdeplot

◆ `seaborn.kdeplot(data,data2=None,shade=False,vertical=False,kernel='gau' , bw='scott',
gridsize=100,cut=3,clip=None,legend=True,cumulative=False,color=None, shade_lowest=True,
cbar=False, cbar_ax=None, cbar_kws=None, ax=None)`

- cumulative False
- color “ r”
- shade_lowest True
- cbar

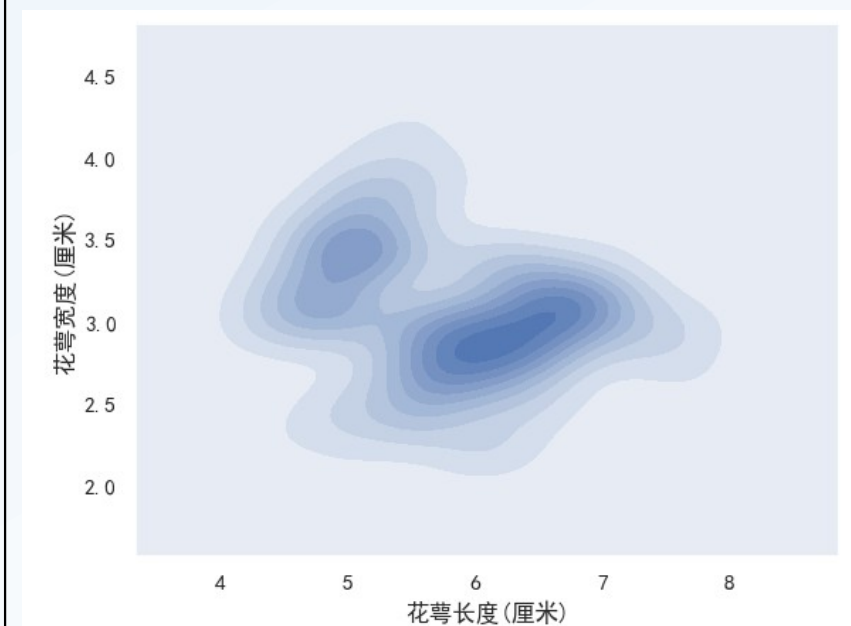
Data visualization



3.

2 kdeplot

```
1 import pandas as pd # -
2 import seaborn as sns # -
3 import matplotlib.pyplot as plt # -
4 sns.set_style('darkgrid') # Seaborn
5 sns.set(font='SimHei') #
6 iris=pd.read_csv("E:\\test\\07\\data\\iris.csv") #
7 length=iris["sepal_length"] #
8 width=iris["sepal_width"] #
9 sns.kdeplot(length,width,shade=True) #kdeplot
10 plt.xlabel(" ( )") # X
11 plt.ylabel(" ( )") # Y
12 plt.show() #
```



3.

3 rugplot

◆ rugplot

kdeplot

◆ `seaborn.rugplot(a,height=0.05,axis='x',ax=None)`

➤ a

➤ height

0.05

➤ axis

“x” X

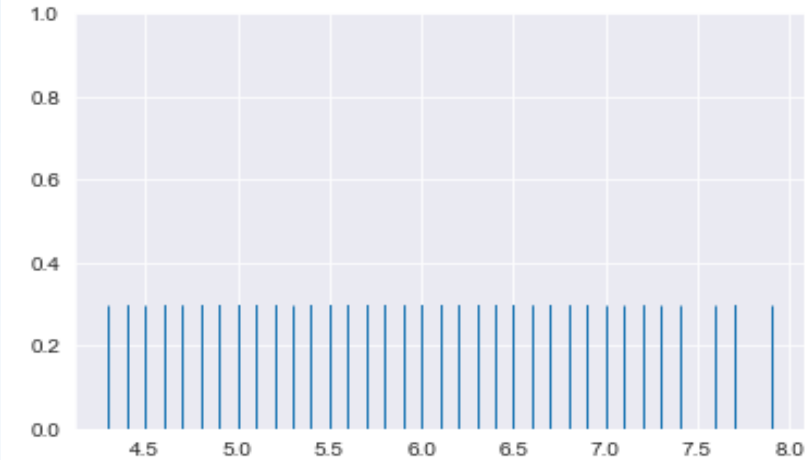
Data visualization



3.

3 rugplot

```
1 import pandas as pd          # -
2 import seaborn as sns        # -
3 import matplotlib.pyplot as plt # -
4 sns.set_style('darkgrid')    # Seaborn
5 iris=pd.read_csv("E:\\test\\07\\data\\iris.csv") #
6 length=iris["sepal_length"]  #
7 sns.rugplot(length,height=0.3) #rugplot
8 plt.show()                   #
```



Data visualization



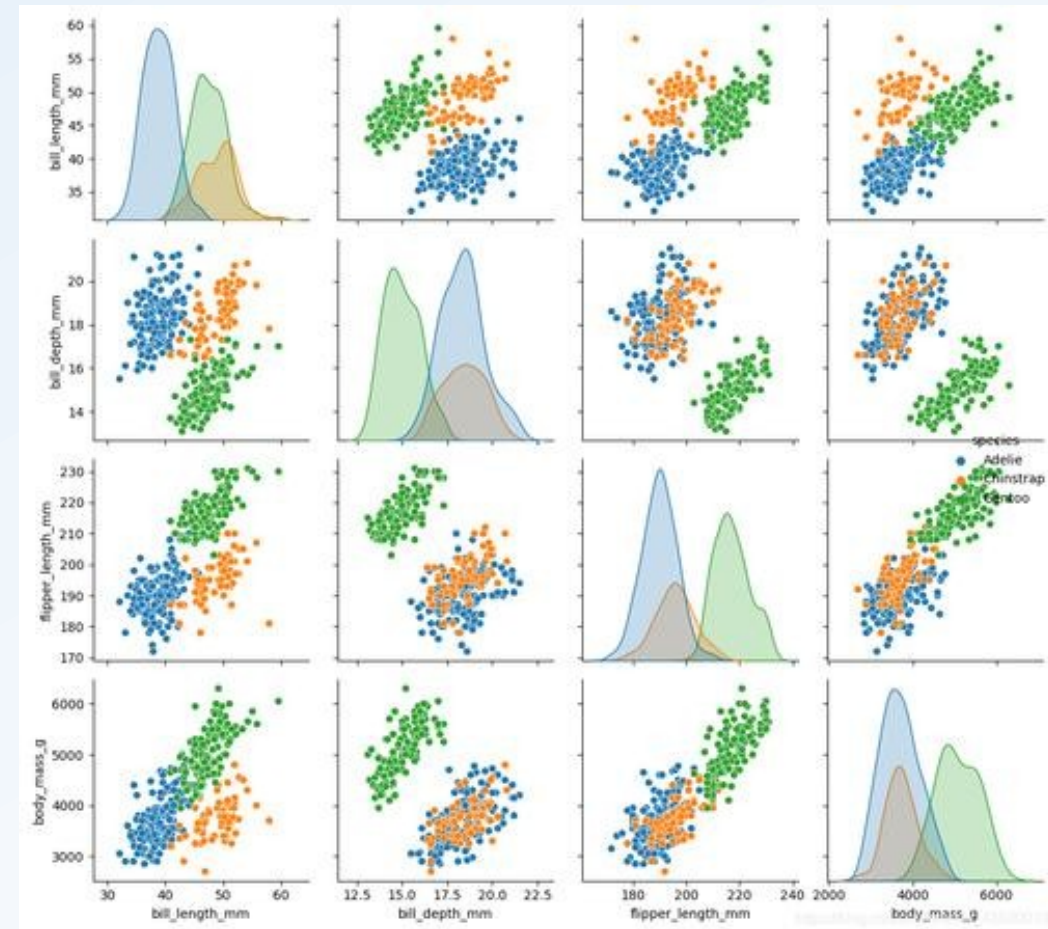
4.



jointplot



pairplot



4.

1

jointplot

◆ Jointplot

2

◆ `serborn.jointplot(x,y,data=None,kind='scatter',stat_func=,color=None,size=6,
ratio=5,space=0.2,dropna=True,xlim=None,ylim=None, joint_kws=None, marginal_kws =None,
annot_kws=None)`

➤ `x y` `data DataFrame` `x y` `DataFrame`
`data None` `x y` `DataFrame`
➤ `data DataFrame` `None`

1

➤	kind	scatter	reg	kde
➤	color			
➤	ratio	5		
➤	space			
➤	xlim			
	ylim			
	X			
	Y			
➤	joint_kws,marginal_kws,annot_kws			

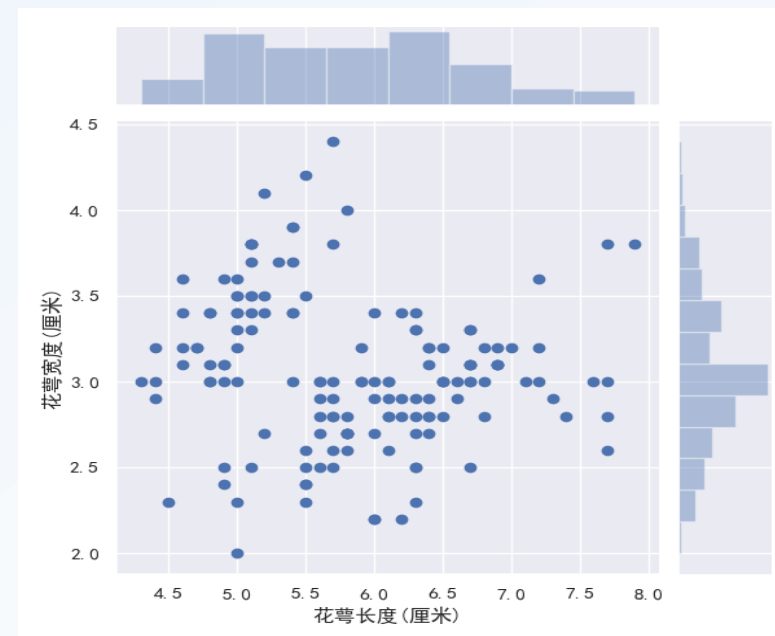
Data visualization



4.

1 jointplot

```
1 import pandas as pd # -
2 import seaborn as sns # -
3 import matplotlib.pyplot as plt # -
4 sns.set_style('darkgrid') # Seaborn
5 sns.set(font='SimHei') #
6 iris=pd.read_csv("E:\\test\\07\\data\\iris.csv") #
7 iris = iris.rename(columns={"sepal_length": " ( )", #
8 "sepal_width": " ( )"}) #
9 sns.jointplot(x=" ( )", # X
10 y=" ( )", # Y
11 data=iris) #
12 plt.show() #
```



4.

2 pairplot

- ◆ pairplot X Y
- ◆ `seaborn.pairplot(data,hue=None,hue_order=None,palette=None,vars=None, x_vars=None,y_vars=None,kind='scatter',diag_kind='auto',markers=None,height=2.5,aspect=1,corner=False,dropna=False,plot_kws=None,diag_kws=None,grid_kws=None,size=None)`
 - data DataFrame
 - hue DataFrame
 - hue_order hue None
 - palette husl pastel muted bright deep colorblind dark
hls Paired Set1 Set2 Blues Greens Reds Purples BuGn_r GnBu_d cubehelix

4.

2 pairplot

◆ `seaborn.pairplot(data,hue=None,hue_order=None,palette=None,vars=None,
x_vars=None,y_vars=None,kind='scatter',diag_kind='auto',markers=None,height=2.5,aspect=1,corner=False,dropna
=False,plot_kws=None,diag_kws=None,grid_kws= None,size=None)`

➤ height	2.5				
➤ aspect	aspect	0.5	0.5*		1
➤ kind	scatter	kde	hist	reg	
➤ corner		False		True	
➤ dropna		False			

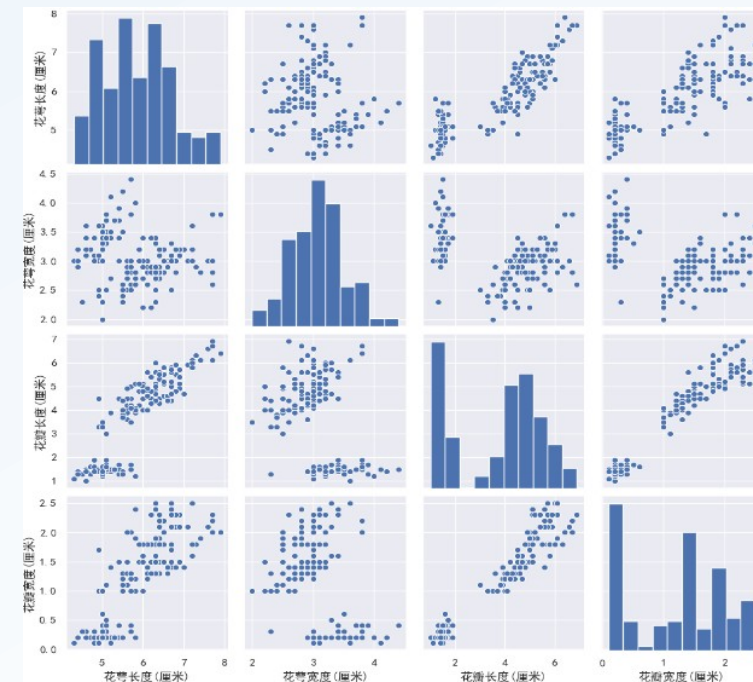
Data visualization



4.

2 pairplot

```
1 import pandas as pd # —
2 import seaborn as sns # —
3 import matplotlib.pyplot as plt # —
4 sns.set_style('darkgrid') # Seaborn
5 sns.set(font='SimHei') #
6 iris=pd.read_csv("E:\\test\\07\\data\\iris.csv") #
7 iris = iris.rename(columns={"sepal_length":" ( )", #
   "sepal_width":" ( )",
8 "petal_length":" ( )",
9 "petal_width":" ( )"})
10 sns.pairplot(data=iris)
11 plt.show() #pairplot
12 import pandas as pd #
```

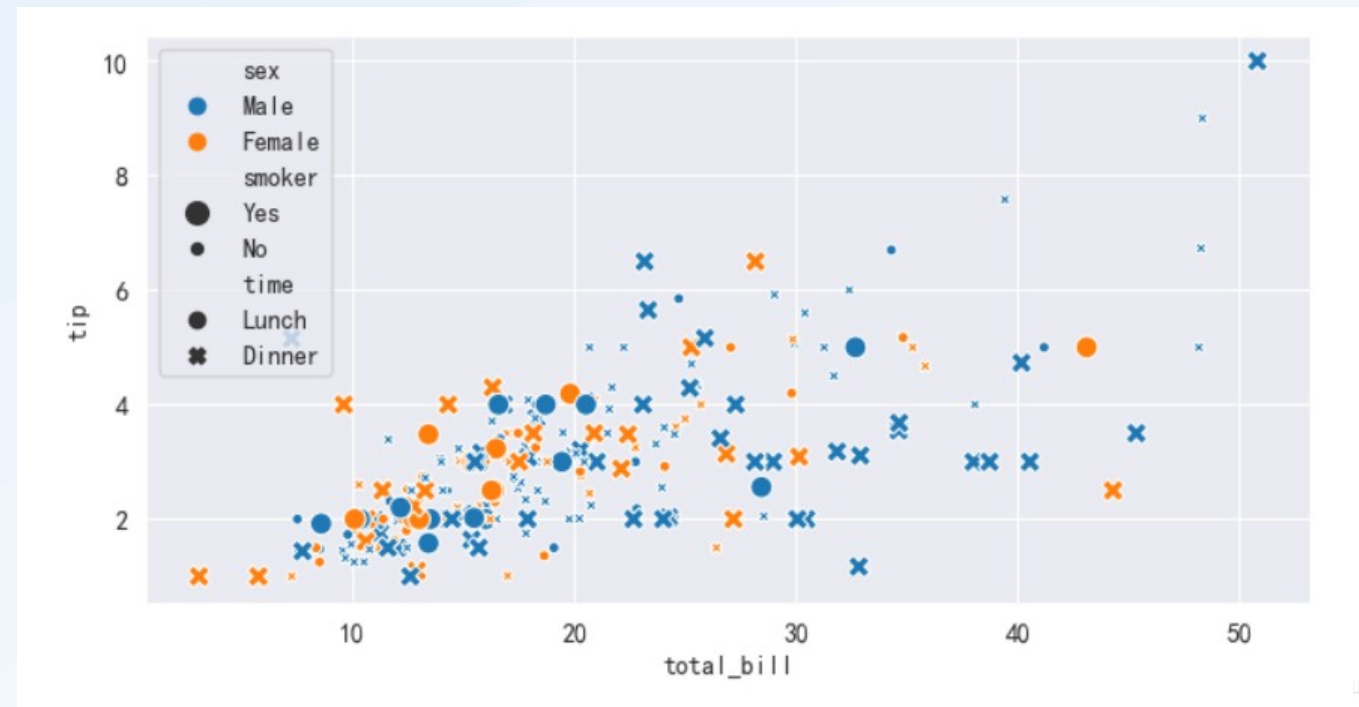


Data visualization



5.

- ◆ Seaborn
- ◆ relplot
- ◆ scatterplot
- ◆ lineplot



5.

1 relplot

◆ relplot

◆ `seaborn.relplot(x=None,y=None, hue=None, size=None, style=None, data=None, row=None, col=None, palette=None, hue_order=None, hue_norm=None, sizes=None, dashes=None, style_order=None, legend='auto', kind='scatter', height=5, aspect=1)`

➤ `x y X Y`

➤ `hue` `categorical`

➤ `data` `DataFrame` `None`

➤ `row col`

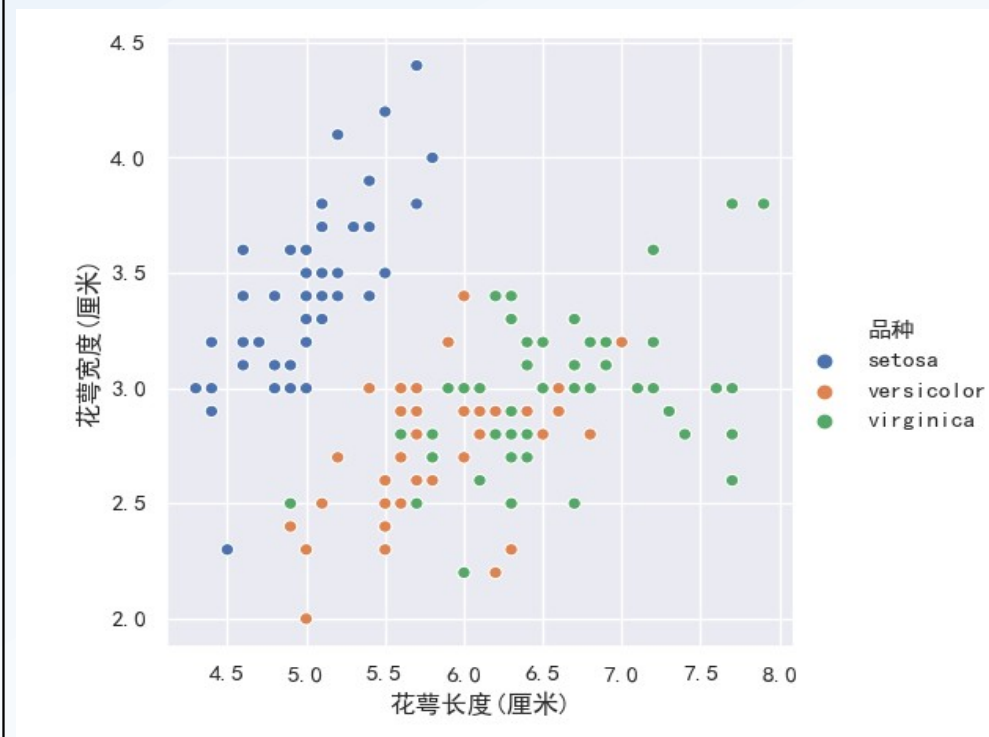
➤ `sizes`

➤ `kind` `kind` `scatter` `line` `scatter` `kind` `scatter` `scatterplot`
`kind` `line` `lineplot`

5.

1 relplot

```
1 import pandas as pd # -
2 import seaborn as sns # -
3 import matplotlib.pyplot as plt # -
4 sns.set_style('darkgrid') # Seaborn
5 sns.set(font='SimHei') #
6 iris=pd.read_csv("E:\\test\\07\\data\\iris.csv") #
7 iris = iris.rename(columns={"sepal_length": "    ( )", #
8 "sepal_width": "    ( )",
9 "species": "    "})
10 sns.relplot(x="    ( )", # X
11 y="    ( )", # Y
12 hue="    ", # hue
13 data=iris) #
14 plt.show() #
```



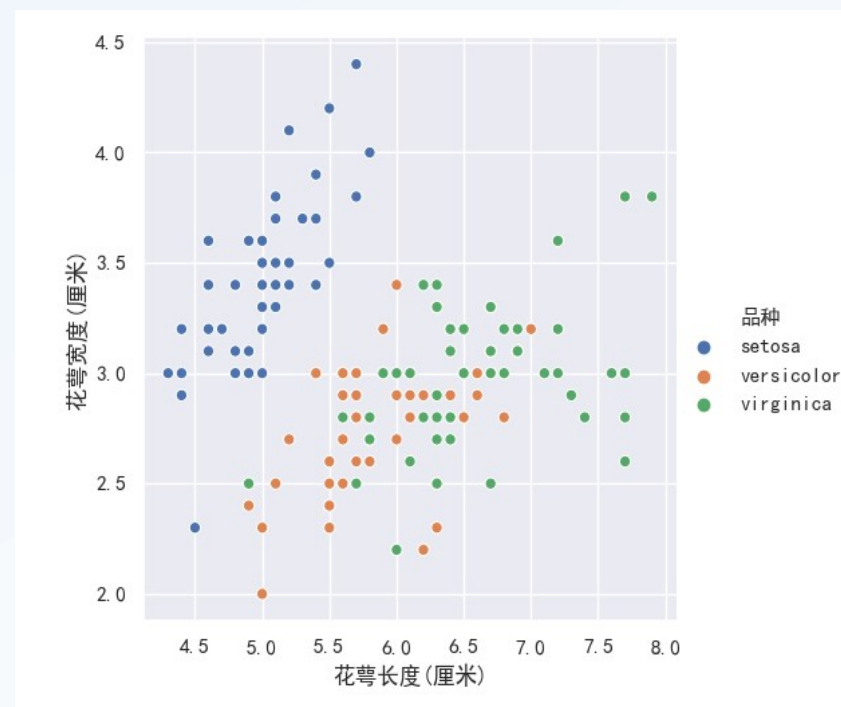
Data visualization



5.

2 scatterplot

```
1 import pandas as pd # -
2 import seaborn as sns # -
3 import matplotlib.pyplot as plt # -
4 sns.set_style('darkgrid') # Seaborn
5 sns.set(font='SimHei') #
6 iris=pd.read_csv("E:\\test\\07\\data\\iris.csv") #
7 iris = iris.rename(columns={"sepal_length": "sepal_width", #
8 "sepal_width": "sepal_length", #
9 "species": "species"}) #
10 sns.scatterplot(x="sepal_width", y="sepal_length", hue="species", # X
11 data=iris, # Y
12 # hue
13 #
14 plt.show() #
```

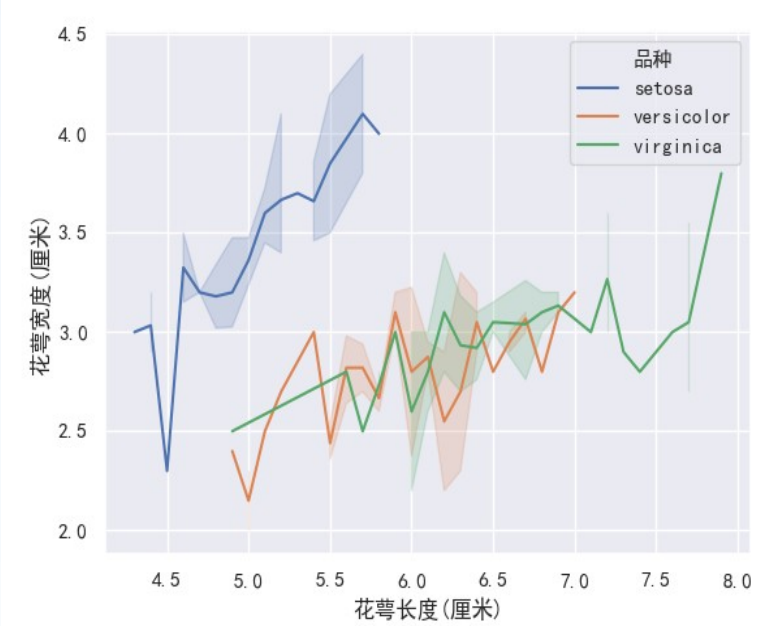




5.

3 lineplot

```
1 import pandas as pd # -
2 import seaborn as sns # -
3 import matplotlib.pyplot as plt # -
4 sns.set_style('darkgrid') # Seaborn
5 sns.set(font='SimHei') #
6 iris=pd.read_csv("E:\\test\\07\\data\\iris.csv") #
7 iris = iris.rename(columns={"sepal_length": " ( )", #
8 "sepal_width": " ( )", #
9 "species": " "}) #
10 sns.lineplot(x=" ( )", # X
11 y=" ( )", # Y
12 hue=" ", # hue
13 data=iris) #
14 plt.show() #
```

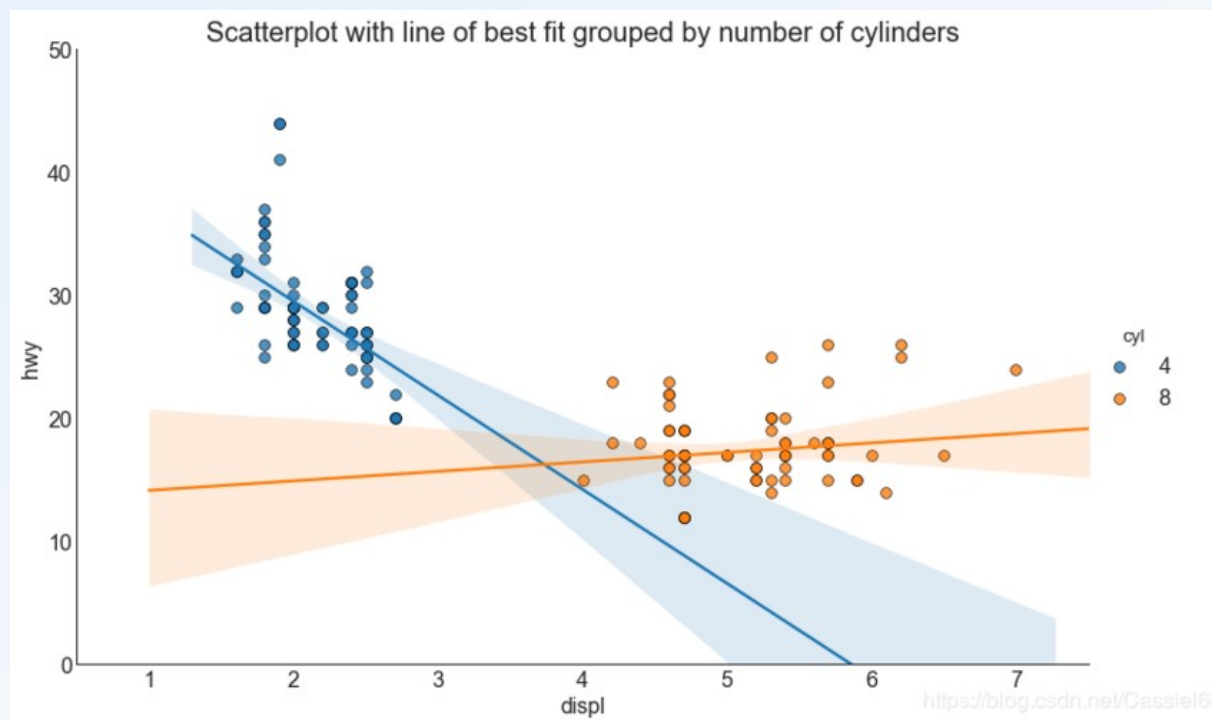


6.



Seaborn

- ◆ regplot
- ◆ residplot
- ◆ implot



6.

1 regplot

◆ regplot +

◆ `seaborn.regplot(x=None,y=None,data=None, x_estimator=None, x_bins=None, x_ci='ci',scatter=True,fit_reg=True,ci=95,n_boot=1000,units=None,order=1,logistic=False,lowess=False,robust=False,logx=False,x_partial=None,y_partial=None,truncate=False, dropna=True, x_jitter=None, y_jitter=None,label=None,color=None,marker='o', scatter_kws=None, line_kws=None, ax=None)`

➤ x y X Y

➤ data DataFrame None

➤ x_estimator x x_estimator x_ci None

➤ x_bins x x_estimator



—

6.

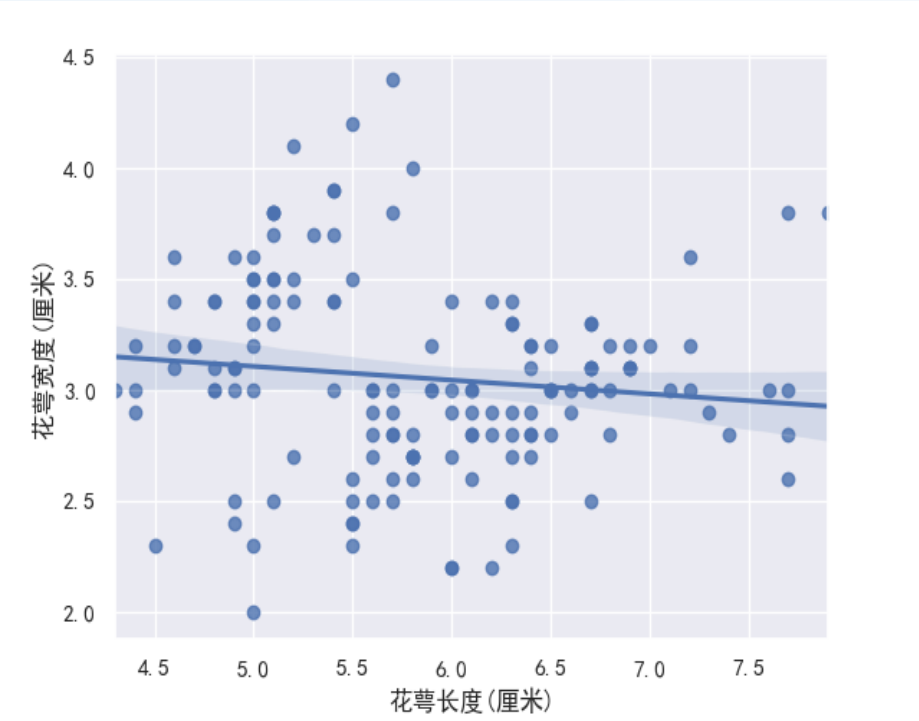
1 regplot

	x_ci	“ci”	“sd”	None	x_ci	“ci”	ci	x_ci	“sd”
	x_bins			“ci”					
	ci	[0,100]			95				
	scatter			True					
	logistic				logistic		False		
	color								
	marker								
	x_jitter	X		None					
	y_jitter	Y		None					

6.

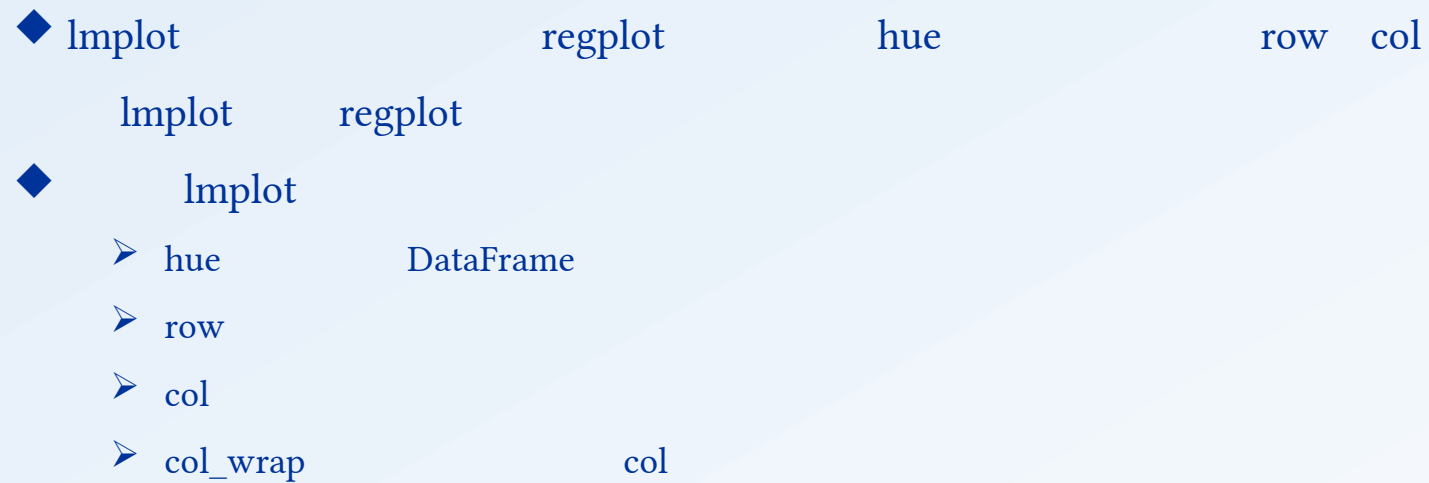
1 regplot

```
1 import pandas as pd # -
2 import seaborn as sns # -
3 import matplotlib.pyplot as plt # -
4 sns.set_style('darkgrid') # Seaborn
5 sns.set(font='SimHei') #
6 iris=pd.read_csv("E:\\test\\07\\data\\iris.csv") #
7 sns.regplot(x="sepal_length", #scatterplot X
8 y="sepal_width", # Y
9 data=iris) #
10 plt.xlabel(" ( )") # X
11 plt.ylabel(" ( )") # Y
12 plt.show() #
```



6.

2 Implot



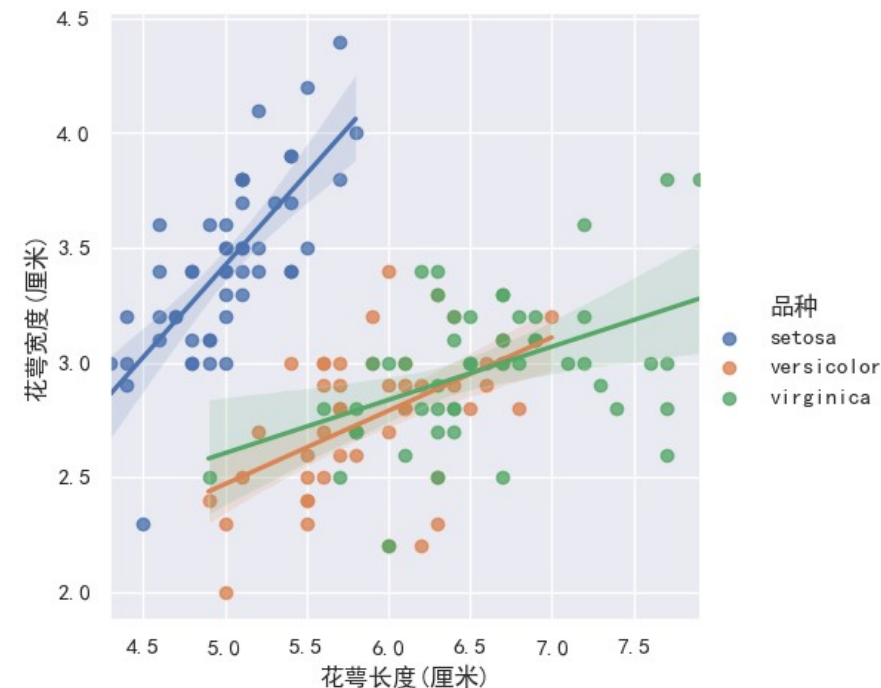
Data visualization



6.

2 Implot

```
1 import pandas as pd # -
2 import seaborn as sns # -
3 import matplotlib.pyplot as plt # -
4 sns.set_style('darkgrid') # Seaborn
5 sns.set(font='SimHei') #
6 iris=pd.read_csv("E:\\test\\07\\data\\iris.csv") #
7 iris = iris.rename(columns={"sepal_length": "花萼长度", #
8 "sepal_width": "花萼宽度", #
9 "species": "品种"}) #
10 sns.lmplot(x="花萼长度", # X
11 y="花萼宽度", # Y
12 hue="品种", # hue
13 data=iris) #
plt.show()
```



6.

3 residplot

◆ residplot lmpplot y=0

◆ seaborn.residplot(x=None, y=None, data=None, lowess=False, x_partial=None, y_partial=None, order=1, robust=False, dropna=True, label=None, color=None, scatter_kws=None, line_kws=None, ax=None)

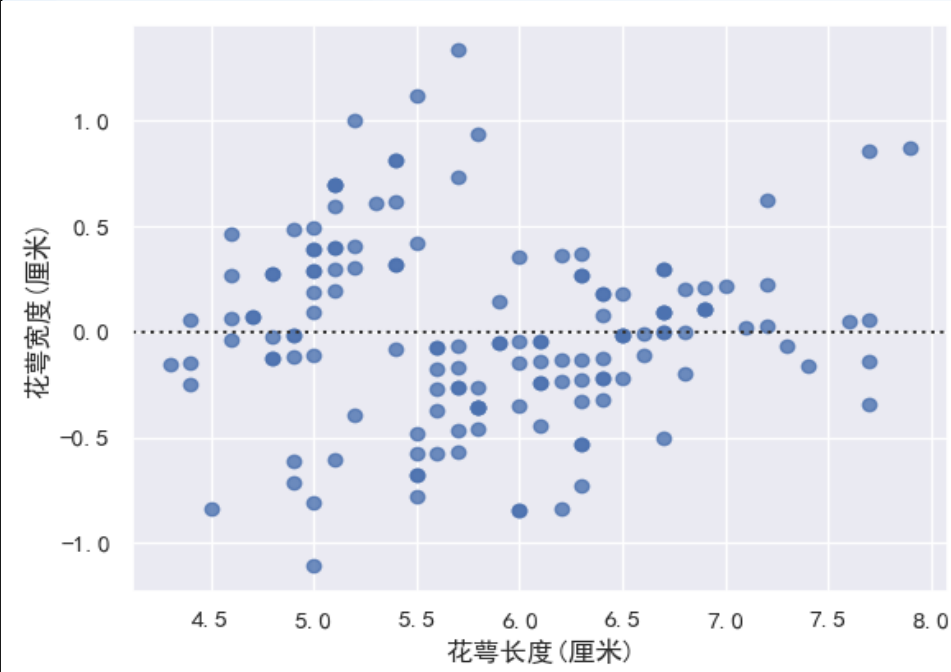
- x y X Y
- data DataFrame None
- lowess Lowess False
- dropna True
- ax



6.

3 residplot

```
1 import pandas as pd # -
2 import seaborn as sns # -
3 import matplotlib.pyplot as plt # -
4 plt.rcParams['axes.unicode_minus'] = False #
5 sns.set_style('darkgrid') # Seaborn
6 sns.set(font='SimHei') #
7 iris=pd.read_csv("E:\\test\\07\\data\\iris.csv") #
8 sns.residplot(x="sepal_length", #residplot , X
9 y="sepal_width", # Y
10 data=iris) #
11 plt.xlabel(" ( )") # X
12 plt.ylabel(" ( )") # Y
13 plt.show() #
```



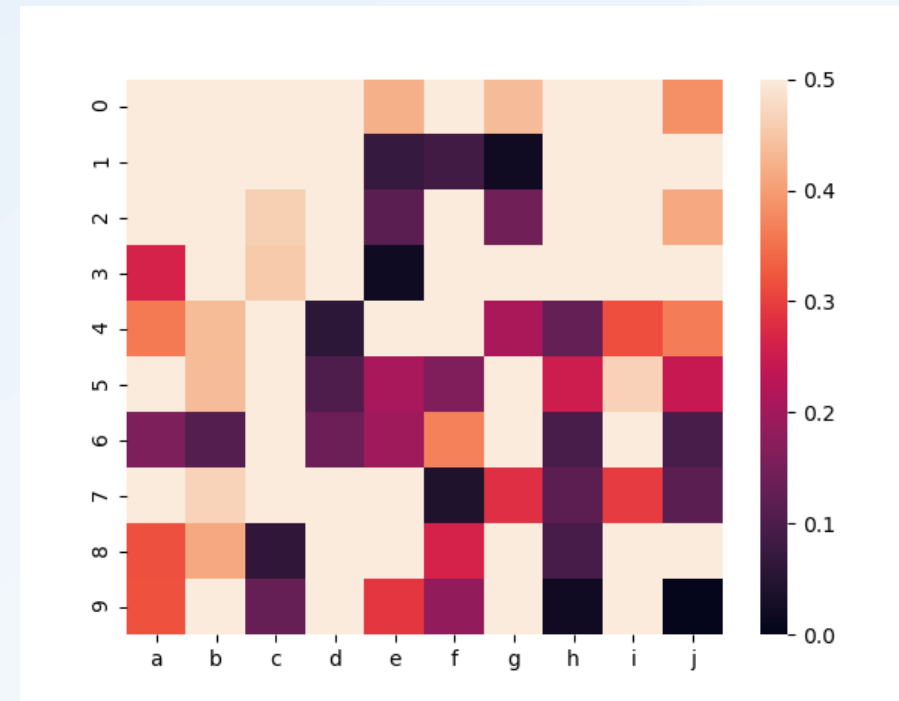
7.



heatmap



clustermap



7.

1 heatmap

◆ heatmap

◆ `seaborn.heatmap(data, vmin=None, vmax=None, cmap=None, center=None, robust=False, annot=None, fmt='.2g', annot_kws=None, linewidths=0, linecolor='white', cbar=True, cbar_kws=None, cbar_ax=None, square=False, xticklabels='auto', yticklabels='auto', mask=None, ax=None)`

➤ data	numpy.array	DataFrame	index	columns
➤ vmin				
➤ vmax				
➤ cmap				
➤ center	cmap	None		

7.

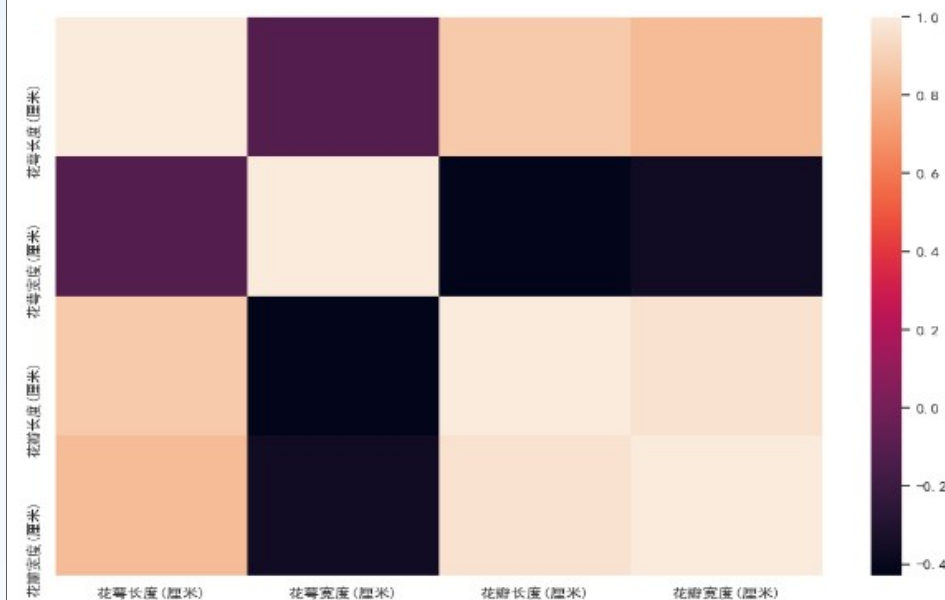
1 heatmap

- annot False
- fmt format
- linewidths
- linecolor
- mask True

7.

1 heatmap

```
1 import pandas as pd # -
2 import seaborn as sns # -
3 import matplotlib.pyplot as plt # -
4 plt.rcParams['axes.unicode_minus'] = False #
5 sns.set_style('darkgrid') # Seaborn
6 sns.set(font='SimHei') #
7 iris=pd.read_csv("E:\\test\\07\\data\\iris.csv") #
8 iris = iris.rename(columns={"sepal_length": "花萼长度(厘米)", #
9 "sepal_width": "花萼宽度(厘米)", #
10 "petal_length": "花瓣长度(厘米)", #
11 "petal_width": "花瓣宽度(厘米)"}) #
12 sns.heatmap(iris.corr()) #
13 plt.show() #
```



7.

1 heatmap



sepal_length	sepal_width	sepal_length	petal_length	petal_width
sepal_width	petal_length	petal_width		

7.

2 clustermap

- ◆ clustermap heatmap
- ◆ `seaborn.clustermap(data,pivot_kws=None, method='average', metric='euclidean', z_score=None,standard_scale=None,figsize=(10,10),cbar_kws=None,row_cluster= True, col_cluster=True,row_linkage=None,col_linkage=None,row_colors=None, col_colors=None, mask=None,dendrogram_ratio=0.2,colors_ratio=0.03,cbar_pos= (0.02, 0.8, 0.05, 0.18), tree_kws=None)`
 - data `numpy.array` `DataFrame` index columns
 - method `scipy.cluster.hierarchy_linkage` “average” “single”
“weighted”“complete”“median”“centroid”



7.

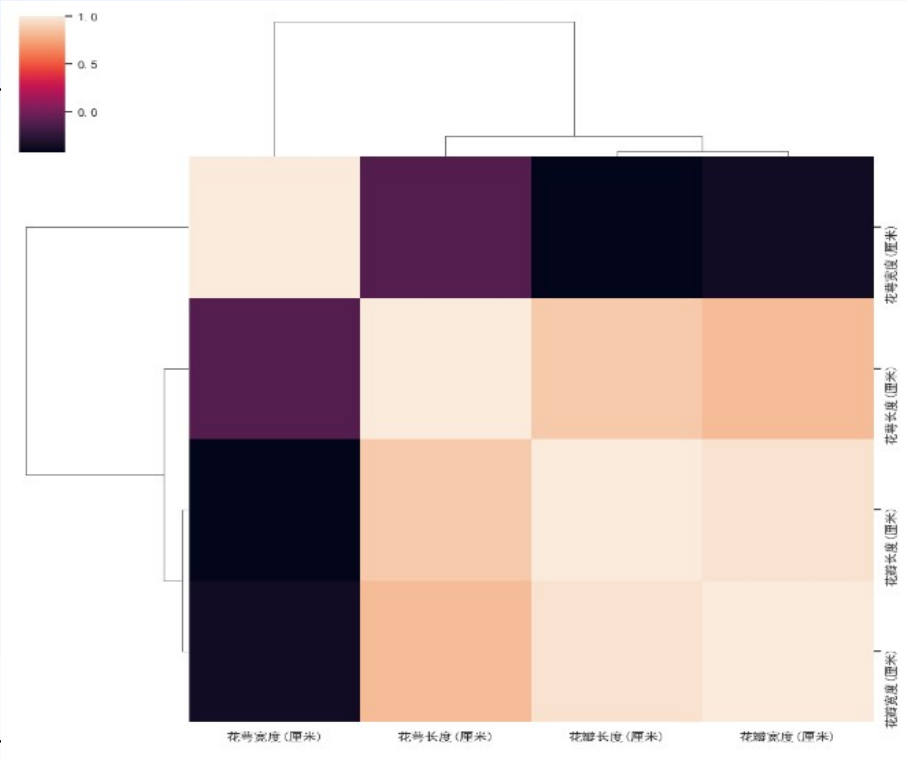
2 clustermmap

- metric
 - “ euclidean”
 - “ braycurtis” “dice” “jaccard”
- ④ z_score
 - z 0 1
 - 0 1
- ⑤ cbar_pos
 - (, , ,)
 - None
- ⑥ mask
 - True clustermmap
 - mask
 - None

7.

2 clustermap

```
1 import pandas as pd # -
2 import seaborn as sns # -
3 import matplotlib.pyplot as plt # -
4 plt.rcParams['axes.unicode_minus'] = False #
5 sns.set_style('darkgrid') # Seaborn
6 sns.set(font='SimHei') #
7 iris=pd.read_csv("E:\\test\\07\\data\\iris.csv") #
8 iris = iris.rename(columns={"sepal_length": " ( )", #
9 "sepal_width": " ( )", #
10 sns.clustermap(iris.corr()) #
11 plt.show() #
```



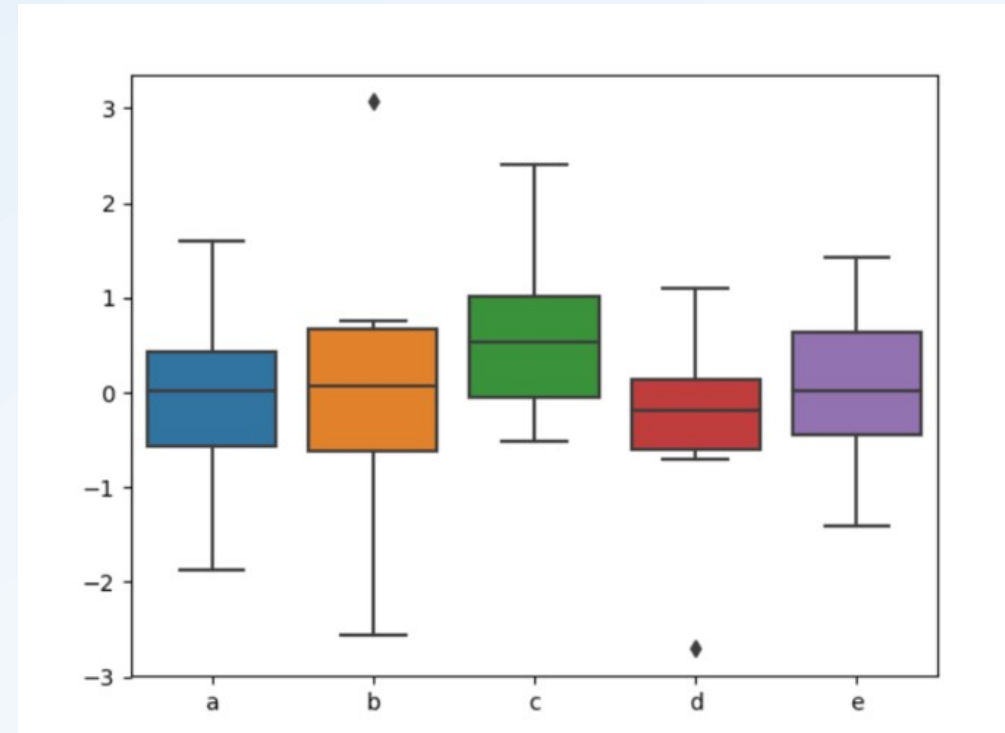
8.



stripplot swarmplot



boxplot boxenplot violinplot



8.

1 -stripplot

◆ Seaborn stripplot swarmplot

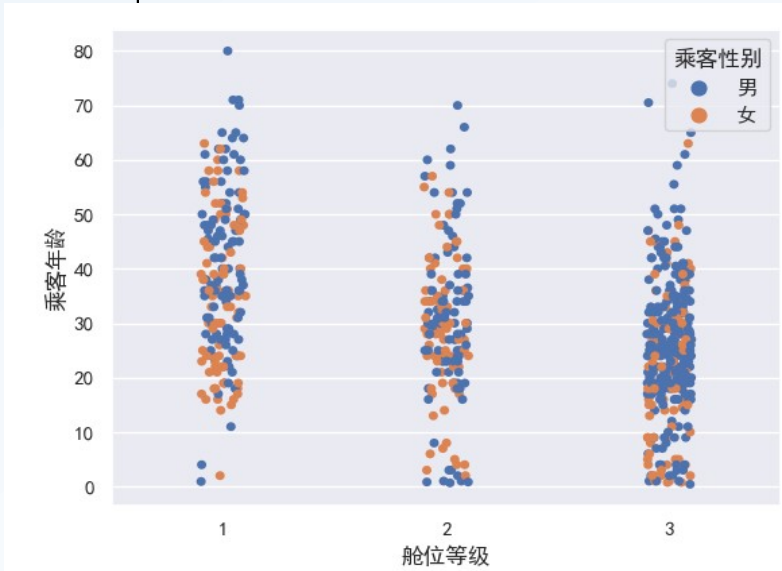
◆ stripplot seaborn.stripplot(x=None,y=None,hue=None,data=None,order=None,
hue_order=None,jitter=True,dodge=False,orient=None,color=None,palette=None,size=5,
edgecolor='gray',linewidth=0,ax=None)

➤ x y data DataFrame x y DataFrame data None
x y DataFrame
➤ data DataFrame None
➤ jitter jitter=True jitter False
➤ dodge
➤ order x

8.

1 -stripplot

```
1 import pandas as pd # -
2 import seaborn as sns # -
3 import matplotlib.pyplot as plt # -
4 sns.set_style('darkgrid') # Seaborn
5 sns.set(font='SimHei') #
6 titanic =pd.read_csv("E:\\test\\07\\data\\ titanic.csv") #
7 titanic['sex'] = [" " if i=="male" else " " for i in titanic['sex']] #
8 titanic['class'] = ['1' if i=="First" else "2" if i=="Second" else "3" #
9 for i in titanic['class']]
10 titanic = titanic.rename(columns={"class":" ", #
11 "age":" ",
12 "sex":" "})
13 sns.stripplot(x=" ", # X
14 y=" ", # Y
15 hue=" ", # hue
16 data=titanic) #
17 plt.show() #
```



8.

1 -swarmplot

◆ `swarmplot` `seaborn.swarmplot(x=None, y=None, hue=None, data=None, order=None, hue_order=None, dodge=False, orient=None, color=None, palette=None, size=5, edgecolor='gray', linewidth=0, ax=None)`

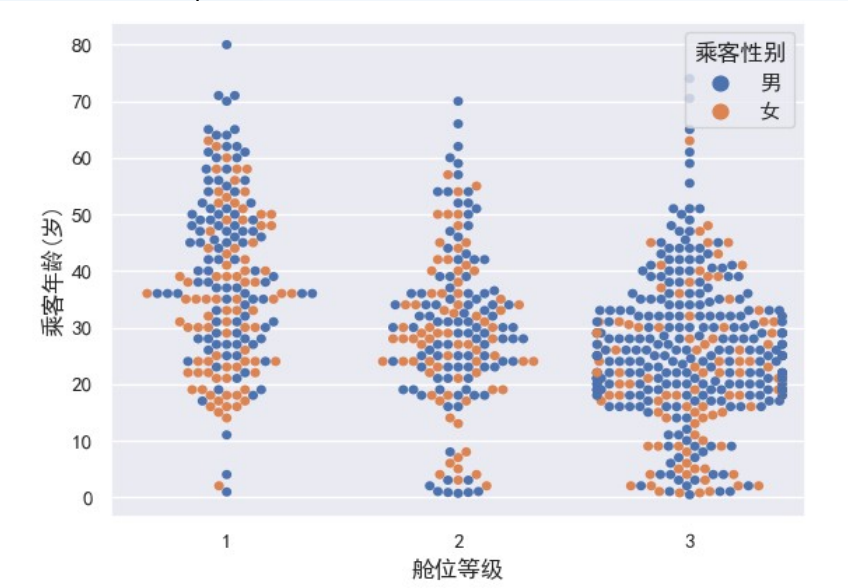
- `x` `y` `data` `DataFrame` `x` `y` `DataFrame` `data`
`None` `x` `y` `DataFrame`
- `data` `DataFrame` `None`
- `jitter` `jitter=True` `jitter` `False`
- `dodge`
- `order` `x`



8.

1 -swarmplot

```
1 import pandas as pd
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4 sns.set_style('darkgrid')
5 sns.set(font='SimHei')
6 titanic = pd.read_csv("E:\\test\\07\\data\\ titanic.csv")
7 titanic['sex'] = [" " if i=="male" else " " for i in titanic['sex']]
8 titanic['class'] = ['1' if i=="First" else "2" if i=="Second" else "3"]
9 for i in titanic['class']]
10 titanic = titanic.rename(columns={"class":" ",
11 "age":" ",
12 "sex":" "})
13 sns.swarmplot(x=" ",
14 y=" ",
15 hue=" ",
16 data=titanic)
17 plt.show()
```



8.

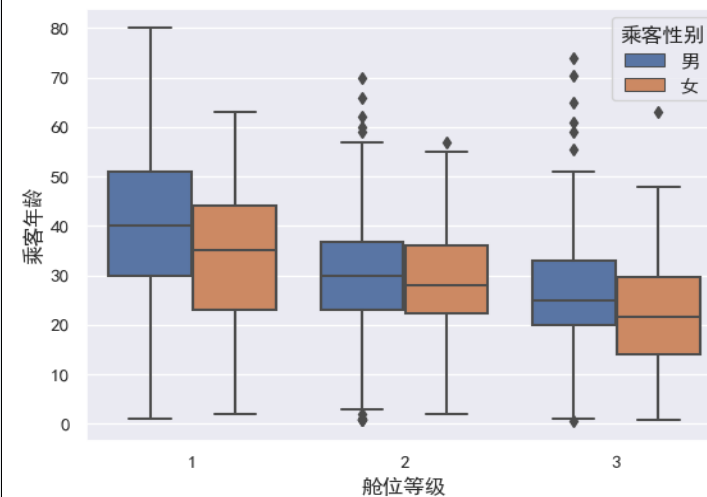
2 -boxplot

- ◆ boxplot Seaborn boxplot
- ◆ `seaborn.boxplot(x=None,y=None,hue=None,data=None,order=None, hue_order=None, orient=None,color=None,palette=None,saturation=0.75,width=0.8, dodge=True,fliersize=5, linewidth=None, whis=1.5, notch=False, ax=None)`
 - `x y` `x y` `X Y`
 - `data` `DataFrame` `None`
 - `hue` `DataFrame`
 - `palette`
 - `order` `hue_order`
 - `orient` `v h` `horizontal` `vertical`

8.

2 -boxplot

```
1 import pandas as pd # -
2 import seaborn as sns # -
3 import matplotlib.pyplot as plt # -
4 sns.set_style('darkgrid') # Seaborn
5 sns.set(font='SimHei') #
6 titanic = pd.read_csv("E:\\test\\07\\data\\ titanic.csv") #
7 titanic['sex'] = [" " if i=="male" else " " for i in titanic['sex']] #
8 titanic['class'] = ['1' if i=="First" else "2" if i=="Second" else "3" #
9 for i in titanic['class']] #
10 titanic = titanic.rename(columns={"class": " ", #
11 "age": " ", #
12 "sex": " "}) #
13 sns.boxplot(x=" ", # X
14 y=" ", # Y
15 hue=" ", # hue
16 data=titanic) #
17 plt.show() #
```



8.

2 **-boxplot**

◆ boxenplot

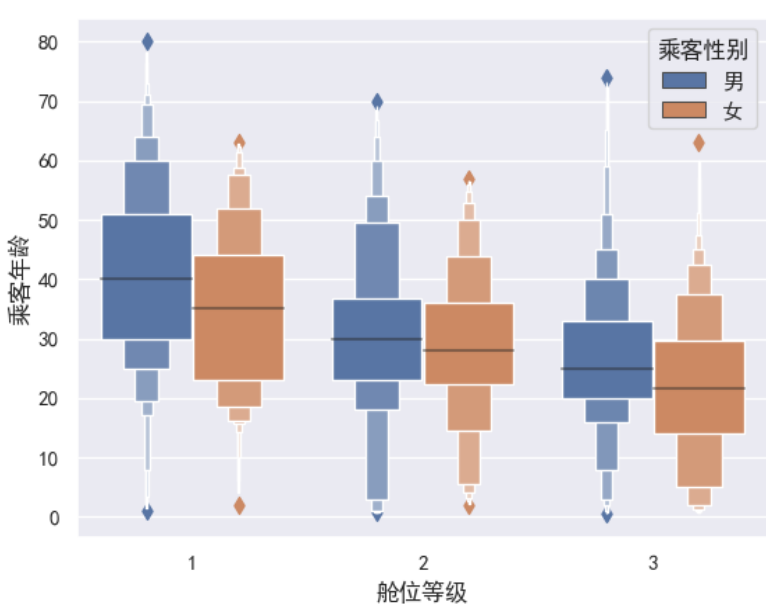
◆ boxenplot boxplot



8.

2 -boxplot

```
1 import pandas as pd
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4 sns.set_style('darkgrid')
5 sns.set(font='SimHei')
6 titanic = pd.read_csv("E:\\test\\07\\data\\ titanic.csv")
7 titanic['sex'] = [" " if i=="male" else " " for i in titanic['sex']]
8 titanic['class'] = ['1' if i=="First" else "2" if i=="Second" else "3"
9 for i in titanic['class']]
10 titanic = titanic.rename(columns={"class": " ",
11 "age": " ",
12 "sex": " "})
13 sns.boxenplot(x=" ",
14 y=" ",
15 hue=" ",
16 data=titanic)
17 plt.show()
```



8.

2 -boxplot

◆ Seaborn violinplot

◆ `seaborn.violinplot(x=None,y=None,hue=None,data=None,order=None,
hue_order=None,bw='scott',cut=2, scale='area', scale_hue=True,gridsize=100, width= 0.8,
inner='box',split=False,dodge=True,orient=None,linewidth=None,color=None, palette=None,
saturation=0.75, ax=None)`

➤ `x y data DataFrame x y DataFrame data`
`None x y DataFrame`
➤ `data DataFrame None`
➤ `bw scott silverman float`



8.

2 -boxplot

- scale area count width area count width
- inner box quartile point stick None box quartiles
point stick None
- split split True
split False

Data visualization



8.

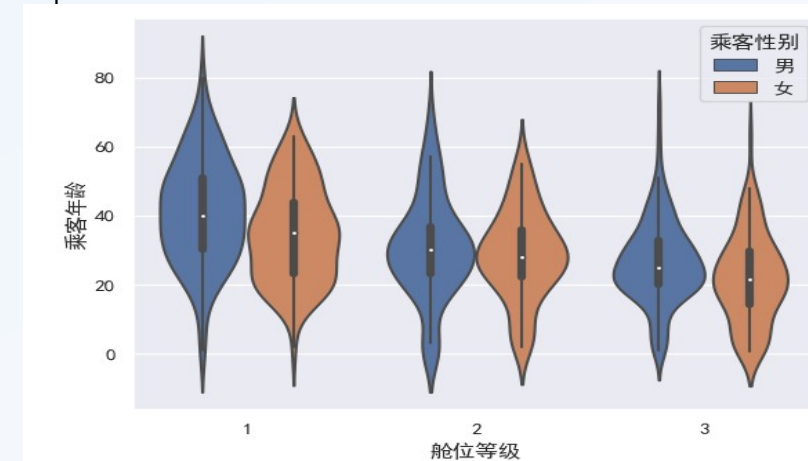
2 -boxplot

```
1 import pandas as pd
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4 sns.set_style('darkgrid')
5 sns.set(font='SimHei')
6 titanic = pd.read_csv("E:\\test\\07\\data\\ titanic.csv")
7 titanic['sex'] = [" " if i=="male" else " " for i in titanic['sex']]
8 titanic['class'] = ['1' if i=="First" else "2" if i=="Second" else "3"
9 for i in titanic['class']]
10 titanic = titanic.rename(columns={"class": " ",
11 "age": " ",
12 "sex": " "})
13 sns.violinplot(x=" ",
14 y=" ",
15 hue=" ",
16 data=titanic)
17 plt.show()
```

-
-
-
Seaborn

X
Y
hue

#





2023

